СОДЕРЖАНИЕ

ВВЕДЕНИЕ 2
1 Постановка задачи 3
1.1 Анализ предметной области 3
1.2 Формулировка функционала 5
2 Разработка классов 6
2.1 Классы сущностей 6
2.2 Управляющие классы 12
2.3 Интерфейсные классы 18
2.4 Используемые паттерны проектирования 22
2.5 Диаграмма классов 29
3 Разработка приложения 31
3.1 Разработка интерфейса приложения 31
3.2 Реализация методов классов 37
3.3 Разработка тестового приложения 43
4 Тестирование 44
4.1 Организация проверки 44
4.2 Наглядные результаты 45
4.3 Контроль правил и границы проверки 50
ЗАКЛЮЧЕНИЕ 52
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 53
ПРИЛОЖЕНИЕ А.
ИСХОДНЫЙ КОД ПРИЛОЖЕНИЯ 55

ВВЕДЕНИЕ

Библиотека обслуживает читателей и хранит сведения о книгах, журналах и других изданиях. Во время работы необходимо находить нужное издание, учитывать свободные экземпляры, оформлять выдачу и следить за возвратом. Если сведения о читателях, фонде и выдачах ведутся раздельно, становится сложнее проверить наличие книги и восстановить историю её использования. Поэтому автоматизация библиотечного учёта представляет практический интерес.

Курсовой проект будет посвящён разработке настольного приложения для организации взаимодействия объектов библиотечной информационной системы. Цель работы заключается в создании программы, которая позволит регистрировать читателей и издания, выполнять поиск, оформлять выдачу и возврат и сохранять сведения между запусками. Для этого потребуется описать предметную область, выделить классы и их связи, выбрать подходящие шаблоны проектирования, реализовать графический интерфейс и проверить основные операции.

Актуальность работы связана с необходимостью согласованного учёта изданий и их выдач. В первой главе будут рассмотрены задачи библиотеки и необходимые функции программы. Далее предполагается описать классы и способы их взаимодействия, затем рассмотреть реализацию приложения и результаты тестирования. Такой порядок позволит связать библиотечные задачи с устройством программы и оценить достижение поставленной цели.

1 Постановка задачи

1.1 Анализ предметной области

Библиотека хранит книги и периодические издания, ведёт сведения о читателях и учитывает выдачу и возврат изданий. Библиотекарю необходимо знать, какие издания есть в фонде, сколько экземпляров доступно и кому они выданы. Для решения этих задач в курсовом проекте предусматривается настольное приложение.

Цель проекта состоит в разработке программы, которая поможет библиотекарю регистрировать читателей, находить нужные издания и оформлять их выдачу и возврат. Основным пользователем программы будет библиотекарь. Для работы с программой ему не потребуется вводить команды: необходимые действия выполняются с помощью полей ввода, списков и кнопок.

В проекте рассматриваются сведения о читателях, библиотекарях, книгах, периодических изданиях, филиалах и выдачах. Под заказом книги в выбранном варианте понимается обращение читателя к библиотекарю: сотрудник находит нужное издание, проверяет наличие и оформляет выдачу. Отдельная очередь бронирования не предусматривается. Сведения о наличии и месте хранения показываются в списке изданий; выгрузка отчётов в отдельный файл, денежные штрафы и электронная доставка книг не входят в выбранный объём работы. Основные понятия предметной области имеют следующий смысл:

— читатель: зарегистрированный пользователь, которому могут выдаваться издания;
— библиотекарь: сотрудник, оформляющий выдачу и связанный с филиалом;
— издание: книга, журнал или другое периодическое издание, сведения о котором учитываются в фонде;
— филиал: место хранения фонда;
— выдача: запись о том, кто получил издание, кто его выдал и когда его необходимо вернуть.

В таблице 1 приведён словарь предметной области. В нём для каждого понятия указаны сведения, которые нужно хранить, и действия, которые с ним связаны.

Таблица 1 — Словарь предметной области

Понятие	Основные свойства	Связанные действия
Читатель	имя, год рождения, адрес, организация	зарегистрировать, найти, просмотреть, удалить
Библиотекарь	имя, должность, филиал	найти, оформить выдачу, принять возврат
Издание	название, авторы, издатель, год, остаток	добавить, найти, просмотреть, выдать, вернуть
Книга	общие сведения об издании, ISBN	учесть, найти по названию или автору
Периодическое издание	общие сведения об издании, номер выпуска	учесть, найти, выдать
Филиал	название, адрес	указать место хранения, выбрать при учёте издания
Выдача	читатель, издание, библиотекарь, даты, состояние	оформить, просмотреть, отметить просрочку, завершить возвратом

Выделенные понятия будут представлены классами программы. Класс описывает сведения об объекте одного вида: например, о читателе или о книге. Отдельный объект класса содержит сведения о конкретном читателе или конкретном издании. Действия библиотекаря определяют необходимые операции: зарегистрировать читателя, добавить издание, найти запись, выдать издание и отметить его возврат.

Один читатель может получить несколько изданий. В свою очередь, одно издание может выдаваться разным читателям: после возврата или одновременно, если в библиотеке есть несколько экземпляров. Так возникает связь «многие ко многим»: с каждой стороны может быть не один, а несколько участников. Каждая отдельная выдача учитывается собственной записью. В ней указывается один читатель, одно издание и один библиотекарь, который оформил выдачу.

Приложение будет работать на компьютере и открываться в отдельном окне. Разработка сайта и доступ через браузер не предусматриваются.

Например, читатель получает книгу и журнал. В программе для него создаются две записи о выдаче. Если другой читатель получает ещё один экземпляр той же книги, создаётся третья запись, уже с его данными. При этом каждая физическая книга не выделяется в отдельный класс: для издания хранятся общее количество экземпляров и количество свободных экземпляров. Выдать издание можно только при наличии свободного экземпляра.

1.2 Формулировка функционала

В таблице 2 перечислены действия, которые планируется предоставить библиотекарю. Программа должна помогать вести сведения о читателях и фонде, а также сохранять историю выдачи и возврата изданий.

Таблица 2 — Функциональные возможности приложения

Группа	Планируемые возможности
Читатели	регистрация читателя, поиск по имени, просмотр сведений и удаление записи; проверка заполнения полей
Издания	создание книг и периодических изданий, поиск по названию или автору, отображение остатка и удаление; выбор филиала
Выдачи	выбор читателя и издания; проверка наличия свободного экземпляра и допустимого количества выдач; оформление выдачи и возврата; просмотр сроков
Проверка данных	запрет удаления читателя или издания, если с ними связаны выдачи; недопущение повторного увеличения остатка при возврате
Учебный пример	загрузка связанного демонстрационного набора; отдельное добавление тестовых книг и читателей без повторов; очистка с подтверждением
Хранение	сохранение сведений в локальной базе данных и их чтение при следующем запуске программы

2 Разработка классов

2.1 Классы сущностей

На основе анализа предметной области выделены шесть классов: читатель, библиотекарь, книга, периодическое издание, филиал и выдача. Далее рядом с русским названием указывается имя класса в исходном коде. Это позволяет сопоставить описание с диаграммами и программой, не подменяя предметные понятия английскими словами.

Читатель (Reader) хранит сведения о зарегистрированном посетителе, а библиотекарь (Librarian) — сведения о сотруднике. У обоих есть имя и служебный идентификатор, поэтому общие свойства описаны в интерфейсе Person. Интерфейс задаёт, какие свойства должен предоставлять класс, но сам не является записью о конкретном человеке. Аналогично интерфейс Publication задаёт общие свойства книги (Book) и периодического издания (Periodical). Филиал описывается классом LibraryBranch, а запись о выдаче — классом Loan.

Общие свойства и связи классов показаны на рисунке 1. Сплошная линия обозначает связь объектов. Стрелка с пустым треугольником направлена к общему типу: например, книга и периодическое издание предоставляют свойства издания. Ниже приведены поля всех классов, в том числе общие поля, которые они получают через интерфейсы.

Поле хранит отдельное свойство объекта, а метод задаёт действие, которое можно выполнить. В таблицах String означает текст, Int — целое число, Long — целое число с большим диапазоном значений, Boolean — логическое значение «да» или «нет». Знак вопроса после типа означает, что значение может отсутствовать; например, для книги можно не указать ISBN. Запись List<T> обозначает список объектов типа T.

Идентификатор — служебное значение, по которому программа различает записи. Он хранится в поле id и создаётся автоматически. Пользователь выбирает читателя по имени и издание по названию, а не вводит их идентификаторы. Общий интерфейс LibraryEntity задаёт это поле для всех основных объектов.

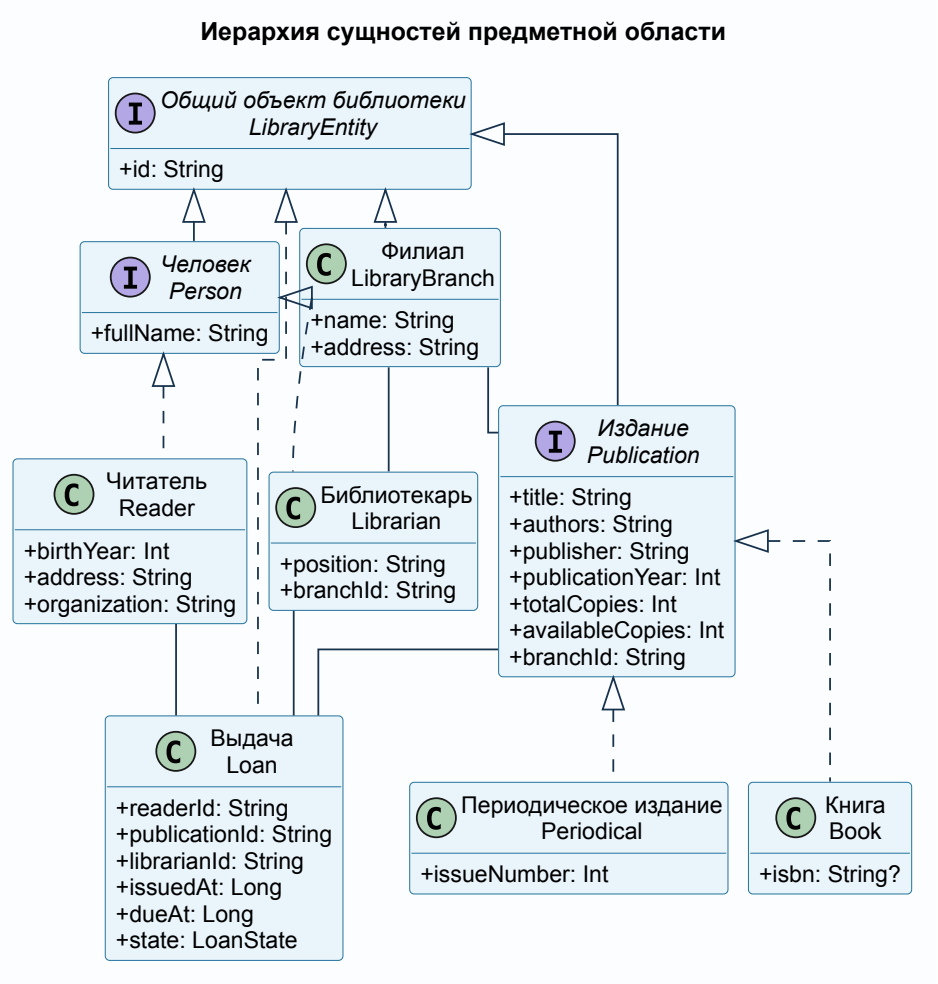


Рисунок 1 — Иерархия классов сущностей

2.1.1 Общий объект библиотеки (LibraryEntity)

Поля типа «Общий объект библиотеки» (LibraryEntity) приведены в таблице 3.

Таблица 3 — Поля типа «Общий объект библиотеки»

Поле	Тип	Назначение поля
id	String	Служебный идентификатор объекта

Интерфейс задаёт только общее поле id. Собственных методов у него нет.

2.1.2 Человек (Person)

Поля типа «Человек» (Person) приведены в таблице 4.

Таблица 4 — Поля типа «Человек»

Поле	Тип	Назначение поля
id	String	Идентификатор человека, унаследованный от LibraryEntity

Поле	Тип	Назначение поля
fullName	String	Фамилия и имя человека

Интерфейс задаёт общие сведения о человеке: идентификатор и имя. Эти свойства есть и у читателя, и у библиотекаря. Собственных методов у интерфейса нет.

2.1.3 Читатель (Reader)

Поля типа «Читатель» (Reader) приведены в таблице 5.

Таблица 5 — Поля типа «Читатель»

Поле	Тип	Назначение поля
id	String	Идентификатор читателя
fullName	String	Фамилия и имя читателя
birthYear	Int	Год рождения
address	String	Адрес читателя
organization	String	Организация читателя

При создании записи читателя проверяется правильность введённых сведений. После создания её поля не меняются напрямую: для исправления сведений используется копия с нужными значениями. Метод copy создаётся языком Kotlin автоматически. Класс читателя хранит данные, а поиск, сохранение и удаление выполняются отдельными управляющими классами.

2.1.4 Библиотекарь (Librarian)

Поля типа «Библиотекарь» (Librarian) приведены в таблице 6.

Таблица 6 — Поля типа «Библиотекарь»

Поле	Тип	Назначение поля
id	String	Идентификатор библиотекаря
fullName	String	Фамилия и имя сотрудника
position	String	Должность
branchId	String	Филиал, к которому относится сотрудник

Запись библиотекаря хранит сведения о сотруднике и его филиале. Собственных методов оформления выдачи в этом классе нет: выдача требует одновременно

проверить читателя, издание и правила библиотеки, поэтому её выполняет отдельный управляющий класс. Для создания изменённой копии записи предусмотрен автоматически создаваемый метод copy.

2.1.5 Издание (Publication)

Поля типа «Издание» (Publication) приведены в таблице 7.

Таблица 7 — Поля типа «Издание»

Поле	Тип	Назначение поля
id	String	Идентификатор издания
title	String	Название издания
authors	String	Авторы книги или редакционная коллегия периодики
publisher	String	Наименование издателя
publicationYear	Int	Год публикации
totalCopies	Int	Общее количество экземпляров
availableCopies	Int	Число экземпляров, доступных для выдачи
branchId	String	Идентификатор филиала хранения

Интерфейс содержит сведения, общие для книги и периодического издания. Он позволяет обращаться к обоим видам изданий одинаково, например получать название или количество свободных экземпляров. Собственных методов в интерфейсе нет.

2.1.6 Книга (Book)

Поля типа «Книга» (Book) приведены в таблице 8.

Таблица 8 — Поля типа «Книга»

Поле	Тип	Назначение поля
id	String	Идентификатор издания
title	String	Название издания
authors	String	Авторы книги или редакционная коллегия периодики
publisher	String	Наименование издателя
publicationYear	Int	Год публикации
totalCopies	Int	Общее количество экземпляров
availableCopies	Int	Число экземпляров, доступных для выдачи
branchId	String	Идентификатор филиала хранения

Поле	Тип	Назначение поля
isbn	String?	ISBN книги; допускается отсутствие

Книга хранит общие сведения об издании и ISBN — международный номер книги. Если номер неизвестен, поле isbn остаётся пустым. Правильность сведений проверяется при создании объекта. Для изменения данных создаётся копия методом copy; поиск, сохранение и выдача выполняются управляющими классами.

2.1.7 Периодическое издание (Periodical)

Поля типа «Периодическое издание» (Periodical) приведены в таблице 9.

Таблица 9 — Поля типа «Периодическое издание»

Поле	Тип	Назначение поля
id	String	Идентификатор издания
title	String	Название издания
authors	String	Авторы книги или редакционная коллегия периодики
publisher	String	Наименование издателя
publicationYear	Int	Год публикации
totalCopies	Int	Общее количество экземпляров
availableCopies	Int	Число экземпляров, доступных для выдачи
branchId	String	Идентификатор филиала хранения
issueNumber	Int	Положительный номер выпуска

Периодическое издание отличается от книги наличием номера выпуска. Например, несколько номеров одного журнала учитываются как разные издания. Номер выпуска должен быть положительным. Метод copy позволяет получить копию записи с исправленными сведениями; поиск, сохранение и выдача выполняются управляющими классами.

2.1.8 Филиал (LibraryBranch)

Поля типа «Филиал» (LibraryBranch) приведены в таблице 10.

Таблица 10 — Поля типа «Филиал»

Поле	Тип	Назначение поля
id	String	Идентификатор филиала
name	String	Название филиала
address	String	Адрес места хранения фонда

Филиал хранит название и адрес места, где находится фонд. Ссылка на филиал позволяет определить место хранения издания и место работы библиотекаря. Собственных прикладных методов у класса нет; метод copy позволяет получить изменённую копию записи.

2.1.9 Выдача (Loan)

Поля типа «Выдача» (Loan) приведены в таблице 11.

Таблица 11 — Поля типа «Выдача»

Поле	Тип	Назначение поля
id	String	Идентификатор события выдачи
readerId	String	Ссылка на читателя
publicationId	String	Ссылка на издание
librarianId	String	Ссылка на оформившего выдачу сотрудника
issuedAt	Long	Время выдачи, представленное числом миллисекунд
dueAt	Long	Срок возврата, представленный числом миллисекунд
state	LoanState	Текущее состояние выдачи

Выдача связывает сведения о читателе, издании и библиотекаре. Поля readerId, publicationId и librarianId содержат их служебные идентификаторы. Поле issuedAt хранит время выдачи, dueAt — срок возврата. Время представлено числом миллисекунд, прошедших с 1 января 1970 года; такой способ позволяет сравнивать даты как числа. Метод copy позволяет создать копию записи, а оформление выдачи и возврата выполняется управляющим классом.

У выдачи предусмотрено три состояния: издание ещё находится у читателя (Active), срок возврата истёк (Overdue), издание возвращено (Returned). Они описаны

общим типом LoanState. Состояние возвращённого издания дополнительно хранит время возврата. Метод returnAt переводит незавершённую выдачу в состояние «возвращено». Повторно выполнять этот переход нельзя. Правила выдачи описаны интерфейсом LoanPolicy: он задаёт проверку допустимости выдачи и определение срока возврата.

Для данных предусмотрены ограничения. Обязательные текстовые поля нельзя оставлять пустыми. Год рождения допускается в диапазоне 1800–2100, год издания — 1450–2100. Общее число экземпляров должно быть положительным, а количество свободных экземпляров — не меньше нуля и не больше общего. Номер выпуска должен быть положительным, срок возврата не может предшествовать времени выдачи. Эти границы выбраны для учебного проекта и не являются обязательными правилами настоящей библиотеки.

2.2 Управляющие классы

Классы сущностей хранят сведения, но сами по себе не организуют обслуживание читателя. Для выполнения действий библиотекаря нужны управляющие классы. Например, при выдаче требуется проверить читателя, наличие экземпляра и срок возврата, затем сохранить запись и уменьшить количество свободных экземпляров. Распределение этих обязанностей следующее:

— LibraryService описывает основные действия: выдать и вернуть издание, удалить запись читателя или издания, загрузить учебный пример и очистить данные. Класс RoomLibraryService выполняет эти действия. Он обращается к хранилищу, проверяет правила выдачи и согласованно изменяет связанные записи.

— LibraryRepositoryBundle объединяет средства чтения и сохранения читателей, сотрудников, филиалов, изданий и выдач. Его реализация RoomRepositoryBundle работает с выбранной базой данных. Управляющие классы получают обычные объекты, а не записи таблиц базы.

— LibraryEntityFactory создаёт объекты читателей, сотрудников, филиалов, книг, периодических изданий и выдач. Этот класс получает необходимые сведения и создаёт служебный идентификатор. Так одинаковые действия при создании объектов собраны в одном месте.

— ReadersViewModel, PublicationsViewModel, LoansViewModel, AdministrationViewModel и TestDataViewModel обрабатывают действия на соответствующих экранах. Они подготавливают списки, проверяют введённые значения и сообщают об ошибках. Пока экран выдач открыт, LoansViewModel раз в минуту проверяет сроки, чтобы просроченная выдача не оставалась отмеченной как своевременная.

— ApplicationContainer создаёт и связывает объекты, необходимые приложению: базу данных, средства работы с записями и управляющий класс. При закрытии приложения он освобождает занятые ресурсы.

2.2.1 Поля и методы управляющих классов

В таблице 12 приведены поля вспомогательных и управляющих типов. Здесь хранится не только информация о библиотеке, но и объекты, необходимые для выполнения действий. Например, поле ids содержит генератор идентификаторов, а поле policy — объект с правилами выдачи. Если собственных полей нет, тип определяет только набор методов, которые должен предоставить реализующий его класс.

Таблица 12 — Поля управляющих типов

Тип	Поля и типы	Назначение
IdGenerator, UuidId Generator	собственных полей нет	правило создания идентификатора и его реализация
LibraryEntityFactory	ids: IdGenerator	объект для получения новых идентификаторов
LoanPolicy	maxActiveLoansPerReader: Int	верхняя граница незавершённых выдач читателю
DefaultLoanPolicy	maxActiveLoansPerReader: Int; bookLoanDays: Long; periodicalLoanDays: Long	лимит выдач и длительность выдачи книги и периодики в днях
PolicyDecision	isAllowed: Boolean; reason: String?	разрешение операции и причина отказа

Тип	Поля и типы	Назначение
LoanState	status: String	машинное имя состояния выдачи
Active, Overdue	status: String	значения ACTIVE и OVERDUE
Returned	status: String; returnedAt: Long	значение RETURNED и время возврата
LibraryService	собственных полей нет	набор основных действий библиотеки
RoomLibraryService	repositories: LibraryRepositoryBundle; policy: LoanPolicy; ids: IdGenerator	доступ к данным, правила выдачи и создание идентификаторов
DemoData	branch: LibraryBranch; librarian: Librarian; readers: List<Reader>; publications: List<Publication>; loan: Loan	связанный демонстрационный набор
ClearAllResult	loans, publications, readers, librarians, branches: Int; totalDeleted: Int	число удалённых записей по таблицам; totalDeleted вычисляется как их сумма

В учебном варианте один читатель может иметь не более пяти невозвращённых изданий. Для книги предусмотрен срок 21 день, для периодического издания — 7 дней. Эти значения хранятся в DefaultLoanPolicy. Просроченные, но ещё не возвращённые издания также учитываются в ограничении. Такие настройки выбраны для примера; при необходимости правила можно заменить.

В таблице 13 перечислены методы и их назначение. Например, nextId создаёт идентификатор, canIssue проверяет возможность выдачи, а dueAt определяет дату, до которой издание нужно вернуть.

Таблица 13 — Методы управляющих типов

Тип	Методы	Назначение
IdGenerator, UuidIdGenerator	nextId(): String	получить новый идентификатор
LibraryEntityFactory	reader, librarian, branch, book, periodical, loan	создать соответствующий объект с новым идентификатором

Тип	Методы	Назначение
LoanPolicy, Default LoanPolicy	canIssue(reader, publication, activeLoansForReader): Policy Decision; dueAt(issuedAt, publication): Long	проверить допустимость выдачи и определить срок
Active, Overdue	toString(): String	вернуть строковое имя состояния
LoanState	returnAt(timestamp: Long): LoanState	функция расширения: переход к Returned; повторный переход запрещён
LibraryService, Room LibraryService	issueLoan, returnLoan, delete Reader, deletePublication, seed Demo, clearAll	оформить выдачу/возврат, защищённое удаление, загрузку примера и очистку

Метод выдачи issueLoan получает идентификаторы читателя, издания и библиотекаря (readerId, publicationId, librarianId типа String), а также текущее время now типа Long. Результат — запись о выдаче типа Loan. Метод возврата returnLoan получает идентификатор выдачи loanId типа String и время now типа Long; он возвращает обновлённую запись Loan. Методы deleteReader и deletePublication получают идентификатор удаляемой записи типа String. Тип результата Unit означает отсутствие отдельного возвращаемого значения. Метод seedDemo возвращает Boolean, то есть признак добавления учебных данных, а clearAll возвращает ClearAllResult — количество удалённых записей. Обозначение suspend у этих методов позволяет выполнять работу с данными без блокирования обычного выполнения программы.

Методы фабрики получают сведения для создаваемого объекта, но не требуют готового id. Для читателя метод reader принимает fullName, address, organization типа String и birthYear типа Int. Для библиотекаря метод librarian принимает fullName, position, branchId типа String. Для филиала метод branch принимает name и address типа String. Метод book принимает title, authors, publisher, branchId типа String, publicationYear, totalCopies типа Int и isbn типа String?. Метод periodical принимает title, authors, publisher, branchId типа String и publicationYear, issueNumber, totalCopies типа Int. Метод loan получает три идентификатора типа String и время issuedAt, dueAt типа Long. Каждый метод возвращает объект соответствующего класса.

2.2.2 Классы работы с данными

Для чтения и сохранения сведений используются репозитории — классы, через которые остальные части программы обращаются к данным. ReaderRepository работает с читателями, LibrarianRepository — с библиотекарями, BranchRepository — с филиалами, PublicationRepository — с изданиями, LoanRepository — с выдачами. Их свойства readers, librarians, branches, publications и loans предоставляют списки объектов. Тип Flow<List<T>> означает, что программа получает не только список, но и его последующие изменения.

Метод findById(id: String) ищет объект по идентификатору и возвращает его либо пустое значение, если запись не найдена. Метод save сохраняет объект, delete удаляет его по идентификатору, если такое действие предусмотрено. Методы readersOnce и publicationsOnce читают списки один раз, без последующего получения изменений. У LoanRepository дополнительно есть issue(loan): Boolean и вариант issue(loan, maxActiveLoansPerReader) для выдачи с проверкой ограничения, returnLoan(id, returnedAt): Loan? для возврата, activeCountForReader(readerId): Int для подсчёта незавершённых выдач и refreshOverdue(now): Int для обновления просроченных записей. Эти методы имеют обозначение suspend.

LibraryAdministrationRepository задаёт два действия: insertDemo(data: DemoData): Boolean добавляет связанные учебные записи, clearAll(): ClearAllResult очищает данные. LibraryRepositoryBundle также предоставляет эти действия и содержит поля readers: ReaderRepository, librarians: LibrarianRepository, branches: BranchRepository, publications: PublicationRepository и loans: LoanRepository. Это позволяет передавать необходимые средства работы с данными одним объектом.

В реализации RoomRepositoryBundle поля database: LibraryDatabase и dao: LibraryDao обеспечивают доступ к базе. Класс хранит пять соответствующих репозиториев и выполняет оба действия управления данными. Вспомогательный метод insertDemoData получает записи филиала BranchRecord, сотрудника LibrarianRecord,

списки читателей List<ReaderRecord> и изданий List<PublicationRecord>, а также выдачу LoanRecord. Результат Boolean показывает, были ли данные добавлены.

Классы RoomReaderRepository, RoomLibrarianRepository, RoomBranchRepository, RoomPublicationRepository и RoomLoanRepository выполняют перечисленные операции в базе данных. У каждого есть поле dao: LibraryDao для обращения к базе и свойство со списком соответствующих объектов. При чтении запись таблицы преобразуется в объект, при сохранении выполняется обратное преобразование. ActiveLoanDeletionException обозначает ошибку удаления связанных записей, InventoryException — ошибку учёта экземпляров. Собственных дополнительных полей у этих ошибок нет.

Для хранения используются отдельные классы записей. ReaderRecord повторяет пять полей читателя, LibrarianRecord — четыре поля библиотекаря, BranchRecord — три поля филиала. PublicationRecord содержит общие поля издания и дополнительно kind: String для вида издания, isbn: String? для номера книги и issueNumber: Int? для номера выпуска. LoanRecord содержит id, readerId, publicationId, librarianId типа String, issuedAt, dueAt типа Long, returnedAt: Long? для времени возврата и status: String для названия состояния. ClearAllCounts содержит пять полей Int с количеством записей в таблицах. Классы записей хранят данные и не имеют собственных методов обслуживания читателей.

LibraryDatabase представляет базу данных; собственных полей у класса нет. Метод libraryDao(): LibraryDao предоставляет объект для чтения и записи. LibraryDatabaseConstructor содержит initialize(): LibraryDatabase для создания базы. LibraryDao — интерфейс без полей. Его методы observeReaders, observeLibrarians, observeBranches, observePublications и observeLoans возвращают списки записей и сообщают об их изменениях: Flow<List<Record>>. Здесь Record обозначает соответствующий класс записи. Методы get, save и count предназначены для чтения, сохранения и подсчёта записей.

Методы issueLoanAtomically, returnLoanAtomically, deleteReaderIfUnused, deletePublicationIfUnused, insertDemoData и clearAllData выполняют действия, затрагивающие несколько связанных записей. Такие действия объединяются в транзакцию: либо сохраняются все изменения, либо не сохраняется ни одно из них. Например, нельзя уменьшить число свободных экземпляров, но не сохранить саму выдачу.

2.3 Интерфейсные классы

Графический интерфейс состоит из отдельных экранных компонентов. Компонент — часть окна, например список читателей или форма добавления издания. Для их построения используется Compose Desktop. Функция main из файла main.kt создаёт ApplicationContainer и открывает окно Window. Функция App подготавливает объекты управления экранами и передаёт их в LibraryScreen. Внутри находятся ReadersPane для читателей, PublicationsPane для изданий, LoansPane для выдач и ArchitecturePane для просмотра устройства программы и учебных действий. Загрузка учебных записей запускается отдельной кнопкой, а не при каждом открытии окна.

Экранные компоненты показывают данные и передают действия пользователя управляющим объектам. Они не обращаются к таблицам базы напрямую и не меняют количество экземпляров самостоятельно. Тип CollectionUiState хранит сведения, необходимые для показа списка: элементы, признак загрузки и сообщение об ошибке. При выводе выдач служебные идентификаторы заменяются понятными пользователю именами читателей и названиями изданий.

В таблице 14 приведены поля классов, которые подготавливают данные для экрана. Название ViewModel обозначает объект управления состоянием экрана. StateFlow хранит текущее состояние и сообщает о его изменениях; MutableStateFlow позволяет это состояние менять. CoroutineScope определяет, где выполняются задачи чтения и сохранения. Эти служебные поля нужны для работы окна и не являются сведениями о читателе или книге.

Таблица 14 — Поля интерфейсного слоя

Тип	Поля и типы	Смысл
CollectionUiState<T>	items: List<T>; isLoading: Boolean; error: String?	элементы списка, признак загрузки и ошибка
ReadersViewModel	repository: ReaderRepository; ids: IdGenerator; service: LibraryService?; scope: CoroutineScope; errors: MutableStateFlow<String?>; searchQuery: MutableStateFlow<String>; query: StateFlow<String>; state: StateFlow<CollectionUiState<Reader>>	доступ к читателям, зависимости, состояние поиска, списка и ошибок
PublicationsViewModel	repository: PublicationRepository; branchesRepository: BranchRepository; ids: IdGenerator; service: LibraryService?; scope: CoroutineScope; errors: MutableStateFlow<String?>; searchQuery: MutableStateFlow<String>; query: StateFlow<String>; state: StateFlow<CollectionUiState<Publication>>; branches: StateFlow<CollectionUiState<LibraryBranch>>	доступ к изданиям и филиалам, зависимости и состояния экрана
LoanRow	loan: Loan; readerName: String; publicationTitle: String	выдача с пользовательскими именами вместо внутренних идентификаторов
LoansViewModel	service: LibraryService; loansRepository: LoanRepository; readersRepository: ReaderRepository; publications Repository: PublicationRepository; librarians Repository: LibrarianRepository; scope: CoroutineScope; overdueRefreshIntervalMillis: Long; currentTimeMillis: () -> Long; operationErrors, refreshErrors: MutableStateFlow<String?>; state: StateFlow<CollectionUiState<LoanRow>>	сценарии выдачи, источники данных, период обновления, часы и раздельные ошибки
AdministrationUiState	isBusy: Boolean; message: String?; error: String?	признак операции, результат и ошибка
AdministrationViewModel	service: LibraryService; scope: CoroutineScope; _state: MutableStateFlow<AdministrationUiState>; state: StateFlow<AdministrationUiState>	зависимости и изменяемое/наблюдаемое состояние администрирования

Тип	Поля и типы	Смысл
ApplicationContainer	closeables: LinkedHashSet<AutoCloseable>; closed: Boolean; repositories: RoomRepositoryBundle; service: RoomLibraryService	ресурсы приложения, признак закрытия и зависимости верхнего уровня

ReadersViewModel содержит методы setSearchQuery(value: String) для изменения текста поиска, add(fullName: String, birthYear: String, address: String, organization: String): Boolean для добавления читателя, delete(reader: Reader) для удаления и close() для завершения работы с экраном. PublicationsViewModel содержит setSearchQuery(value: String), addBook(title, authors, publisher, year, copies, branchId, isbn: String), addPeriodical(title, editors, publisher, year, issue, copies, branchId: String), delete(publication: Publication) и close(). Методы addBook и addPeriodical добавляют книгу и периодическое издание. Все их входные значения имеют тип String, поскольку получаются из полей ввода; год, номер выпуска и число экземпляров затем преобразуются в числа и проверяются.

LoansViewModel содержит issue(reader: Reader?, publication: Publication?) для выдачи, returnLoan(loan: Loan) для возврата и close() для завершения работы. Знак вопроса у reader и publication учитывает случай, когда пользователь ещё не выбрал читателя или издание. AdministrationViewModel содержит seedDemo() для загрузки учебного примера, clearAll() для очистки, close() для завершения работы и внутренний clearBusyAfterStandaloneCancellation() для снятия признака выполняемой операции после её отмены. ApplicationContainer содержит register(closeable: AutoCloseable) для регистрации ресурса, который нужно закрыть, и close() для закрытия таких ресурсов. Кроме add, возвращающего Boolean, перечисленные методы управления экраном возвращают Unit. CollectionUiState, LoanRow и AdministrationUiState хранят состояние и не имеют собственных прикладных методов.

Для сохранения из форм используется тип SaveUiState. Его поля isSaving: Boolean, error: String? и revision: Int показывают, выполняется ли действие, есть

ли ошибка и сколько успешных изменений выполнено. Объект SaveOperation хранит scope: CoroutineScope, обработчик onError: (String?) -> Unit и mutable: MutableStateFlow; свойство state предоставляет состояние только для чтения. Метод submit(operation: suspend () -> Unit): Boolean запускает действие и сообщает, принято ли оно, а reject(message: String) записывает ошибку. Форма закрывается только после успешной записи, а при ошибке сохраняет введённые значения. Пока идёт сохранение, повторное нажатие не создаёт второй объект.

ReadersViewModel и PublicationsViewModel дополнительно предоставляют saveState: StateFlow, LoansViewModel — actionState того же типа. Каждая модель использует собственный SaveOperation. Это отделяет факт нажатия кнопки от результата записи в базу.

Для добавления учебных записей выделен интерфейс TestDataSource с методами addBooks(): Int и addReaders(): Int. Реализация TestDataSeeder хранит repositories: LibraryRepositoryBundle, random: Random, factory: LibraryEntityFactory, mutex: Mutex и список readerNames. Методы добавляют не более трёх книг или пяти читателей за один вызов и возвращают число добавленных записей. Список книг описывает объект TestBookCatalogue: его поле books содержит List, а метод find(isbn: String?): TestBook? ищет образец по ISBN. TestBook хранит coverKey, isbn, title и authors типа String и year типа Int. Он представляет учебный образец, а не новый класс сущности библиотеки.

TestDataViewModel получает source: TestDataSource и scope: CoroutineScope, использует SaveOperation и предоставляет state и message для экрана. Внутреннее поле mutableMessage хранит TestDataMessage с полями kind: String и text: String. Методы addBooks() и addReaders() запускают соответствующее добавление, а close() прекращает фоновые задачи.

LibraryDesign хранит параметры интерфейса и названия разделов. LibraryTheme задаёт шрифт, цвета и формы компонентов. LibraryIcons предоставляет векторные значки. PublicationCover отображает встроенную обложку для известного ISBN или нейтральную заглушку. Функции formatLoanDate и statusLabel подготавливают

понятные даты и подписи состояний; преобразование времени в календарную дату реализовано в JVM-части приложения. Эти средства оформления не меняют правила выдачи и хранения.

App, LibraryScreen, ReadersPane, PublicationsPane, LoansPane и ArchitecturePane представлены функциями Compose, а не отдельными классами. Они получают данные и показывают соответствующие части окна. Поэтому в разделе интерфейсных классов отдельно описаны объекты управления экранами, а экранные функции рассматриваются как средства взаимодействия с пользователем. Значения, временно введённые в форму, ещё не являются полями сохранённого читателя или издания.

2.4 Используемые паттерны проектирования

Паттерн, или шаблон проектирования, — типовой способ организации классов для решения повторяющейся задачи. Ниже для каждого выбранного шаблона объясняется, какую задачу он решает в библиотечной программе и какие классы в нём участвуют. Русское название поясняет смысл, английское позволяет сопоставить описание с общепринятым названием шаблона.

2.4.1 Репозиторий (Repository)

Шаблон «Репозиторий» выделяет работу с данными в отдельные классы. Например, экрану читателей нужен список читателей, но ему не нужно знать, в какой таблице этот список хранится и как выполняется запрос. Он обращается к ReaderRepository. Аналогично PublicationRepository работает с изданиями, LoanRepository — с выдачами. Их реализации с префиксом Room читают и сохраняют данные через LibraryDao. Если способ хранения изменится, экран не придётся переделывать из-за изменения запросов. Участники шаблона показаны на рисунке 2.

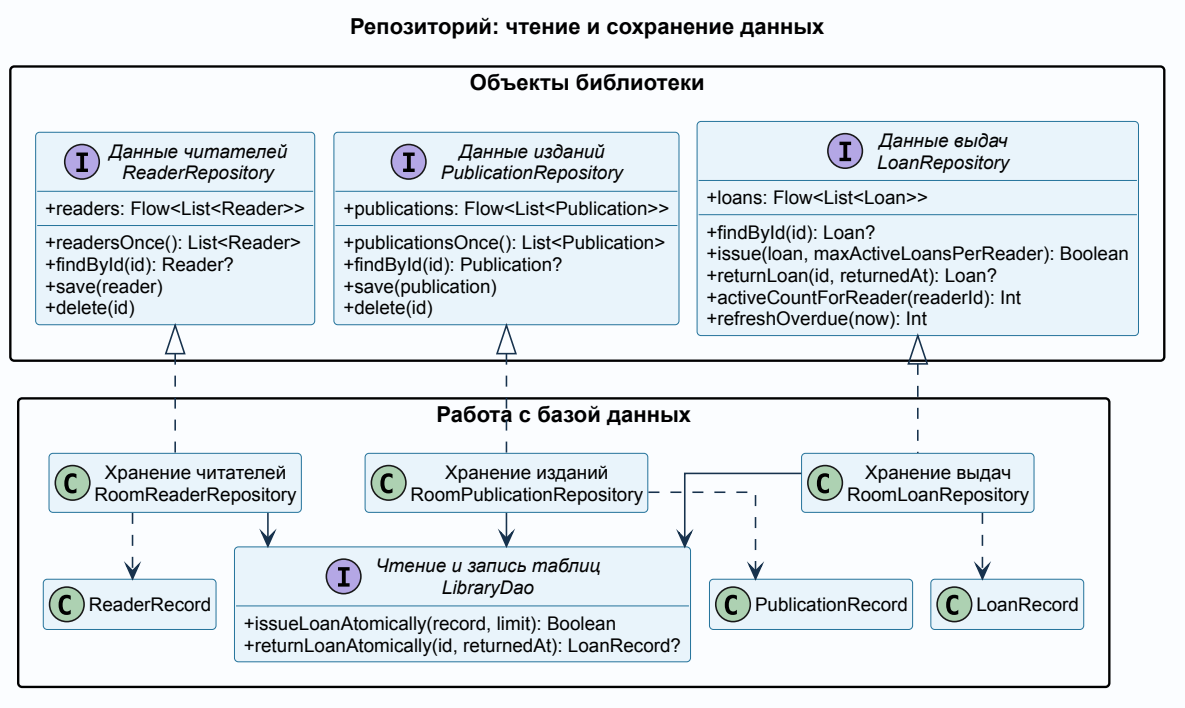


Рисунок 2 — Шаблон «Репозиторий»: отделение чтения и сохранения данных

2.4.2 Разделение данных и экрана (MVVM)

MVVM — сокращение от Model–View–ViewModel, то есть «модель — представление — модель представления». Модель содержит сведения и операции библиотеки. Представление показывает окно и его элементы. Модель представления подготавливает данные для окна и обрабатывает действия пользователя. Например, ReadersPane показывает список читателей, а ReadersViewModel принимает текст поиска и подготавливает подходящие записи. LibraryScreen и панели показывают состояние CollectionUiState, а основные операции выполняются через RoomLibraryService и репозитории. Такое разделение позволяет менять оформление окна отдельно от правил работы с данными. Связи показаны на рисунке 3.

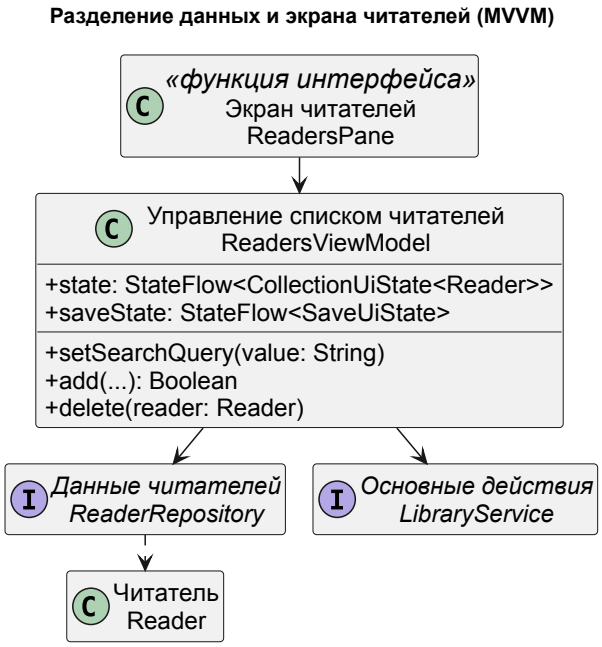


Рисунок 3 — Разделение данных, управления экраном и отображения

2.4.3 Стратегия (Strategy)

Шаблон «Стратегия» позволяет вынести изменяемое правило в отдельный объект. В библиотеке таким правилом является порядок выдачи: сколько изданий можно выдать одному читателю и на какой срок. Интерфейс LoanPolicy задаёт методы canIssue для проверки разрешения и dueAt для определения срока. Реализация DefaultLoanPolicy устанавливает для книги 21 день, для периодического издания — 7 дней. RoomLibraryService использует эти правила, но не хранит их внутри себя. Поэтому другой вариант правил можно подключить без изменения действий выдачи и хранения. Схема приведена на рисунке 4.

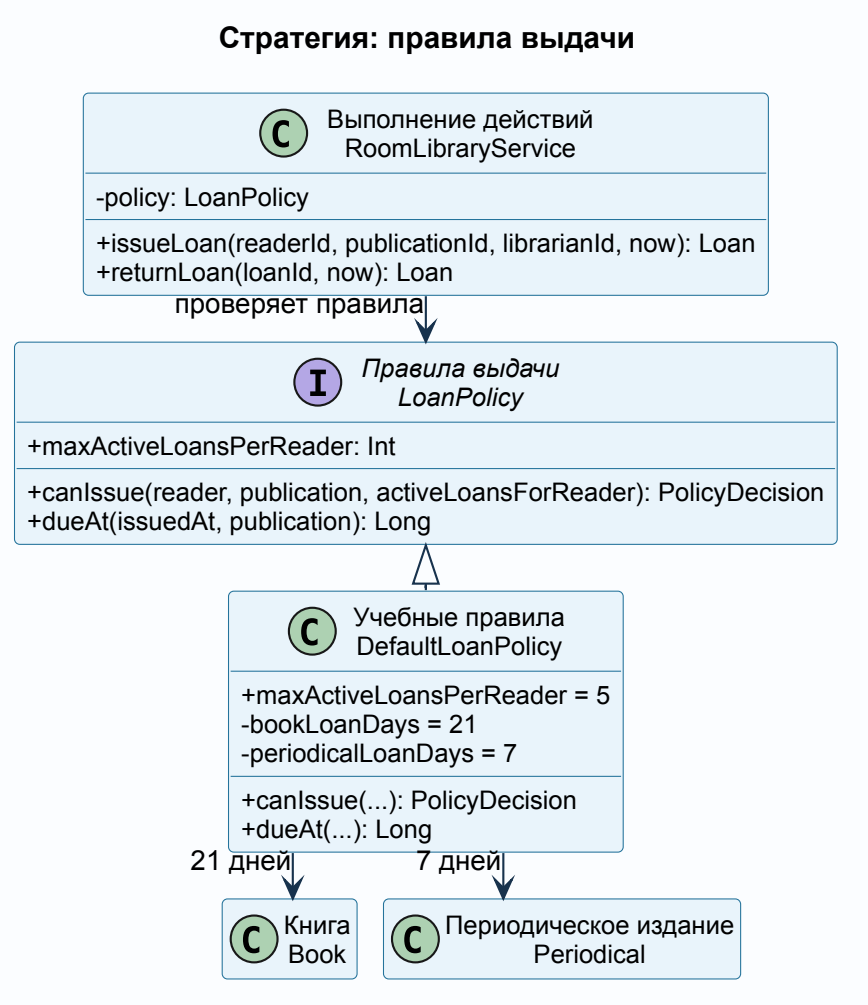


Рисунок 4 — Шаблон «Стратегия»: отдельный объект с правилами выдачи

2.4.4 Фабрика (Factory)

Шаблон «Фабрика» собирает создание объектов в одном классе. Например, для новой книги нужно передать название, авторов, год и другие сведения, а также получить новый идентификатор. Эти действия выполняет LibraryEntityFactory, используя генератор IdGenerator. При загрузке учебного примера фабрика создаёт филиал, библиотекаря, двух читателей, книгу, периодическое издание и выдачу. Затем связанные записи сохраняются вместе. Такая организация устраняет повторение одинаковых действий в разных частях программы. Здесь используется простая фабрика, а не отдельная иерархия фабричных методов. Схема создания объектов приведена на рисунке 5.

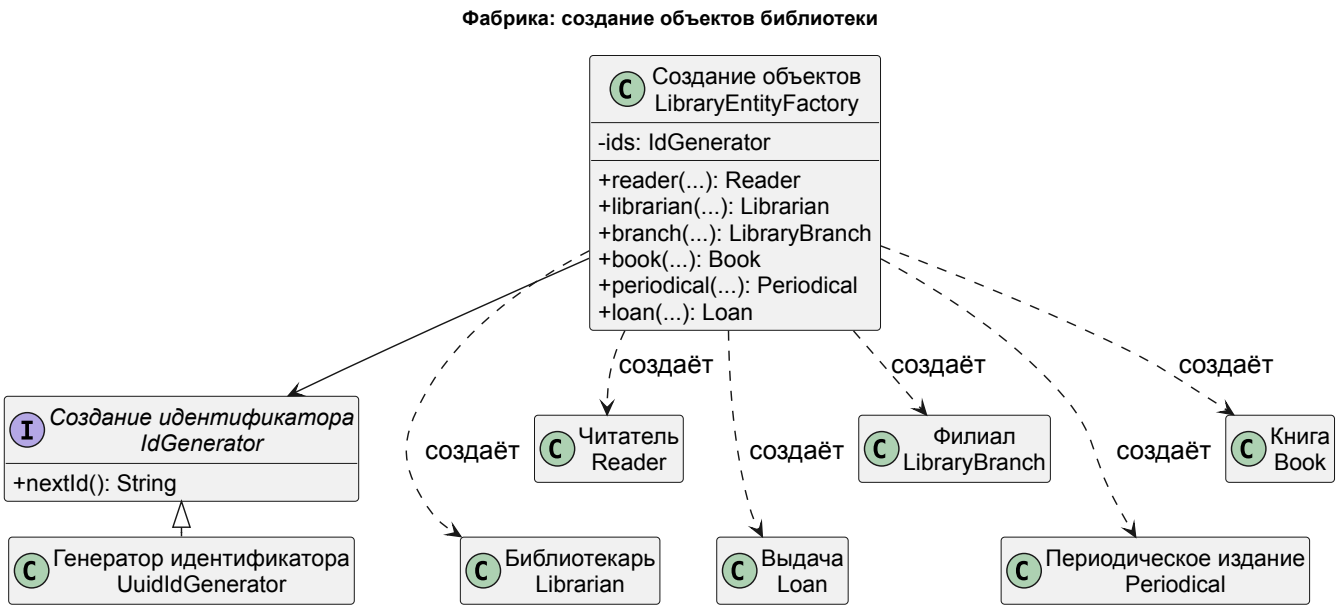


Рисунок 5 — Шаблон «Фабрика»: создание объектов библиотеки

2.4.5 Наблюдатель (Observer)

Шаблон «Наблюдатель» нужен, чтобы экран обновлялся после изменения данных. Например, после добавления читателя новая запись должна появиться в списке без ручной перезагрузки окна. LibraryDao сообщает об изменении записей через Flow. Репозиторий преобразует их в объекты читателей, а ReadersViewModel подготавливает новое состояние списка. Функция ReadersPane получает его и обновляет отображение. Для этого используются средства stateIn и collectAsState, которые связывают изменения данных с состоянием экрана. Для изданий и выдач действует такой же порядок. Цепочка обновления показана на рисунке 6.



Рисунок 6 — Шаблон «Наблюдатель»: обновление списка при изменении данных

2.4.6 Состояние (State)

Шаблон «Состояние» позволяет явно описать, на каком этапе находится выдача. Класс Loan хранит состояние LoanState. Active означает, что издание выдано и срок ещё не истёк, Overdue — что срок истёк, Returned — что издание возвращено. Метод returnAt допускает переход к возврату из первых двух состояний. Из завершённого состояния повторный переход запрещён. При сохранении возврата также проверяется, что количество свободных экземпляров не увеличивается второй раз. Здесь состояния представлены отдельными типами; сложные правила поведения внутри каждого из них не требуются. Связи и переходы приведены на рисунке 7.



Рисунок 7 — Шаблон «Состояние»: выдача, просрочка и возврат

2.4.7 Фасад (Facade)

Шаблон «Фасад» предоставляет простое действие вместо нескольких внутренних вызовов. Чтобы выдать издание, экран обращается к одному методу issueLoan через LibraryService. RoomLibraryService сам находит читателя, издание и библиотекаря, проверяет правила LoanPolicy, получает идентификатор выдачи и передаёт изменения в хранилище. Экрану не приходится выполнять каждый шаг самостоятельно. Слово «фасад» здесь означает общий вход к группе связанных операций. Порядок взаимодействия классов показан на рисунке 8.

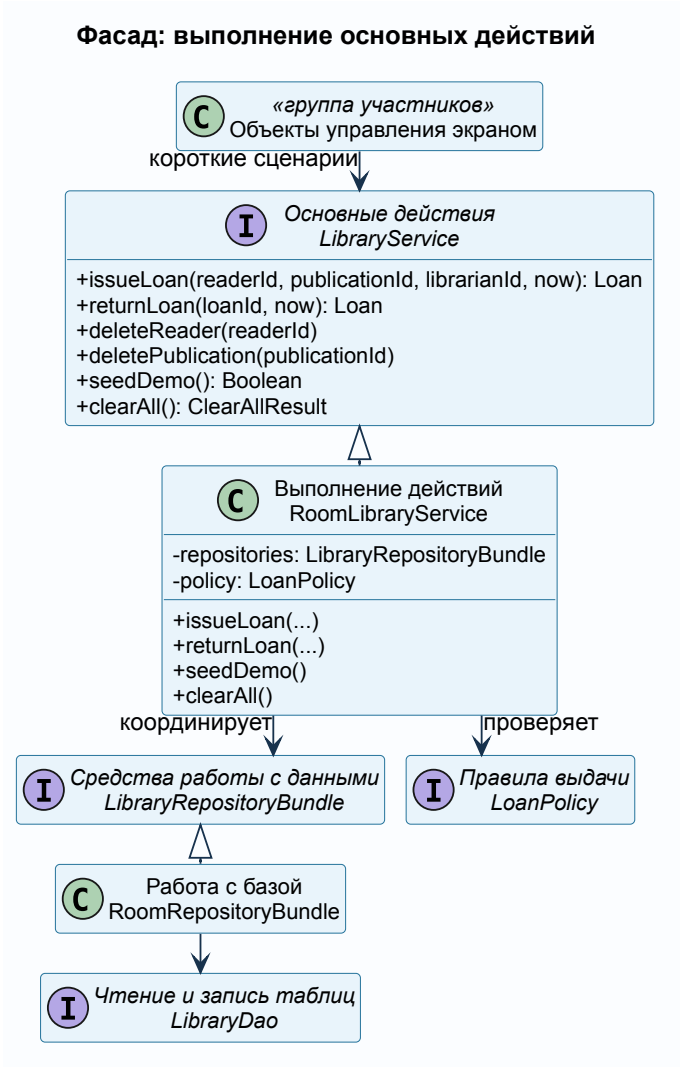


Рисунок 8 — Шаблон «Фасад»: выполнение выдачи через один управляющий метод

2.5 Диаграмма классов

Общая диаграмма классов приведена на рисунке 9. Объекты библиотеки хранят сведения о читателях, изданиях и выдачах. Управляющие классы выполняют действия, экранные компоненты показывают результат, а классы хранения читают и сохраняют записи. Стрелки показывают, какой класс использует другой: например, экран выдач обращается к управляющему классу, а тот — к средствам работы с данными.

При выдаче нужно одновременно сохранить запись и уменьшить количество свободных экземпляров. При возврате нужно завершить выдачу и восстановить остаток. Эти изменения выполняются вместе, чтобы сведения не противоречили друг другу. История выдач сохраняется, поэтому связанного с ней читателя или издание нельзя удалить.

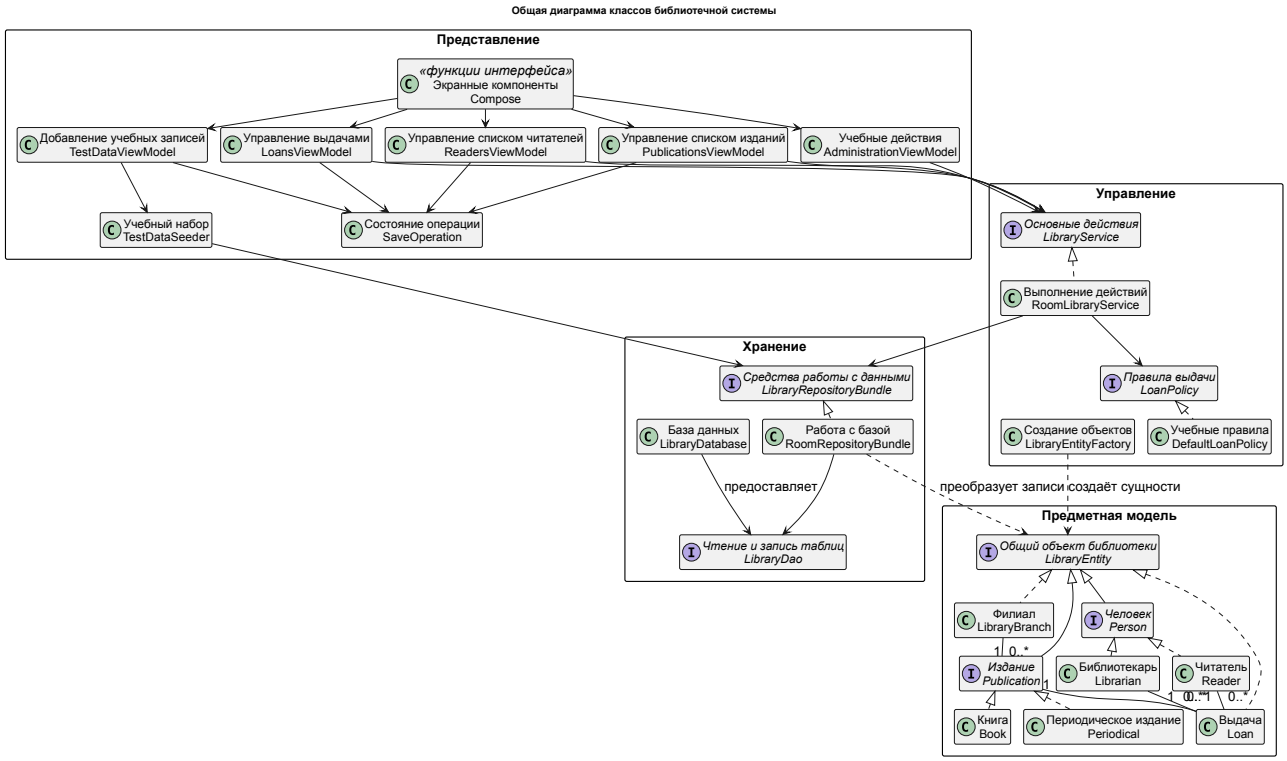


Рисунок 9 — Общая диаграмма классов библиотечной системы

3 Разработка приложения

3.1 Разработка интерфейса приложения

Окно содержит четыре раздела: «Читатели», «Издания», «Выдачи» и «Архитектура». Постоянное боковое меню позволяет переходить между задачами. В рабочей области находятся заголовок, поиск, основные действия и список записей. Формы открываются отдельно и при просмотре каталога не занимают место.

Для оформления выбрана светлая нейтральная тема: белые рабочие поверхности, серый фон, тёмный текст и единые векторные значки. Выбранный раздел выделяется цветом и индикатором. При уменьшении ширины поля формы располагаются друг под другом. Длинные названия изданий допускают перенос. Все пользовательские подписи и сообщения представлены на русском языке.

Изображения получены при выполнении автоматических проверок настоящих компонентов Compose. Проверки используют временные учебные базы и выполняют реальные операции программы. Это реализованный интерфейс, а не макет и не личные записи пользователя.

3.1.1 Работа с читателями

Каталог показывает имя, год рождения, адрес и организацию читателя. Поиск сравнивает введённый текст с именами. Состояние после регистрации показано на рисунке 10.

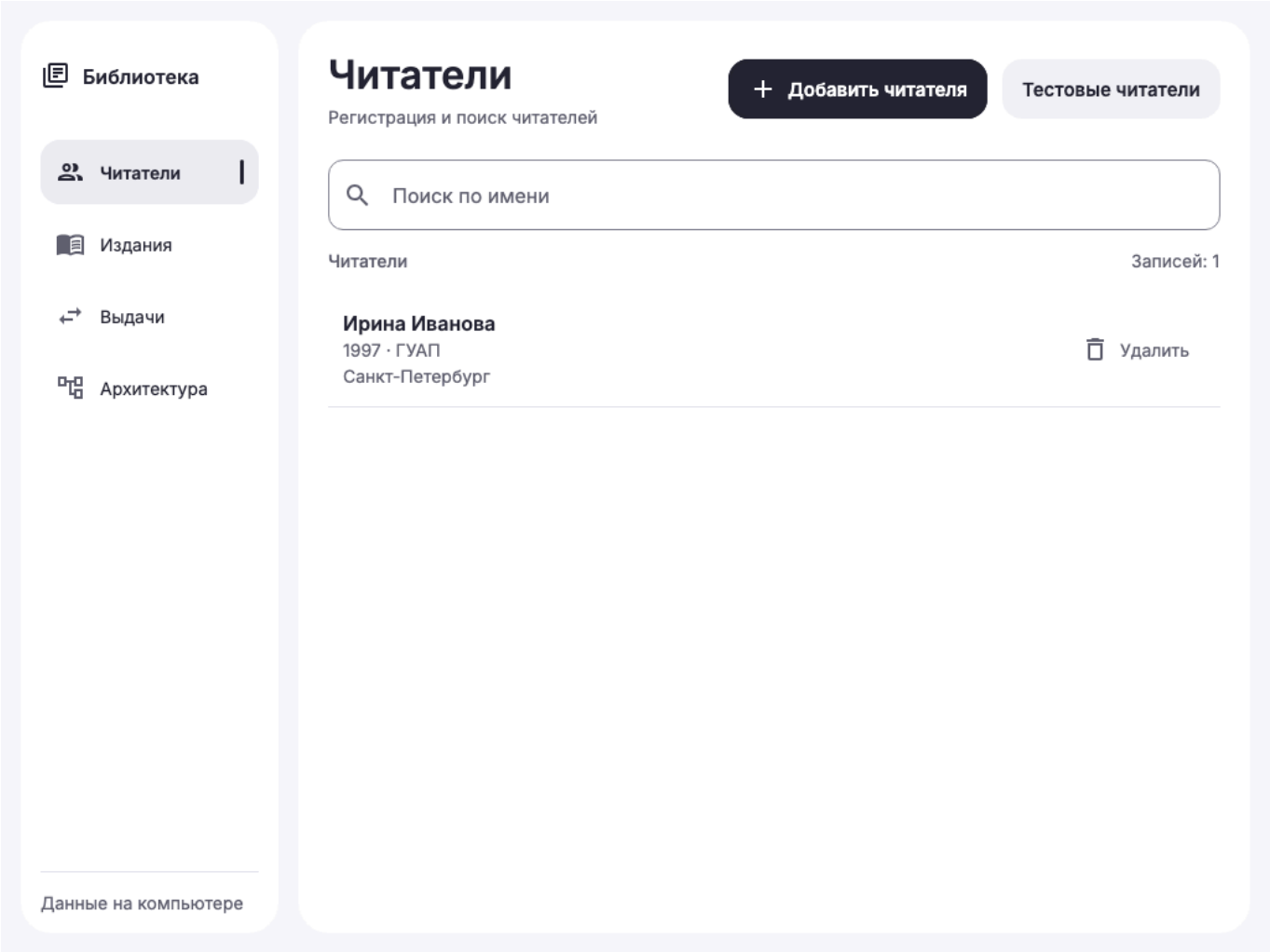


Рисунок 10 — Каталог после регистрации читателя

Кнопка «Добавить читателя» открывает форму. Сохранение начинается после проверки обязательных полей и преобразования года в число. Форма закрывается только после успешной записи. При ошибке она остаётся открытой вместе с введёнными значениями. Пока выполняется сохранение, повторное нажатие не создаёт второй объект.

Удаление требует подтверждения; ответ «Отмена» сохраняет запись. Если читатель связан с выдачами, удалить его нельзя даже после возврата всех изданий: история должна оставаться связанной с существующими сведениями. Кнопка «Тестовые читатели» добавляет до пяти вымышленных читателей за нажатие; уже добавленные имена из учебного списка исключаются.

3.1.2 Работа с изданиями

Каталог показывает книги и периодику: название, авторов или редакторов, издательство, год, филиал и свободный остаток. Поиск работает по названию и автору. Для книги выводится ISBN, для периодического издания — номер выпуска.

Кнопка «Добавить издание» открывает форму с выбором вида. Пользователь указывает сведения, количество экземпляров и филиал. Новое издание получает свободный остаток, равный общему числу экземпляров. Заполненная форма книги показана на рисунке 11.

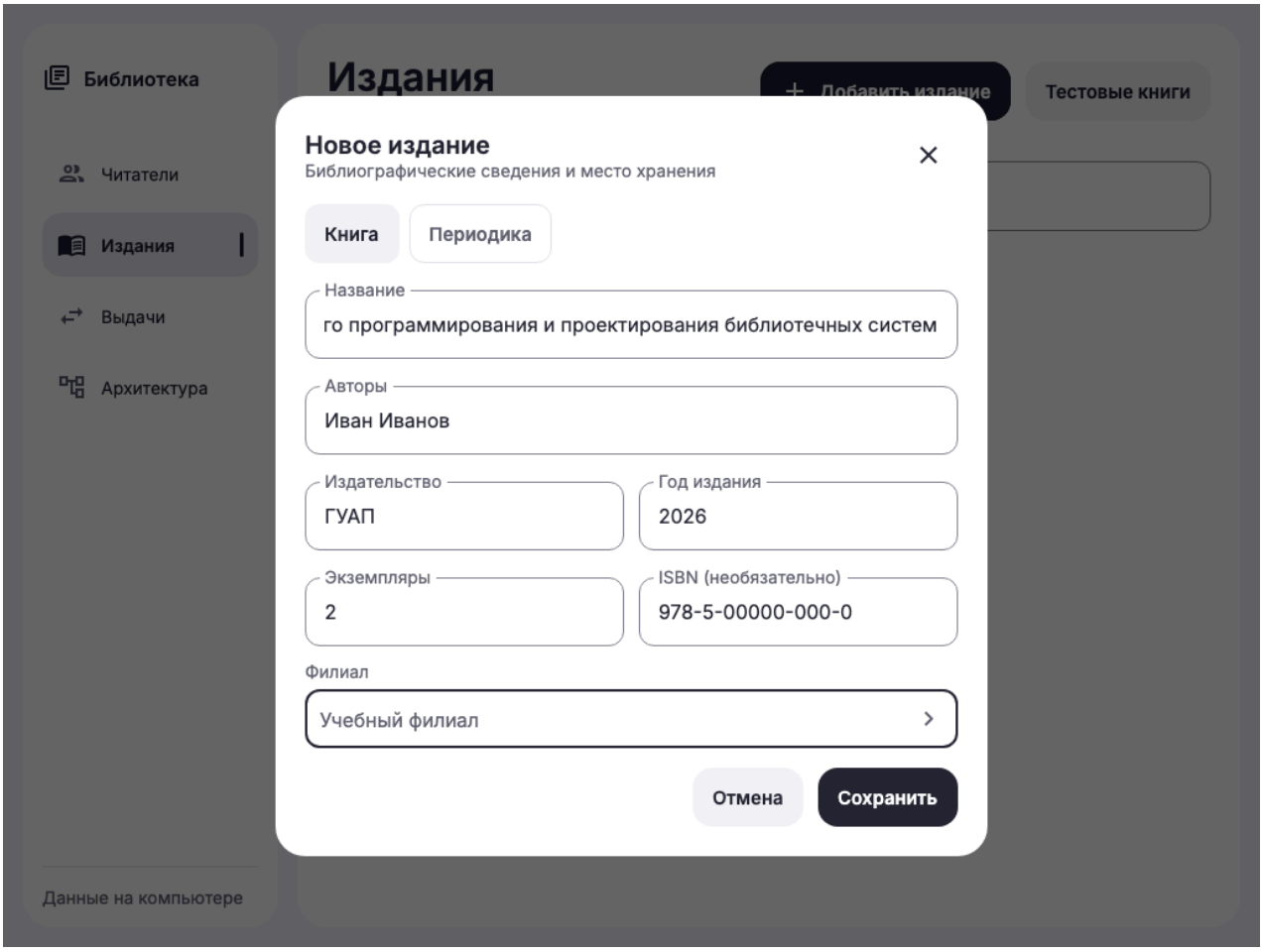


Рисунок 11 — Форма регистрации книги

«Тестовые книги» добавляет до трёх случайно выбранных книг из набора шести настоящих изданий. Сверка по ISBN предотвращает повторное добавление того же образца. При необходимости создаются служебные учебные сведения о филиале и библиотекаре. Обложки включены в ресурсы и работают без интернета. Их цвета

приглушены; для неизвестного ISBN используется нейтральная заглушка. Каталог с обложками показан на рисунке 12.

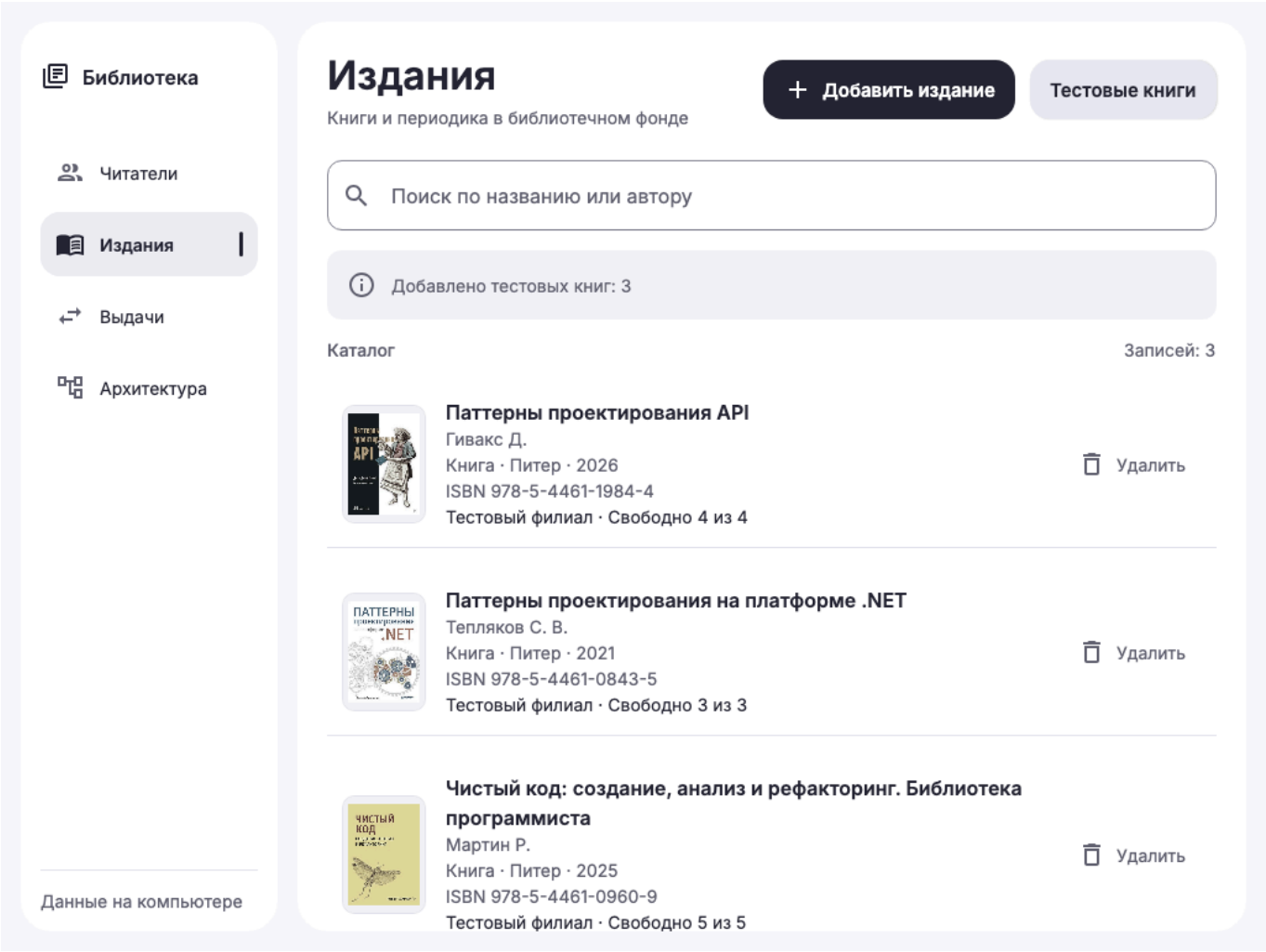


Рисунок 12 — Каталог учебных книг со встроенными обложками

Удаление издания требует подтверждения и допускается только без связанных выдач. Изменение описательных сведений существующей записи не затирает остаток, изменённый выдачами.

3.1.3 Выдача и возврат

В разделе «Выдачи» показаны читатель, издание, даты, срок и состояние. Служебные идентификаторы заменены именами и названиями. Кнопка «Оформить выдачу» открывает выбор участников, показанный на рисунке 13.

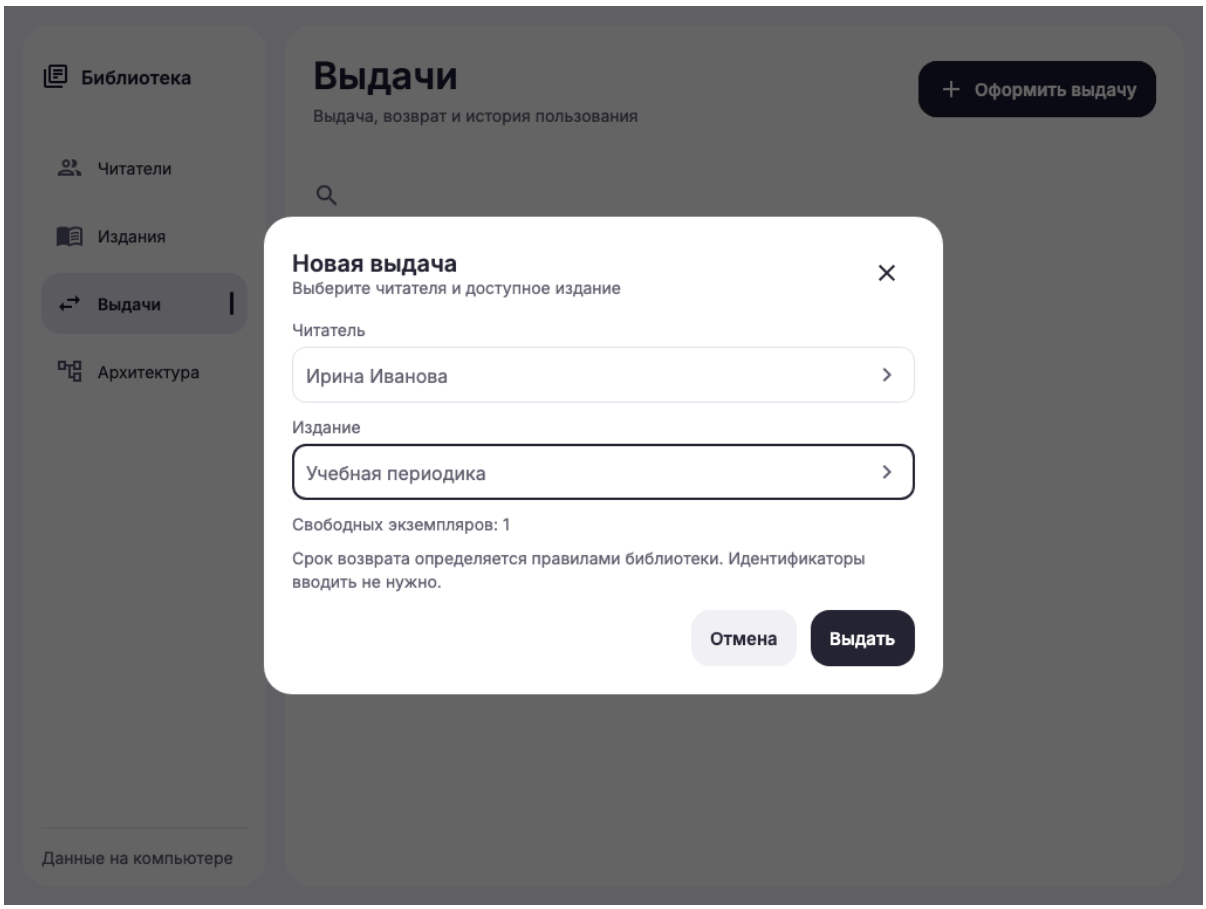


Рисунок 13 — Выбор читателя и доступного издания

До выбора обоих участников кнопка «Выдать» недоступна. Если издание исчезло или свободные экземпляры закончились, прежний выбор становится недействительным. Проверяются наличие экземпляра и допустимое число невозвращённых выдач. Учебные правила задают срок 21 день для книги, 7 дней для периодики и не более пяти невозвращённых выдач читателю.

После успешной выдачи форма закрывается, новая запись появляется в списке, а свободный остаток уменьшается. Возврат выполняется из строки выдачи. История сохраняется, состояние меняется на «Возвращено», остаток увеличивается ровно один раз. Для возвращённой записи кнопка «Вернуть» недоступна. Сроки проверяются раз в минуту, пока приложение работает; истёкшие невозвращённые выдачи отмечаются как просроченные.

3.1.4 Архитектура и учебные действия

Раздел «Архитектура» объясняет основные объекты, связь читателей с изданиями и шаблоны проектирования. Здесь расположены загрузка связанного демонстрационного набора и очистка учебной базы. Раздел приведён на рисунке 14.

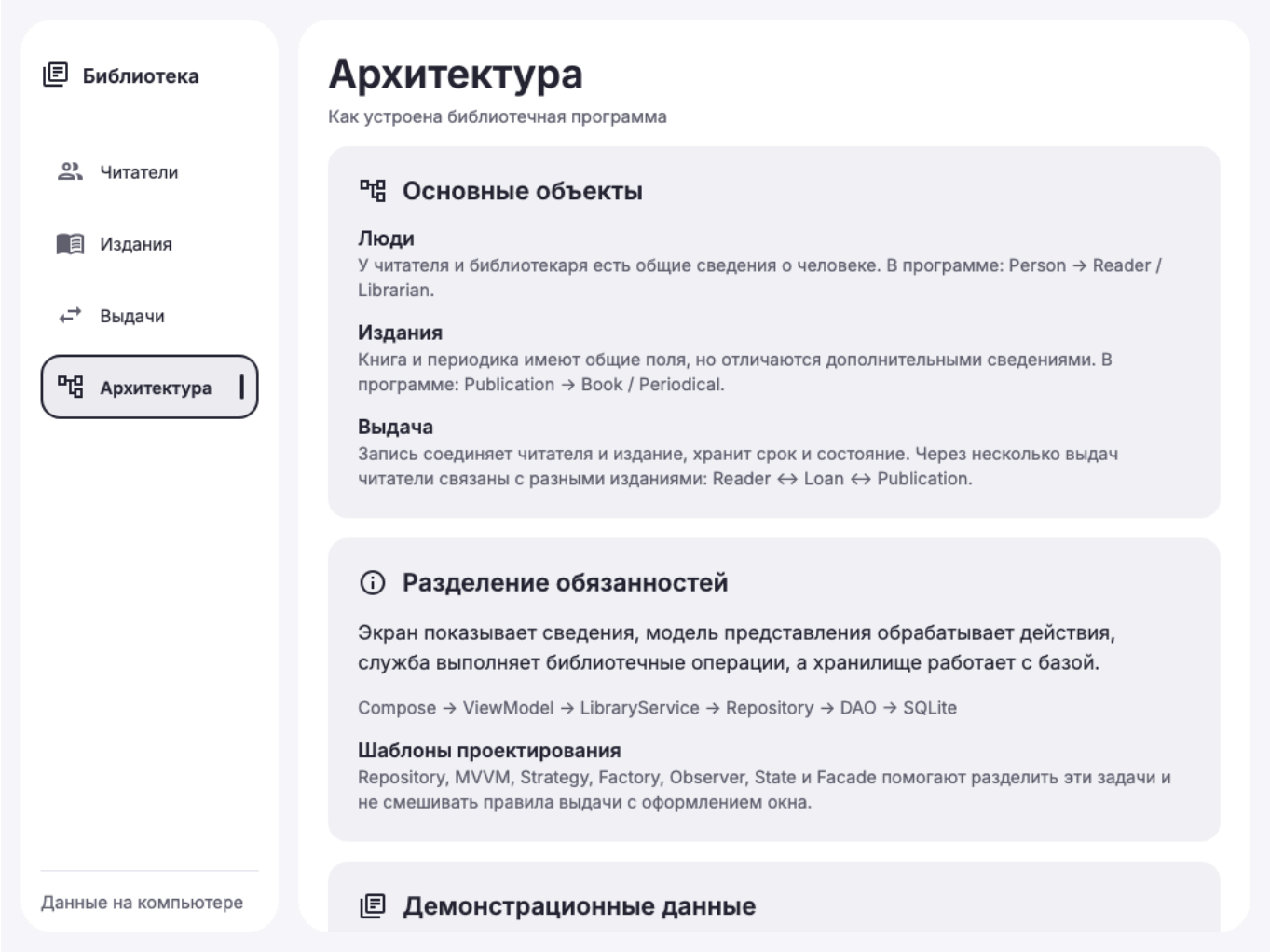


Рисунок 14 — Объяснение устройства программы

Связанный набор содержит филиал, сотрудника, двух читателей, книгу, периодическое издание и выдачу. Это отдельный сценарий, не равнозначный кнопкам тестовых книг и читателей. При наличии филиала набор не загружается повторно. Очистка выполняется только по команде с подтверждением. Закрытие окна не удаляет данные.

3.2 Реализация методов классов

3.2.1 Средства разработки и структура

Программа написана на Kotlin. Графическое окно создано с помощью Compose Multiplatform; здесь используется настольная платформа JVM, а не сайт. Для сохранения выбрана SQLite, хранящая данные в локальном файле. Room связывает программные записи с таблицами и выполняет запросы. Gradle подключает библиотеки, компилирует исходники и запускает проверки. Gradle Wrapper использует версию сборщика, заданную проектом.

Модуль core содержит библиотечные объекты, правила и хранение. Модуль app содержит окно и модели представления. Модуль docs обеспечивает сборку записки. Правила можно проверять отдельно от графического окна. При нажатии «Выдать» экран вызывает модель представления, она обращается к службе библиотеки, служба проверяет правила и изменяет базу. Наблюдаемые списки передают обновление экрану. Общая схема приведена на рисунке 9.

Объекты библиотеки отделены от записей базы: Reader представляет читателя в программе, ReaderRecord — строку таблицы. Функции преобразования переводят сведения между представлениями, чтобы остальные части программы не зависели от устройства запроса.

3.2.2 Обоснование выбранных технологий

Для реализации выбраны технологии, которые соответствуют характеру задачи и доступны в учебной среде. Обоснование ниже связывает каждое решение с конкретным требованием проекта.

Язык Kotlin выбран потому, что предметная область описывается иерархией типов, а необходимые объектно-ориентированные средства — интерфейсы, наследование, неизменяемые данные, проверка значений в конструкторах — в нём выражены кратко и строго. Требование иметь не менее пяти классов сущностей и общую иерархию выполняется без дополнительных оболочек. Альтернативой был бы Java: он также объектно-ориентирован, но потребовал бы больше служебного кода для тех же

моделей, а свойства данных и проверки пришлось бы писать вручную. Kotlin выбран не как самоцель, а как средство уложить модель предметной области в компактный и проверяемый код.

Compose Multiplatform выбран для графического интерфейса, потому что методичка отдаёт предпочтение графическому приложению, а не консольному. Он позволяет описать окно, формы, списки и состояния на одном языке с остальной программой и не требует отдельного языка разметки и шаблонов. В проекте используется только настольная платформа JVM, то есть обычное окно операционной системы, а не сайт и не браузерное приложение: методичка прямо не допускает веб-приложения. Compose распространяется и на другие платформы, но в этом проекте они не собираются и не проверялись, поэтому за подтверждённые принимаются только настольные сборки.

Для хранения выбрана SQLite в связке с Room. Библиотечный учёт требует связанных записей, внешних ключей и транзакций, поэтому встроенная реляционная база подходит лучше простого файла или сериализации в память. SQLite хранит данные в одном локальном файле, не требует отдельного сервера и работает без сети, что соответствует локальному учебному характеру работы. Room добавлен как слой над SQLite: он превращает классы предметной области в таблицы, проверяет запросы во время сборки и берёт на себя транзакции. Альтернатива — писать SQL напрямую через драйвер — дала бы больше ручного кода и меньше проверок, поэтому отвергнута. Отдельные объекты предметной области не заменяются записями базы: Reader описывает читателя в программе, а ReaderRecord — строку таблицы.

Gradle с обёрткой выбран как средство сборки, потому что проект состоит из нескольких модулей и подключает внешние библиотеки. Обёртка фиксирует версию сборщика в репозитории, поэтому проект собирается одинаково на другом компьютере без ручной установки Gradle. Это заметно упрощает запуск и повторную проверку работы.

Для автоматических проверок используется библиотека kotlin.test. Она входит в состав языка, не требует отдельного фреймворка и позволяет проверять и объекты

предметной области, и модели представления, и настоящие компоненты интерфейса. Проверки выполняются на временных базах, поэтому обычные данные пользователя не затрагиваются.

Итог: каждый выбранный компонент решает конкретную задачу проекта — описание иерархии классов, настольный интерфейс, связанное локальное хранение, воспроизводимую сборку и проверку поведения. Технологии подобраны под требования работы, а не взяты как наиболее распространённые.

3.2.3 Хранение и согласованность данных

Используются пять таблиц: readers, librarians, branches, publications и loans. Идентификаторы связывают строки. В таблице изданий хранится признак вида; при чтении по нему восстанавливается книга или периодическое издание. Схема приведена на рисунке 15.

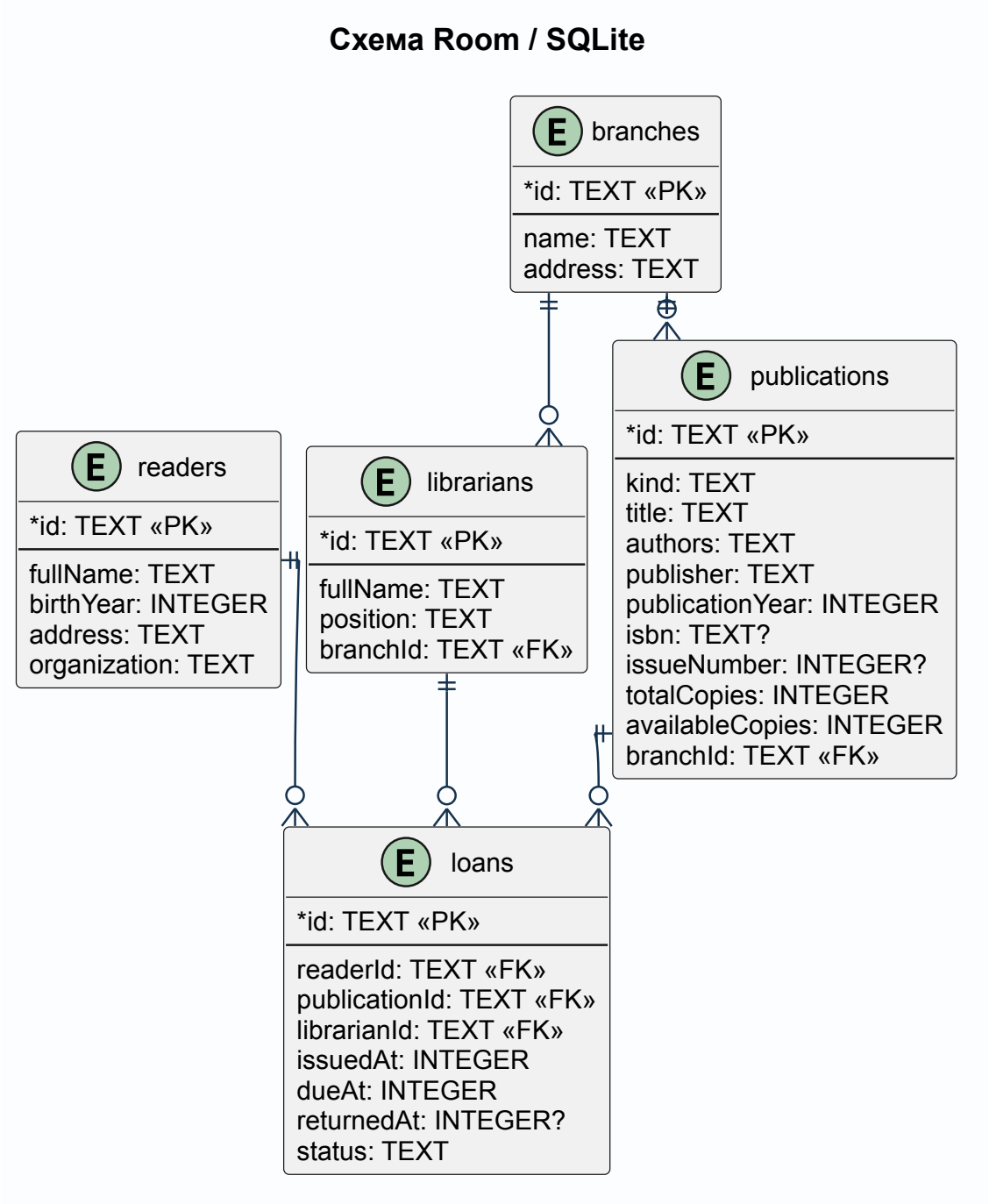


Рисунок 15 — Таблицы и связи локальной базы

Выдача хранит ссылки на одного читателя, сотрудника и издание. Несколько записей реализуют связь «многие ко многим». Внешние ключи защищают используемые записи от удаления. Сохранение выдачи и уменьшение свободного остатка объединены транзакцией: сохраняются либо все изменения, либо ни одно. Возврат аналогично объединяет завершение выдачи и восстановление остатка.

В macOS файл library.db находится в Library/Application Support/CourseOopLibrary домашнего каталога, в Linux — в .local/share/course-oop-library, в

Windows — в CourseOopLibrary внутри LOCALAPPDATA. При обычном закрытии данные сохраняются.

3.2.4 Создание и сохранение

LibraryEntityFactory создаёт объекты и получает идентификатор через IdGenerator. Конструкторы проверяют обязательные сведения и допустимые значения. Вводимые числа сначала преобразуются из текста. SaveOperation не допускает повторного запуска одной операции: isSaving показывает выполнение, error — причину отказа, revision увеличивается после успешной записи. Экран закрывает форму после изменения revision. При ошибке введённые значения остаются. Отмена фоновой задачи не выдаётся за ошибку библиотечных данных.

Хранилище преобразует объект в запись и передаёт её DAO — интерфейсу операций базы. Чтение выполняет обратное преобразование. При обновлении существующего издания изменяются описательные сведения, но сохраняются общее число экземпляров и свободный остаток.

3.2.5 Выдача и возврат

Метод issueLoan службы RoomLibraryService выполняет следующие шаги:

1. Находит читателя, издание и сотрудника; при отсутствии участника прекращает действие.
2. Проверяет свободный экземпляр и ограничение невозвращённых выдач.
3. Создаёт новую выдачу и определяет срок по виду издания.
4. В транзакции повторно проверяет состояние, лимит и остаток, уменьшает остаток и сохраняет выдачу.
5. Возвращает созданную запись.

Повторная проверка внутри транзакции защищает от двух почти одновременных обращений. Ниже приведён короткий фрагмент реального метода базы; полный файл находится в приложении А.

```
    @Transaction
    suspend fun issueLoanAtomically(record: LoanRecord, maxActiveLoansPerReader:
Int): Boolean {
        if (record.status ≠ "ACTIVE" || record.returnedAt ≠ null) return false
        if (countActiveLoans(record.readerId) ⩾ maxActiveLoansPerReader) return
false
        if (decrementAvailable(record.publicationId) ≠ 1) return false
        insertLoan(record)
        return true
    }
```

Метод returnLoan выполняет следующие шаги:

1. Находит выдачу по идентификатору; если записи нет, возвращает пустой результат.
2. Проверяет дату: возврат не может предшествовать выдаче.
3. Если выдача уже возвращена, возвращает её без изменения остатка.
4. В одной транзакции отмечает возврат и увеличивает свободный остаток издания.
5. Если остаток не удалось увеличить, отменяет всю транзакцию, чтобы не оставить частичных изменений.
6. Возвращает обновлённую запись выдачи.

3.2.6 Удаление и учебные данные

Удаление читателя и издания проверяет все связанные выдачи, включая историю: пока с записью связана хотя бы одна выдача, удаление запрещено, а внешние ключи базы поддерживают это правило даже при изменении связей во время операции.

Полная очистка выполняется отдельным методом и состоит из шагов:

1. В одной транзакции подсчитывает текущее число записей в каждой таблице.
2. Удаляет выдачи, затем издания, читателей, сотрудников и филиалы.
3. Порядок удаления обязателен: выдачи удаляются первыми, потому что связи с ними запрещают удаление связанных записей.
4. Возвращает число удалённых записей по группам, которое программа показывает пользователю.

Загрузка связанного демонстрационного набора выполняется в одной транзакции по шагам:

1. Проверяет, есть ли уже филиал; при его наличии набор не загружается повторно и метод сообщает об отказе.
2. Создаёт через фабрику филиал, сотрудника, двух читателей, книгу, периодическое издание и демонстрационную выдачу.
3. Сохраняет связанные записи и уменьшает свободный остаток выдачей.
4. При ошибке на любом шаге отменяет всю транзакцию, поэтому частично загруженный набор не остаётся в базе.

Отдельные учебные кнопки работают иначе и вызываются по одной. TestDataSeeder добавляет книги и читателей к существующим сведениям, не очищая базу. Книги сверяются по ISBN, читатели — по именам. Уже добавленные учебные образцы исключаются из выбора.

3.3 Разработка тестового приложения

Тестовое приложение объединяет разработанные классы в графическом интерфейсе. Через формы пользователь добавляет читателей и издания, выполняет поиск, оформляет выдачу и возврат. Действия передаются управляющим классам; изменения сохраняются в базе и отображаются в соответствующих списках.

Для проверки подготавливаются учебные записи и выполняется последовательность «создание читателя — добавление издания — выдача — возврат». При этом проверяется связь объектов и изменение свободного остатка. Сценарии и результаты проверки приведены в следующем разделе.

4 Тестирование

4.1 Организация проверки

Разработано 67 автоматических тестов. Все тесты успешно выполнены. Результаты выполнения показаны на рисунке 16. Группы проверок приведены в таблице 15.

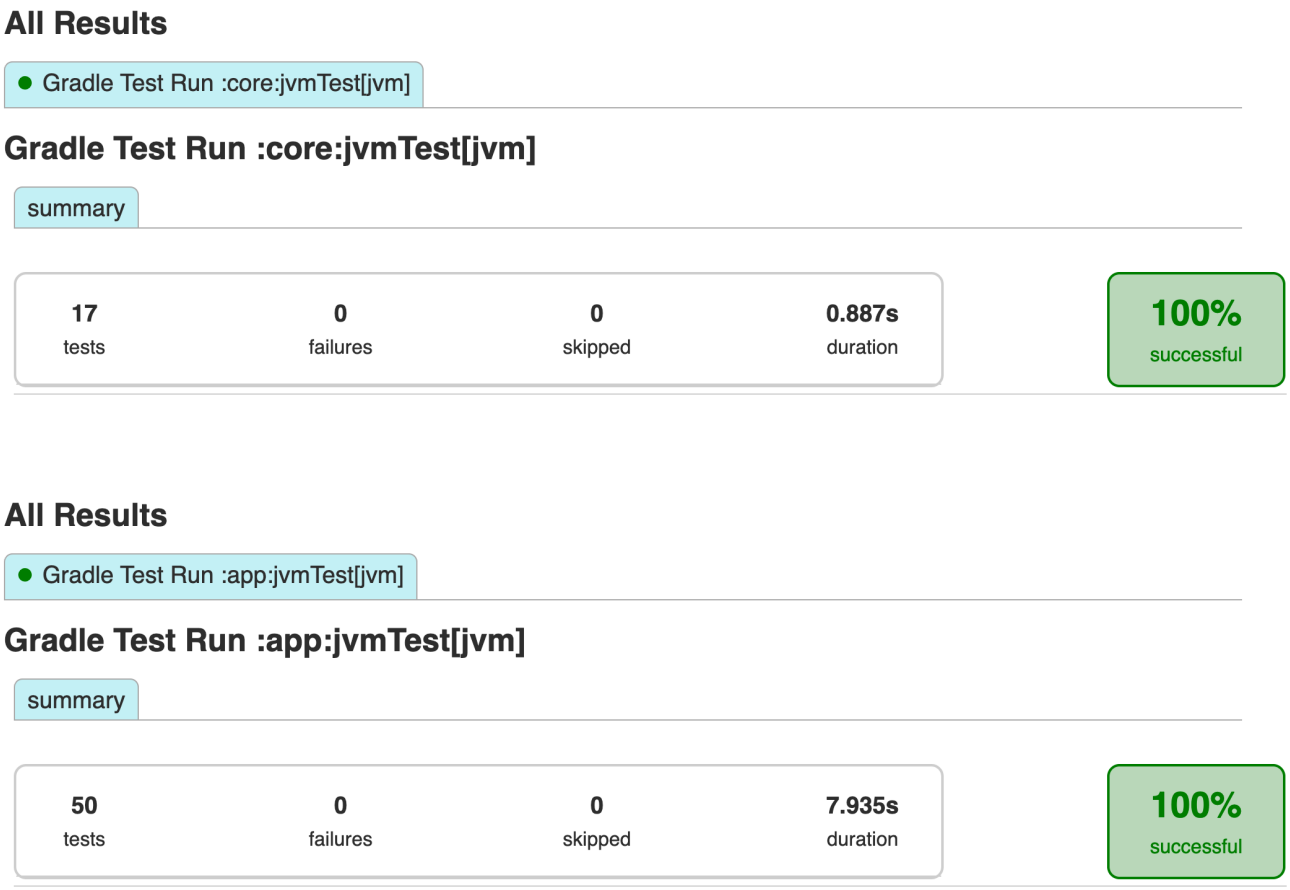


Рисунок 16 — Отчёты о выполнении автоматических тестов

Таблица 15 — Результаты автоматических проверок

Тестовый класс	Число	Что проверяется
DomainRewriteTest	4	Проверки библиотечных объектов и правил
ControlScenarioTest	1	Связанный демонстрационный набор без повторов
RoomAdministrationTest	2	Загрузка и транзакционная очистка
ManyToManyTest	1	Одно издание и разные читатели
ReviewRegressionTest	5	Остаток, лимит выдач, состояние и дата возврата

Тестовый класс	Число	Что проверяется
RoomInvariantTest	2	Согласованность сохранения и повторные операции
RoomPersistenceTest	2	Запись, связи и повторное открытие базы
CatalogInteractionTest	12	Формы, поиск, выдача, возврат, ошибки и адаптивное окно
LibraryDesignTest	3	Параметры темы, контраст и навигация
LoanDateFormatterTest	1	Понятное представление даты выдачи
LoansSelectionTest	3	Очистка устаревшего выбора издания
PublicationSaveInteractionTest	2	Форма издания при успехе и ошибке
ReaderSaveInteractionTest	1	Форма читателя при ошибке и повторном нажатии
TestDataInteractionTest	5	Учебные кнопки, встроенные обложки и отсутствие повторов
TestDataViewModelTest	3	Состояния добавления учебных записей
ViewModelLifecycleAndErrorTest	15	Ошибки, отмена задач, просрочка и завершение работы
ViewModelOperationsTest	4	Операции выдачи, возврата и управления
ViewModelValidationTest	1	Проверка обязательных значений формы

4.2 Наглядные результаты

4.2.1 Исправление ввода и удаление читателя

Введены имя «Ирина Иванова» и недопустимый текст вместо года. После «Сохранить» форма осталась открытой, показала ошибку и сохранила имя. Состояние приведено на рисунке 17. После замены года на 1997 запись появилась в каталоге на рисунке 10. Сценарий также проверил отмену удаления и подтверждённое удаление записи без выдач.

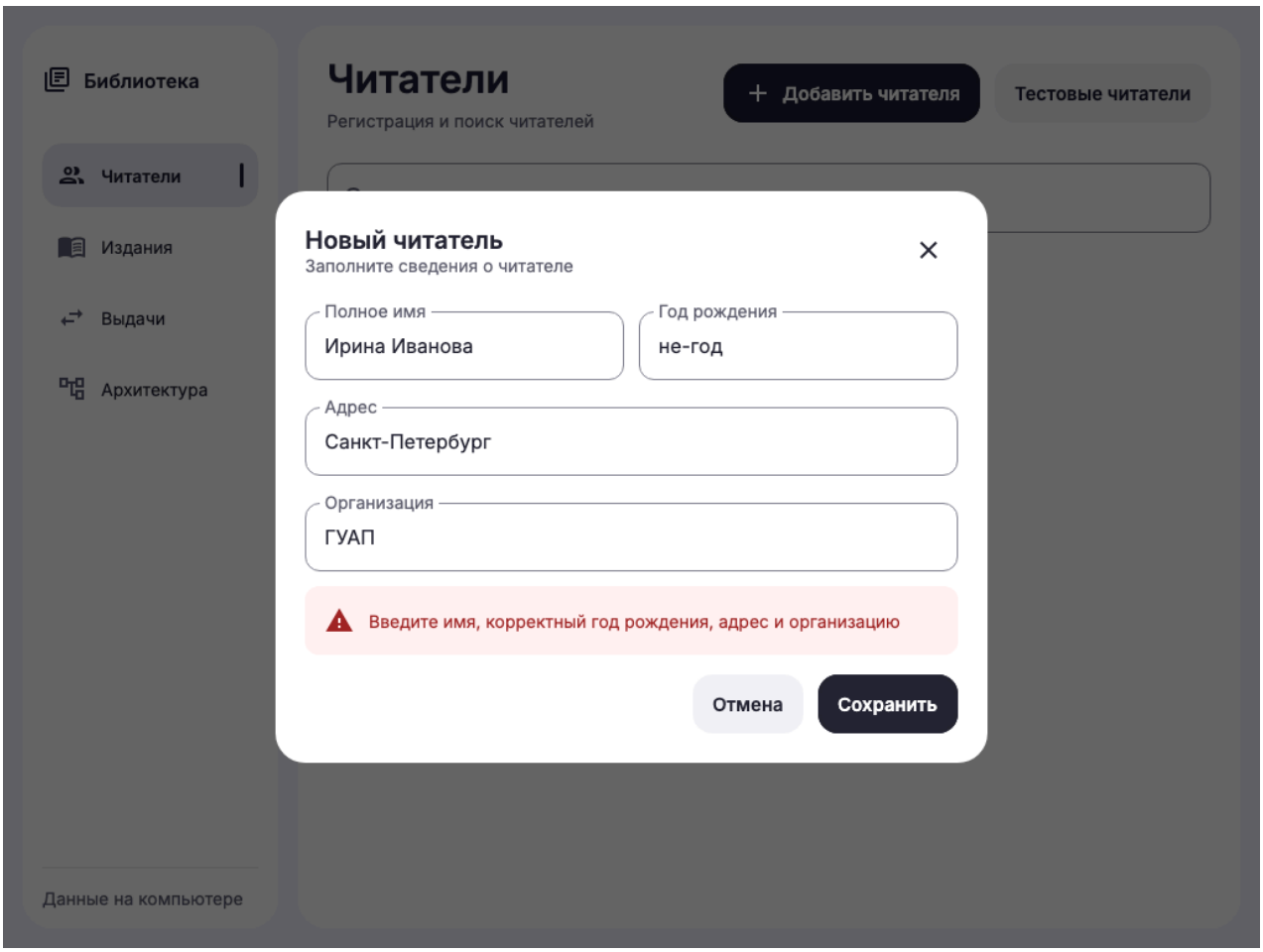


Рисунок 17 — Ошибка ввода без потери заполненных значений

4.2.2 Поиск без совпадений

Ввод строки «несуществующий читатель» привёл к сообщению «Ничего не найдено» на рисунке 18. Очистка поиска вернула обычное состояние каталога. Это не сообщение об ошибке чтения базы.

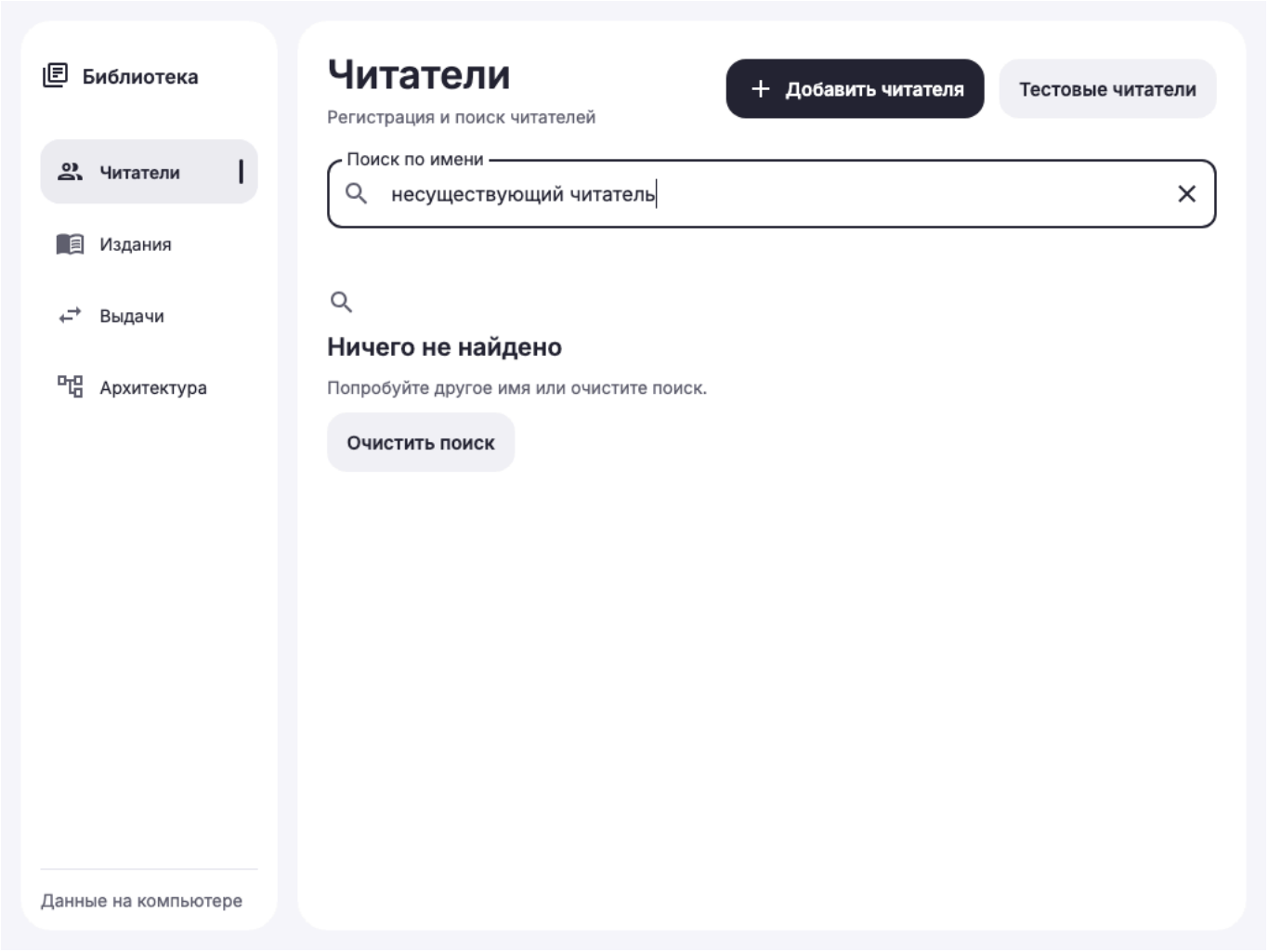


Рисунок 18 — Результат поиска отсутствующего читателя

4.2.3 Выдача и возврат

В сценарии созданы читатель, сотрудник, филиал, книга с двумя экземплярами и периодическое издание с одним. Через форму выбраны читатель и периодика. После выдачи появилась строка «На руках», показанная на рисунке 19, а свободный остаток периодики уменьшился до нуля.

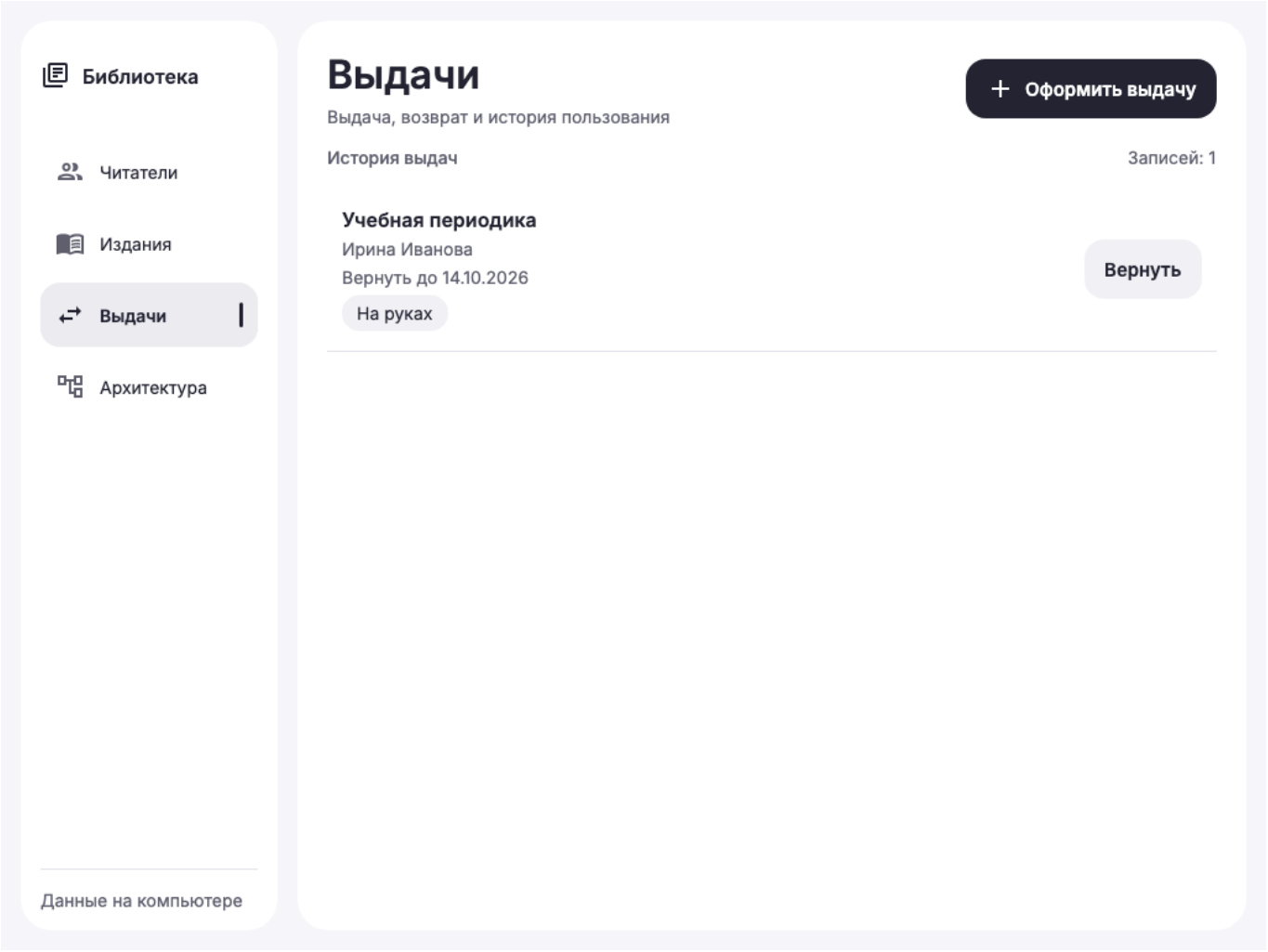


Рисунок 19 — Список после выдачи периодического издания

После «Вернуть» строка получила подпись «Возвращено», кнопка возврата стала недоступной. Результат показан на рисунке 20. Проверка базы подтвердила один свободный экземпляр. Отдельный тест операций хранения проверяет, что повторный возврат не увеличивает остаток.

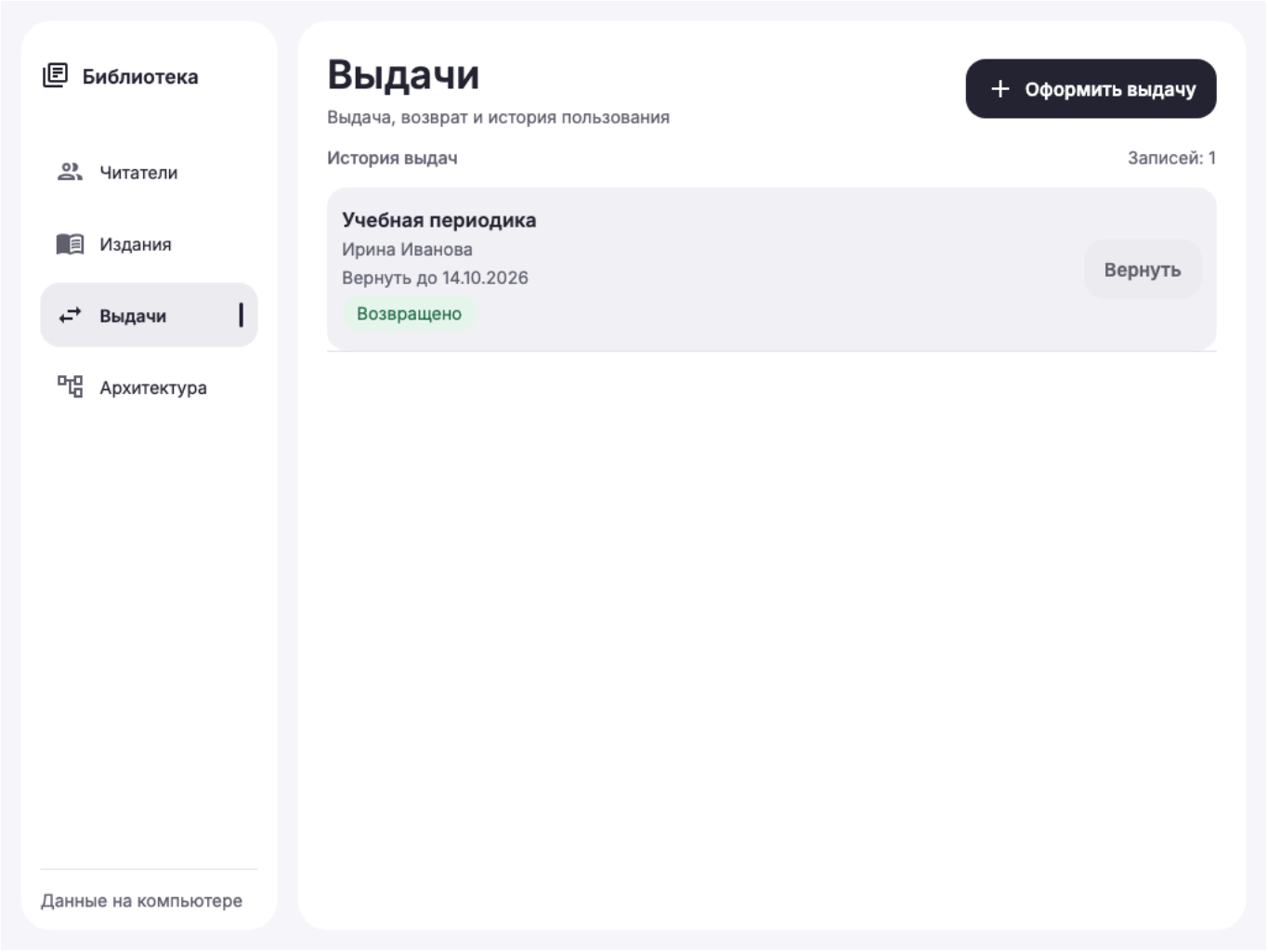


Рисунок 20 — История выдачи после возврата

4.2.4 Очистка с подтверждением

В отдельном сценарии после «Очистить данные» сначала выбран ответ «Отмена»; запись сохранилась. Затем очистка подтверждена, программа сообщила число удалённых записей, а база больше не содержала читателя. Результат приведён на рисунке 21. Операция выполнялась только на временной базе.

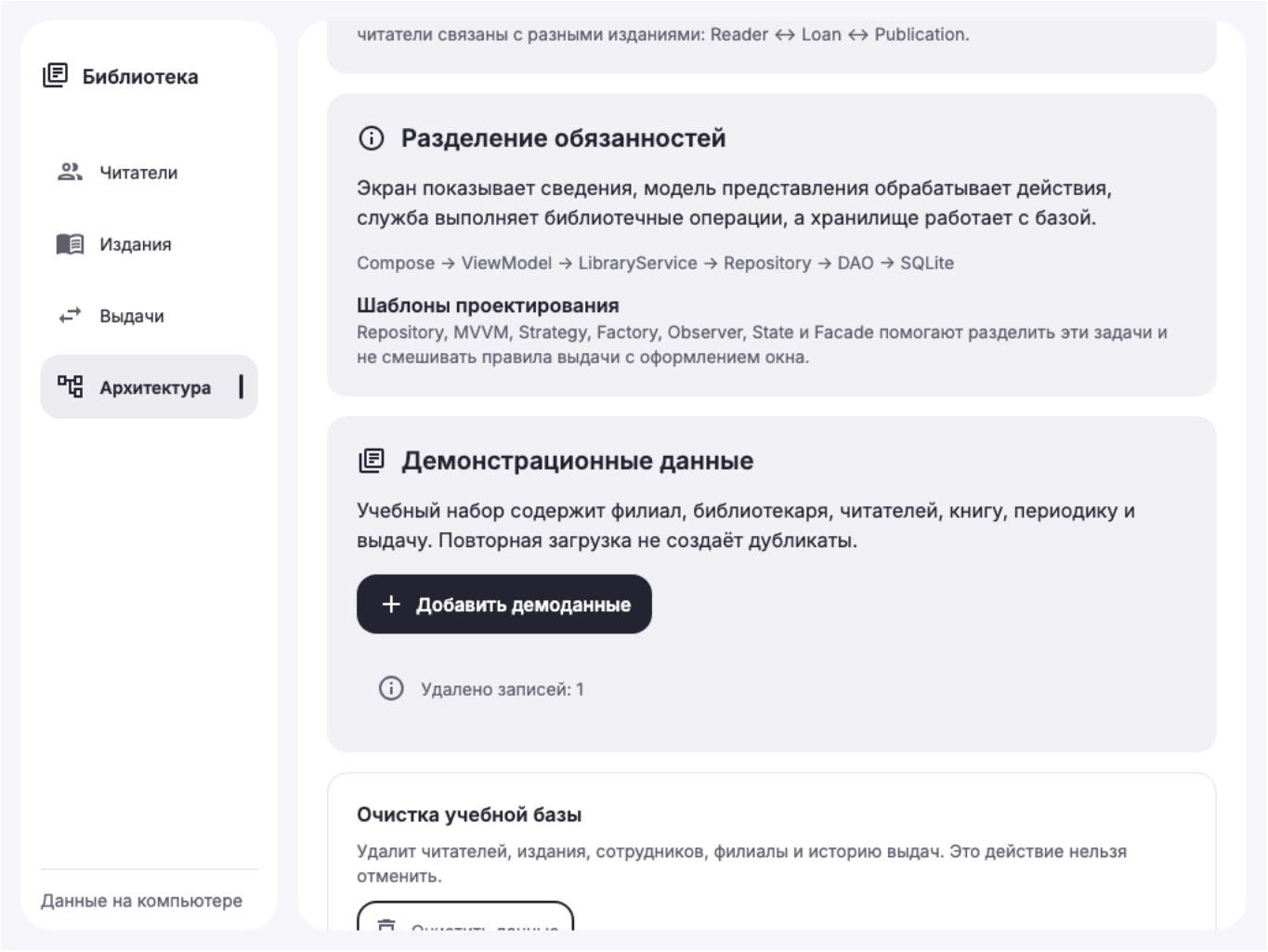


Рисунок 21 — Результат подтверждённой очистки

4.2.5 Учебные данные и обложки

TestDataInteractionTest нажимает «Тестовые книги» и «Тестовые читатели», ожидает обновления и сверяет сохранённые записи. Для известного ISBN проверяется встроенная обложка. Повторное добавление не создаёт дубликатов образцов. Каталог на рисунке 12 получен в таком сценарии.

4.3 Контроль правил и границы проверки

Проверки хранения рассматривают издание с двумя свободными экземплярами. После выдач двум читателям остаток равен нулю, следующая выдача отклоняется. Возврат восстанавливает остаток до одного, повторный возврат не меняет его. Две выдачи подтверждают связь одного издания с разными читателями. Проверены также лимит выдач и почти одновременные обращения.

Файловая база проверяется после закрытия и повторного открытия. Для моделей представления проверяется отмена задач без зависшего состояния. Интерфейс проверяется в обычной и уменьшенной области с увеличенным шрифтом. Снимки показывают отдельные состояния настоящих компонентов Compose, но не заменяют испытания установки и всех возможных действий на каждой ОС.

Программа рассчитана на локальную учебную базу. Совместная работа по сети, отдельное управление учётными записями сотрудников, очередь бронирования и финансовые штрафы не реализованы. Экземпляры учитываются количеством без отдельного инвентарного номера каждого физического экземпляра.

ЗАКЛЮЧЕНИЕ

Разработано настольное приложение для организации взаимодействия объектов библиотечной информационной системы. Оно позволяет регистрировать читателей, учитывать книги и периодику, искать записи, оформлять выдачу и возврат и сохранять данные между запусками. Поставленная цель достигнута в пределах выбранного локального учебного варианта.

Предметная область представлена шестью классами сущностей. Общие свойства человека и издания вынесены в интерфейсы. Выдачи сохраняют историю и реализуют связь «многие ко многим». Для разделения создания объектов, правил, хранения и экрана применены семь описанных способов организации программы.

Преимущество реализации — согласованность выдач и остатка. Транзакции исключают частичные изменения, повторный возврат не увеличивает остаток, а защищённое удаление сохраняет историю. Формы сообщают ошибки без потери введённых значений. Учебные данные добавляются только по явной команде. Светлый интерфейс и русские подписи упрощают демонстрацию.

Выполнено 67 автоматических проверок без неуспешных и пропущенных тестов. Подтверждены перечисленные правила хранения и контрольные действия интерфейса. Программа ограничена локальной базой и количественным учётом экземпляров; установочные пакеты всех ОС не испытывались. Дальнейшее развитие возможно за счёт индивидуальных инвентарных номеров, управления сотрудниками и филиалами, расширенного поиска и бронирования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Объектно-ориентированное программирование: методические указания к выполнению курсового проекта / сост. Е. О. Шумова. — СПб.: ГУАП, 2026. — 16 с.

2. Паттерны объектно-ориентированного проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Д. Влиссидес. — СПб.: Питер, 2026. — 448 с. — ISBN 978-5-4461-1595-2.

3. Classes [Электронный ресурс] // Kotlin Documentation. URL: https://kotlinlang.org/docs/classes.html (дата обращения: 07.10.2026).

4. Interfaces [Электронный ресурс] // Kotlin Documentation. URL: https://kotlinlang.org/docs/interfaces.html (дата обращения: 07.10.2026).

5. Inheritance [Электронный ресурс] // Kotlin Documentation. URL: https://kotlinlang.org/docs/inheritance.html (дата обращения: 07.10.2026).

6. Class Diagram syntax and features [Электронный ресурс] // PlantUML Documentation. URL: https://plantuml.com/class-diagram (дата обращения: 07.10.2026).

7. Flows [Электронный ресурс] // Kotlin Documentation. URL: https://kotlinlang.org/docs/coroutines-flow.html (дата обращения: 07.10.2026).

8. Compose Multiplatform [Электронный ресурс] // JetBrains. URL: https://github.com/JetBrains/compose-multiplatform (дата обращения: 07.10.2026).

9. StateFlow [Электронный ресурс] // kotlinx.coroutines API. URL: https://kotlinlang.org/api/kotlinx.coroutines/kotlinx-coroutines-core/kotlinx.coroutines.flow/-state-flow (дата обращения: 07.10.2026).

10. Hieatt E., Mee R. Repository [Электронный ресурс] // Martin Fowler. — 2003. URL: https://martinfowler.com/eaaCatalog/repository.html (дата обращения: 07.10.2026).

11. Model-View-ViewModel [Электронный ресурс] // Microsoft Learn. URL: https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm (дата обращения: 07.10.2026).

12. Set up Room database for KMP [Электронный ресурс] // Android Developers. URL: https://developer.android.com/kotlin/multiplatform/room (дата обращения: 07.10.2026).

13. Gradle Wrapper [Электронный ресурс] // Gradle User Manual. URL: https://docs.gradle.org/current/userguide/gradle_wrapper.html (дата обращения: 07.10.2026).

14. SQLite Foreign Key Support [Электронный ресурс] // SQLite Documentation. URL: https://www.sqlite.org/foreignkeys.html (дата обращения: 07.10.2026).

15. Transaction [Электронный ресурс] // SQLite Documentation. URL: https://www.sqlite.org/lang_transaction.html (дата обращения: 07.10.2026).

16. kotlin-test [Электронный ресурс] // Kotlin Core API. URL: https://kotlinlang.org/api/core/kotlin-test (дата обращения: 07.10.2026).

ПРИЛОЖЕНИЕ А.

ИСХОДНЫЙ КОД ПРИЛОЖЕНИЯ

Ниже приведены полные тексты всех производственных Kotlin-файлов. Они включены напрямую из текущих исходников без пропусков, сокращённых методов и псевдокода. Настройки сборки, тесты, ресурсы и лицензии находятся в архиве. Автоматически созданный код Room и Compose в листинги не включён.

app/src/commonMain/kotlin/App.kt

```
import androidx.compose.runtime.Composable
import di.ApplicationContainer
import ui.App as LibraryApp

@Composable
fun App(container: ApplicationContainer) {
    LibraryApp(container)
}
```

app/src/commonMain/kotlin/di/ApplicationContainer.kt

```
package di

import application.RoomLibraryService
import persistence.LibraryDatabase
import persistence.RoomRepositoryBundle
import persistence.openDefaultDatabase

/** Composition root. All collaborators are explicit dependencies; there are no glob
al repositories. */
class ApplicationContainer(
    database: LibraryDatabase = openDefaultDatabase(),
) : AutoCloseable {
    private val closeables = LinkedHashSet<AutoCloseable>()
    private var closed = false
    val repositories: RoomRepositoryBundle = RoomRepositoryBundle(database)
    val service: RoomLibraryService = RoomLibraryService(repositories)

    fun register(closeable: AutoCloseable) {
        if (closed) closeable.close() else closeables += closeable
    }

    override fun close() {
        if (closed) return
        closed = true
        closeables.toList().asReversed().forEach(AutoCloseable::close)
        closeables.clear()
        repositories.database.close()
    }
}
```

app/src/commonMain/kotlin/ui/LibraryIcons.kt

```
package ui

import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.graphics.vector.PathParser
import androidx.compose.ui.unit.dp

/** Google Material Icons Outlined. Sources and Apache-2.0 notice are kept in the pr
oject. */
internal enum class LibraryIcon(private val paths: List<String>) {
    Group(listOf("M9 13.75c-2.34 0-7 1.17-7 3.5V19h14v-1.75c0-2.33-4.66-3.5-7-3.5zM4
.34 17c.84-.58 2.87-1.25 4.66-1.25s3.82.67 4.66 1.25H4.34zM9 12c1.93 0 3.5-1.57 3.5-
3.5S10.93 5 9 5 5.5 6.57 5.5 8.5 7.07 12 9 12zm0-5c.83 0 1.5.67 1.5 1.5S9.83 10 9 10
s-1.5-.67-1.5-1.5S8.17 7 9 7zm7.04 6.81c1.16.84 1.96 1.96 1.96 3.44V19h4v-1.75c0-2.0
2-3.5-3.17-5.96-3.44zM15 12c1.93 0 3.5-1.57 3.5-3.5S16.93 5 15 5c-.54 0-1.04.13-1.5.
35.63.89 1 1.98 1 3.15s-.37 2.26-1 3.15c.46.22.96.35 1.5.35z")),
    MenuBook(listOf("M21,5c-1.11-0.35-2.33-0.5-3.5-0.5c-1.95,0-4.05,0.4-5.5,1.5c-1.4
5-1.1-3.55-1.5-5.5-1.5S2.45,4.9,1,6v14.65 c0,0.25,0.25,0.5,0.5,0.5c0.1,0,0.15-0.05,0
.25-0.05C3.1,20.45,5.05,20,6.5,20c1.95,0,4.05,0.4,5.5,1.5c1.35-0.85,3.8-1.5,5.5-1.5
c1.65,0,3.35,0.3,4.75,1.05c0.1,0.05,0.15,0.05,0.25,0.05c0.25,0,0.5-0.25,0.5-0.5V6C22
.4,5.55,21.75,5.25,21,5z M21,18.5 c-1.1-0.35-2.3-0.5-3.5-0.5c-1.7,0-4.15,0.65-5.5,1.
5V8c1.35-0.85,3.8-1.5,5.5-1.5c1.2,0,2.4,0.15,3.5,0.5V18.5z", "M17.5,10.5c0.88,0,1.73
,0.09,2.5,0.26V9.24C19.21,9.09,18.36,9,17.5,9c-1.7,0-3.24,0.29-4.5,0.83v1.66 C14.13,
10.85,15.7,10.5,17.5,10.5z", "M13,12.49v1.66c1.13-0.64,2.7-0.99,4.5-0.99c0.88,0,1.73
,0.09,2.5,0.26V11.9c-0.79-0.15-1.64-0.24-2.5-0.24 C15.8,11.66,14.26,11.96,13,12.49z"
, "M17.5,14.33c-1.7,0-3.24,0.29-4.5,0.83v1.66c1.13-0.64,2.7-0.99,4.5-0.99c0.88,0,1.7
3,0.09,2.5,0.26v-1.52 C19.21,14.41,18.36,14.33,17.5,14.33z")),
    SwapHoriz(listOf("M6.99 11L3 15l3.99 4v-3H14v-2H6.99v-3zM21 9l-3.99-4v3H10v2h7.0
1v3L21 9z")),
    AccountTree(listOf("M22,11V3h-7v3H9V3H2v8h7V8h2v10h4v3h7v-8h-7v3h-2V8h2v3H22z M7
,9H4V5h3V9z M17,15h3v4h-3V15z M17,5h3v4h-3V5z")),
    Search(listOf("M15.5 14h-.79l-.28-.27C15.41 12.59 16 11.11 16 9.5 16 5.91 13.09
3 9.5 3S3 5.91 3 9.5 5.91 16 9.5 16c1.61 0 3.09-.59 4.23-1.57l.27.28v.79l5 4.99L20.4
9 19l-4.99-5zm-6 0C7.01 14 5 11.99 5 9.5S7.01 5 9.5 5 14 7.01 14 9.5 11.99 14 9.5 14
z")),
    Add(listOf("M19 13h-6v6h-2v-6H5v-2h6V5h2v6h6v2z")),
    Close(listOf("M19 6.41L17.59 5 12 10.59 6.41 5 5 6.41 10.59 12 5 17.59 6.41 19 1
2 13.41 17.59 19 19 17.59 13.41 12 19 6.41z")),
    Delete(listOf("M16 9v10H8V9h8m-1.5-6h-5l-1 1H5v2h14V4h-3.5l-1-1zM18 7H6v12c0 1.1
.9 2 2 2h8c1.1 0 2-.9 2-2V7z")),
    CheckCircle(listOf("M12 2C6.48 2 2 6.48 2 12s4.48 10 10 10 10-4.48 10-10S17.52 2
12 2zm0 18c-4.41 0-8-3.59-8-8s3.59-8 8-8 8 3.59 8 8-3.59 8-8 8zm4.59-12.42L10 14.17l
-2.59-2.58L6 13l4 4 8-8z")),
    LibraryBooks(listOf("M4 6H2v14c0 1.1.9 2 2 2h14v-2H4V6zm16-4H8c-1.1 0-2 .9-2 2v1
2c0 1.1.9 2 2 2h12c1.1 0 2-.9 2-2V4c0-1.1-.9-2-2-2zm0 14H8V4h12v12zM10 9h8v2h-8zm0 3
h4v2h-4zm0-6h8v2h-8z")),
    ChevronRight(listOf("M10 6L8.59 7.41 13.17 12l-4.58 4.59L10 18l6-6-6-6z")),
    Info(listOf("M11 7h2v2h-2zm0 4h2v6h-2zm1-9C6.48 2 2 6.48 2 12s4.48 10 10 10 10-4
.48 10-10S17.52 2 12 2zm0 18c-4.41 0-8-3.59-8-8s3.59-8 8-8 8 3.59 8 8-3.59 8-8 8z"))
,
    Warning(listOf("M1 21h22L12 2 1 21zm12-3h-2v-2h2v2zm0-4h-2v-4h2v4z")),
    Refresh(listOf("M17.65 6.35C16.2 4.9 14.21 4 12 4c-4.42 0-7.99 3.58-7.99 8s3.57
8 7.99 8c3.73 0 6.84-2.55 7.73-6h-2.08c-.82 2.33-3.04 4-5.65 4-3.31 0-6-2.69-6-6s2.6
9-6 6-6c1.66 0 3.14.69 4.22 1.78L13 11h7V4l-2.35 2.35z"));
```

```
    val vector: ImageVector by lazy {
        ImageVector.Builder(name, 24.dp, 24.dp, 24f, 24f).apply {
            paths.forEach { data ->
                addPath(pathData = PathParser().parsePathString(data).toNodes(), fil
l = SolidColor(Color.Black))
            }
        }.build()
    }
}
```

app/src/commonMain/kotlin/ui/LibraryScreen.kt

```
package ui

import androidx.compose.foundation.BorderStroke
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.hoverable
import androidx.compose.foundation.interaction.MutableInteractionSource
import androidx.compose.foundation.interaction.collectIsHoveredAsState
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.LazyListScope
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.selection.selectable
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalDensity
import androidx.compose.ui.focus.onFocusChanged
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.semantics.Role
import androidx.compose.ui.semantics.contentDescription
import androidx.compose.ui.semantics.semantics
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.Dp
import androidx.compose.ui.unit.dp
import androidx.compose.ui.window.Dialog
import di.ApplicationContainer
import domain.*
import kotlinx.coroutines.flow.catch

internal fun resolveAvailableSelection(selectedId: String?, availableIds: Set<String
>): String? =
    selectedId?.takeIf(availableIds::contains)

@Composable
fun App(container: ApplicationContainer) {
    val readers = remember(container) { ReadersViewModel(container.repositories.read
ers, service = container.service) }
    val publications = remember(container) {
        PublicationsViewModel(container.repositories.publications, container.reposit
```

```
ories.branches, service = container.service)
    }
    val loans = remember(container) {
        LoansViewModel(container.service, container.repositories.loans, container.re
positories.readers,
            container.repositories.publications, container.repositories.librarians)
    }
    val administration = remember(container) { AdministrationViewModel(container.ser
vice) }
    val testData = remember(container) { TestDataViewModel(TestDataSeeder(container.
repositories)) }
    val readerOptions = remember(container) { container.repositories.readers.readers
.catch { emit(emptyList()) } }
    val publicationOptions = remember(container) { container.repositories.publicatio
ns.publications.catch { emit(emptyList()) } }
    val allReaders by readerOptions.collectAsState(emptyList())
    val allPublications by publicationOptions.collectAsState(emptyList())
    DisposableEffect(container) {
        container.register(readers)
        container.register(publications)
        container.register(loans)
        container.register(administration)
        container.register(testData)
        onDispose {
            readers.close()
            publications.close()
            loans.close()
            administration.close()
            testData.close()
        }
    }
    LibraryTheme {
        var tab by remember { mutableIntStateOf(0) }
        BoxWithConstraints(Modifier.fillMaxSize().background(LibraryDesign.backgroun
d)) {
            val narrow = maxWidth < 960.dp
            val navigationWidth = 208.dp * LocalDensity.current.fontScale.coerceAtLe
ast(1f)
            val pagePadding = if (maxWidth < 1080.dp) 24.dp else 32.dp
            Row(Modifier.fillMaxSize().padding(16.dp), horizontalArrangement = Arran
gement.spacedBy(16.dp)) {
                LibraryNavigation(tab, navigationWidth) { tab = it }
                Surface(Modifier.weight(1f).fillMaxHeight(), shape = RoundedCornerSh
ape(24.dp), color = LibraryDesign.surface) {
                    when (tab) {
                        0 -> ReadersPane(readers, testData, pagePadding, narrow)
                        1 -> PublicationsPane(publications, testData, pagePadding, n
arrow)
                        2 -> LoansPane(loans, allReaders, allPublications, pagePaddi
ng, narrow)
                        else -> ArchitecturePane(administration, pagePadding)
                    }
                }
            }
        }
    }
}
```

```
@Composable
private fun LibraryNavigation(selected: Int, width: Dp, onSelected: (Int) -> Unit)
{
    val icons = listOf(LibraryIcon.Group, LibraryIcon.MenuBook, LibraryIcon.SwapHori
z, LibraryIcon.AccountTree)
    Surface(Modifier.width(width).fillMaxHeight(), shape = RoundedCornerShape(24.dp)
, color = LibraryDesign.surface) {
        Column(Modifier.padding(16.dp)) {
            Row(Modifier.padding(vertical = 16.dp), verticalAlignment = Alignment.Ce
nterVertically,
                horizontalArrangement = Arrangement.spacedBy(10.dp)) {
                Glyph(LibraryIcon.LibraryBooks)
                Text("Библиотека", style = MaterialTheme.typography.titleMedium)
            }
            Spacer(Modifier.height(24.dp))
            LibraryDesign.navigationLabels.forEachIndexed { index, title ->
                val active = selected == index
                val interaction = remember { MutableInteractionSource() }
                val hovered by interaction.collectIsHoveredAsState()
                Surface(
                    modifier = Modifier.fillMaxWidth().heightIn(min = 48.dp).focusRi
ng()
                        .hoverable(interaction)
                        .selectable(active, role = Role.Tab, onClick = { onSelected(
index) }),
                    shape = RoundedCornerShape(16.dp),
                    color = if (active) LibraryDesign.primaryContainer else if (hove
red) LibraryDesign.control else Color.Transparent,
                ) {
                    Row(Modifier.padding(horizontal = 12.dp, vertical = 14.dp),
                        verticalAlignment = Alignment.CenterVertically, horizontalAr
rangement = Arrangement.spacedBy(12.dp)) {
                        Glyph(icons[index], tint = if (active) LibraryDesign.primary
else LibraryDesign.secondaryText)
                        Text(title, style = MaterialTheme.typography.bodySmall,
                            fontWeight = if (active) FontWeight.SemiBold else FontWe
ight.Normal, modifier = Modifier.weight(1f))
                        if (active) Box(Modifier.width(3.dp).height(20.dp).backgroun
d(LibraryDesign.primary, RoundedCornerShape(2.dp)))
                    }
                }
                Spacer(Modifier.height(6.dp))
            }
            Spacer(Modifier.weight(1f))
            HorizontalDivider(color = LibraryDesign.divider)
            Spacer(Modifier.height(16.dp))
            Text("Данные на компьютере", style = MaterialTheme.typography.bodySmall,
color = LibraryDesign.secondaryText)
        }
    }
}

@Composable
private fun PaneHeader(title: String, subtitle: String, action: @Composable () -> Un
it) {
    BoxWithConstraints(Modifier.fillMaxWidth()) {
```

```
        if (maxWidth < 650.dp) {
            Column(verticalArrangement = Arrangement.spacedBy(16.dp)) {
                HeaderText(title, subtitle)
                action()
            }
        } else {
            Row(Modifier.fillMaxWidth(), verticalAlignment = Alignment.CenterVertica
lly, horizontalArrangement = Arrangement.spacedBy(24.dp)) {
                Column(Modifier.weight(1f)) { HeaderText(title, subtitle) }
                action()
            }
        }
    }
}

@Composable
private fun HeaderText(title: String, subtitle: String) {
    Text(title, style = MaterialTheme.typography.headlineLarge)
    Spacer(Modifier.height(6.dp))
    Text(subtitle, style = MaterialTheme.typography.bodySmall, color = LibraryDesign
.secondaryText)
}

@Composable
private fun Catalogue(padding: Dp, content: LazyListScope.() -> Unit) {
    LazyColumn(Modifier.fillMaxSize(), contentPadding = PaddingValues(padding),
        verticalArrangement = Arrangement.spacedBy(16.dp), content = content)
}

@Composable
private fun ReadersPane(vm: ReadersViewModel, testData: TestDataViewModel, padding:
Dp, narrow: Boolean) {
    val testState = testData.state.collectAsState().value
    val testMessage = testData.message.collectAsState().value
    val state by vm.state.collectAsState()
    val save by vm.saveState.collectAsState()
    val query by vm.query.collectAsState()
    var editor by remember { mutableStateOf(false) }
    var deleted by remember { mutableStateOf<Reader?>(null) }
    Catalogue(padding) {
        item {
            PaneHeader("Читатели", "Регистрация и поиск читателей") {
                FlowRow(horizontalArrangement = Arrangement.spacedBy(12.dp), vertica
lArrangement = Arrangement.spacedBy(12.dp)) {
                    CatalogButton("Добавить читателя", LibraryIcon.Add) { editor = t
rue }
                    SecondaryButton("Тестовые читатели", !testState.isSaving, testDa
ta::addReaders)
                }
            }
        }
        item { SearchField(query, vm::setSearchQuery, "Поиск по имени") }
        if (testState.error != null) item { Notice("Не удалось добавить тестовые дан
ные", error = true) }
        testMessage?.takeIf { it.kind == "readers" }?.let { message -> item { Notice
(message.text) } }
        if (state.error != null && !editor) item { Notice(state.error!!, error = tru
```

```
e) }
        if (state.isLoading) item { Notice("Загружаем читателей", loading = true) }
        else if (state.items.isEmpty() && state.error == null) item {
            EmptyState(if (query.isBlank()) "Читателей пока нет" else "Ничего не най
дено",
                if (query.isBlank()) "Добавьте первого читателя с помощью кнопки выш
e." else "Попробуйте другое имя или очистите поиск.",
                if (query.isNotBlank()) ({ vm.setSearchQuery("") }) else null)
        }
        if (!state.isLoading && state.items.isNotEmpty()) {
            item { ListHeading("Читатели", state.items.size, query.isNotBlank()) }
            items(state.items, key = { it.id }) { reader ->
                RecordRow {
                    Column(Modifier.weight(1f), verticalArrangement = Arrangement.sp
acedBy(4.dp)) {
                        Text(reader.fullName, style = MaterialTheme.typography.title
Medium)
                        Text("${reader.birthYear} · ${reader.organization}", style =
MaterialTheme.typography.bodySmall,
                            color = LibraryDesign.secondaryText)
                        Text(reader.address, style = MaterialTheme.typography.bodySm
all, color = LibraryDesign.secondaryText)
                    }
                    QuietAction("Удалить", "Удалить читателя ${reader.fullName}", Li
braryIcon.Delete) { deleted = reader }
                }
            }
        }
    }
    if (editor) ReaderEditor(vm, save, narrow) { editor = false }
    deleted?.let { reader ->
        ConfirmDialog("Удалить читателя?", "${reader.fullName}. Если у читателя есть
история выдач, удаление будет запрещено.", "Удалить",
            onDismiss = { deleted = null }, onConfirm = { deleted = null; vm.delete(
reader) })
    }
}

@Composable
private fun ReaderEditor(vm: ReadersViewModel, save: SaveUiState, narrow: Boolean, o
nDismiss: () -> Unit) {
    var name by remember { mutableStateOf("") }
    var year by remember { mutableStateOf("") }
    var address by remember { mutableStateOf("") }
    var organization by remember { mutableStateOf("") }
    val revision = remember { save.revision }
    LaunchedEffect(save.revision) { if (save.revision > revision) onDismiss() }
    EditorDialog("Новый читатель", "Заполните сведения о читателе", save.isSaving, s
ave.error, onDismiss,
        onSave = { vm.add(name, year, address, organization) }) {
        FieldsRow(narrow) {
            Field("Полное имя", name, { name = it }, Modifier.weight(1f), !save.isSa
ving)
            Field("Год рождения", year, { year = it }, Modifier.weight(1f), !save.is
Saving)
        }
        Field("Адрес", address, { address = it }, enabled = !save.isSaving)
```

```
        Field("Организация", organization, { organization = it }, enabled = !save.is
Saving)
    }
}

@Composable
private fun PublicationsPane(vm: PublicationsViewModel, testData: TestDataViewModel,
padding: Dp, narrow: Boolean) {
    val testState = testData.state.collectAsState().value
    val testMessage = testData.message.collectAsState().value
    val state by vm.state.collectAsState()
    val save by vm.saveState.collectAsState()
    val branches by vm.branches.collectAsState()
    val query by vm.query.collectAsState()
    var editor by remember { mutableStateOf(false) }
    var deleted by remember { mutableStateOf<Publication?>(null) }
    Catalogue(padding) {
        item {
            PaneHeader("Издания", "Книги и периодика в библиотечном фонде") {
                FlowRow(horizontalArrangement = Arrangement.spacedBy(12.dp), vertica
lArrangement = Arrangement.spacedBy(12.dp)) {
                    CatalogButton("Добавить издание", LibraryIcon.Add) { editor = tr
ue }
                    SecondaryButton("Тестовые книги", !testState.isSaving, testData:
:addBooks)
                }
            }
        }
        item { SearchField(query, vm::setSearchQuery, "Поиск по названию или автору"
) }
        if (testState.error != null) item { Notice("Не удалось добавить тестовые дан
ные", error = true) }
        testMessage?.takeIf { it.kind == "books" }?.let { message -> item { Notice(m
essage.text) } }
        val error = state.error ?: branches.error
        if (error != null && !editor) item { Notice(error, error = true) }
        if (state.isLoading) item { Notice("Загружаем издания", loading = true) }
        else if (state.items.isEmpty() && error == null) item {
            EmptyState(if (query.isBlank()) "Изданий пока нет" else "Ничего не найде
но",
                if (query.isBlank()) "Добавьте книгу или периодическое издание." els
e "Попробуйте другое название или имя автора.",
                if (query.isNotBlank()) ({ vm.setSearchQuery("") }) else null)
        }
        if (!state.isLoading && state.items.isNotEmpty()) {
            item { ListHeading("Каталог", state.items.size, query.isNotBlank()) }
            items(state.items, key = { it.id }) { publication ->
                RecordRow {
                    PublicationCover(publication)
                    Column(Modifier.weight(1f), verticalArrangement = Arrangement.sp
acedBy(4.dp)) {
                        Text(publication.title, style = MaterialTheme.typography.tit
leMedium)
                        Text(publication.authors, style = MaterialTheme.typography.b
odySmall, color = LibraryDesign.secondaryText)
                        val kind = when (publication) {
                            is Book -> "Книга"
```

```
                            is Periodical -> "Периодика № ${publication.issueNumber}
"
                            else -> "Издание"
                        }
                        Text("$kind · ${publication.publisher} · ${publication.publi
cationYear}",
                            style = MaterialTheme.typography.bodySmall, color = Libr
aryDesign.secondaryText)
                        if (publication is Book && !publication.isbn.isNullOrBlank()
) {
                            Text("ISBN ${publication.isbn}", style = MaterialTheme.t
ypography.bodySmall, color = LibraryDesign.secondaryText)
                        }
                        val branchName = branches.items.firstOrNull { it.id == publi
cation.branchId }?.name ?: "Филиал недоступен"
                        Text("$branchName · Свободно ${publication.availableCopies}
из ${publication.totalCopies}", style = MaterialTheme.typography.bodySmall)
                    }
                    QuietAction("Удалить", "Удалить издание ${publication.title}", L
ibraryIcon.Delete) { deleted = publication }
                }
            }
        }
    }
    if (editor) PublicationEditor(vm, save, branches.items, narrow) { editor = false
}
    deleted?.let { publication ->
        ConfirmDialog("Удалить издание?", "«${publication.title}». Издание с истори
й выдач удалить нельзя.", "Удалить",
            onDismiss = { deleted = null }, onConfirm = { deleted = null; vm.delete(
publication) })
    }
}

@Composable
private fun PublicationEditor(vm: PublicationsViewModel, save: SaveUiState, branches
: List<LibraryBranch>, narrow: Boolean, onDismiss: () -> Unit) {
    var periodical by remember { mutableStateOf(false) }
    var title by remember { mutableStateOf("") }
    var authors by remember { mutableStateOf("") }
    var publisher by remember { mutableStateOf("") }
    var year by remember { mutableStateOf("") }
    var copies by remember { mutableStateOf("") }
    var isbn by remember { mutableStateOf("") }
    var issue by remember { mutableStateOf("1") }
    var branchId by remember { mutableStateOf<String?>(null) }
    val validBranch = branches.firstOrNull { it.id == branchId }
    val revision = remember { save.revision }
    LaunchedEffect(save.revision) { if (save.revision > revision) onDismiss() }
    EditorDialog("Новое издание", "Библиографические сведения и место хранения", sav
e.isSaving, save.error, onDismiss,
        saveEnabled = validBranch != null && !save.isSaving,
        onSave = {
            if (periodical) vm.addPeriodical(title, authors, publisher, year, issue,
copies, validBranch?.id ?: "")
            else vm.addBook(title, authors, publisher, year, copies, validBranch?.id
?: "", isbn)
```

```
        }) {
        Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
            FilterChip(selected = !periodical, onClick = { periodical = false }, ena
bled = !save.isSaving,
                label = { Text("Книга") }, modifier = Modifier.heightIn(min = 48.dp)
)
            FilterChip(selected = periodical, onClick = { periodical = true }, enabl
ed = !save.isSaving,
                label = { Text("Периодика") }, modifier = Modifier.heightIn(min = 48
.dp))
        }
        Field("Название", title, { title = it }, enabled = !save.isSaving)
        Field(if (periodical) "Редакторы" else "Авторы", authors, { authors = it },
enabled = !save.isSaving)
        FieldsRow(narrow) {
            Field("Издательство", publisher, { publisher = it }, Modifier.weight(1f)
, !save.isSaving)
            Field("Год издания", year, { year = it }, Modifier.weight(1f), !save.isS
aving)
        }
        FieldsRow(narrow) {
            Field("Экземпляры", copies, { copies = it }, Modifier.weight(1f), !save.
isSaving)
            if (periodical) Field("Номер выпуска", issue, { issue = it }, Modifier.w
eight(1f), !save.isSaving)
            else Field("ISBN (необязательно)", isbn, { isbn = it }, Modifier.weight(
1f), !save.isSaving)
        }
        ChoiceField("Филиал", validBranch?.name, branches.map { it.id to it.name },
!save.isSaving) { branchId = it }
        if (branches.isEmpty()) Notice("Филиалов пока нет. Загрузите демонстрационны
е данные в разделе «Архитектура».")
    }
}

@Composable
private fun LoansPane(vm: LoansViewModel, readers: List<Reader>, publications: List<
Publication>, padding: Dp, narrow: Boolean) {
    val state by vm.state.collectAsState()
    val action by vm.actionState.collectAsState()
    var editor by remember { mutableStateOf(false) }
    Catalogue(padding) {
        item {
            PaneHeader("Выдачи", "Выдача, возврат и история пользования") {
                CatalogButton("Оформить выдачу", LibraryIcon.Add, enabled = !state.i
sLoading && !action.isSaving) { editor = true }
            }
        }
        if (state.error != null && !editor) item { Notice(state.error!!, error = tru
e) }
        if (state.isLoading) item { Notice("Загружаем выдачи", loading = true) }
        else if (state.items.isEmpty() && state.error == null) item { EmptyState("Вы
дач пока нет", "Выберите читателя и доступное издание, чтобы оформить первую выдачу.
") }
        if (!state.isLoading && state.items.isNotEmpty()) {
            item { ListHeading("История выдач", state.items.size) }
            items(state.items, key = { it.loan.id }) { row ->
```

```
                RecordRow {
                    Column(Modifier.weight(1f), verticalArrangement = Arrangement.sp
acedBy(6.dp)) {
                        Text(row.publicationTitle, style = MaterialTheme.typography.
titleMedium)
                        Text(row.readerName, style = MaterialTheme.typography.bodySm
all, color = LibraryDesign.secondaryText)
                        Text("Вернуть до ${formatLoanDate(row.loan.dueAt)}", style =
MaterialTheme.typography.bodySmall, color = LibraryDesign.secondaryText)
                        LoanBadge(row.loan.state)
                    }
                    SecondaryButton("Вернуть", enabled = row.loan.state !is LoanStat
e.Returned && !action.isSaving) { vm.returnLoan(row.loan) }
                }
            }
        }
    }
    if (editor) LoanEditor(vm, action, readers, publications, narrow) { editor = fal
se }
}

@Composable
private fun LoanEditor(vm: LoansViewModel, action: SaveUiState, readers: List<Reader
>, publications: List<Publication>, narrow: Boolean, onDismiss: () -> Unit) {
    var readerId by remember { mutableStateOf<String?>(null) }
    var publicationId by remember { mutableStateOf<String?>(null) }
    val available = publications.filter { it.availableCopies > 0 }
    val validReaderId = resolveAvailableSelection(readerId, readers.mapTo(mutableSet
Of()) { it.id })
    val validPublicationId = resolveAvailableSelection(publicationId, available.mapT
o(mutableSetOf()) { it.id })
    val reader = readers.firstOrNull { it.id == validReaderId }
    val publication = available.firstOrNull { it.id == validPublicationId }
    LaunchedEffect(validReaderId) { readerId = validReaderId }
    LaunchedEffect(validPublicationId) { publicationId = validPublicationId }
    val revision = remember { action.revision }
    LaunchedEffect(action.revision) { if (action.revision > revision) onDismiss() }
    EditorDialog("Новая выдача", "Выберите читателя и доступное издание", action.isS
aving, action.error, onDismiss,
        saveLabel = "Выдать", saveEnabled = reader != null && publication != null &&
!action.isSaving,
        onSave = { vm.issue(reader, publication) }) {
        ChoiceField("Читатель", reader?.fullName, readers.map { it.id to it.fullName
}, !action.isSaving) { readerId = it }
        ChoiceField("Издание", publication?.title, available.map { it.id to it.title
}, !action.isSaving) { publicationId = it }
        if (readers.isEmpty()) Notice("Сначала зарегистрируйте читателя.")
        if (available.isEmpty()) Notice("Нет свободных экземпляров. Добавьте издание
или оформите возврат.")
        if (publication != null) Text("Свободных экземпляров: ${publication.availabl
eCopies}", style = MaterialTheme.typography.bodySmall,
            color = LibraryDesign.secondaryText)
        Text("Срок возврата определяется правилами библиотеки. Идентификаторы вводит
ь не нужно.",
            style = MaterialTheme.typography.bodySmall, color = LibraryDesign.second
aryText)
    }
```

```
}

@Composable
private fun LoanBadge(state: LoanState) {
    val label: String
    val background: Color
    val foreground: Color
    when (state) {
        is LoanState.Active -> { label = "На руках"; background = LibraryDesign.cont
rol; foreground = LibraryDesign.primary }
        is LoanState.Overdue -> { label = "Просрочено"; background = LibraryDesign.w
arningContainer; foreground = LibraryDesign.warning }
        is LoanState.Returned -> { label = "Возвращено"; background = LibraryDesign.
successContainer; foreground = LibraryDesign.success }
    }
    Surface(color = background, shape = RoundedCornerShape(100.dp)) {
        Text(label, Modifier.padding(horizontal = 12.dp, vertical = 6.dp), style = M
aterialTheme.typography.bodySmall, color = foreground)
    }
}

@Composable
private fun ArchitecturePane(vm: AdministrationViewModel, padding: Dp) {
    val state by vm.state.collectAsState()
    var clearConfirmation by remember { mutableStateOf(false) }
    Catalogue(padding) {
        item { PaneHeader("Архитектура", "Как устроена библиотечная программа") {}
}
        item {
            InformationBlock("Основные объекты", LibraryIcon.AccountTree) {
                Explanation("Люди", "У читателя и библиотекаря есть общие сведения о
человеке. В программе: Person → Reader / Librarian.")
                Explanation("Издания", "Книга и периодика имеют общие поля, но отлич
аются дополнительными сведениями. В программе: Publication → Book / Periodical.")
                Explanation("Выдача", "Запись соединяет читателя и издание, хранит с
рок и состояние. Через несколько выдач читатели связаны с разными изданиями: Reader
↔ Loan ↔ Publication.")
            }
        }
        item {
            InformationBlock("Разделение обязанностей", LibraryIcon.Info) {
                Text("Экран показывает сведения, модель представления обрабатывает д
ействия, служба выполняет библиотечные операции, а хранилище работает с базой.")
                Text("Compose → ViewModel → LibraryService → Repository → DAO → SQLi
te", style = MaterialTheme.typography.bodySmall, color = LibraryDesign.secondaryText
)
                Explanation("Шаблоны проектирования", "Repository, MVVM, Strategy, F
actory, Observer, State и Facade помогают разделить эти задачи и не смешивать правил
а выдачи с оформлением окна.")
            }
        }
        item {
            InformationBlock("Демонстрационные данные", LibraryIcon.LibraryBooks) {
                Text("Учебный набор содержит филиал, библиотекаря, читателей, книгу,
периодику и выдачу. Повторная загрузка не создаёт дубликаты.")
                CatalogButton("Добавить демоданные", LibraryIcon.Add, !state.isBusy,
vm::seedDemo)
```

```
                when {
                    state.isBusy -> Notice("Выполняем операцию", loading = true)
                    state.error != null -> Notice(state.error!!, error = true)
                    state.message != null -> Notice(state.message!!)
                }
            }
        }
        item {
            Surface(Modifier.fillMaxWidth(), shape = RoundedCornerShape(16.dp), bord
er = BorderStroke(1.dp, LibraryDesign.divider)) {
                Column(Modifier.padding(24.dp), verticalArrangement = Arrangement.sp
acedBy(12.dp)) {
                    Text("Очистка учебной базы", style = MaterialTheme.typography.ti
tleMedium)
                    Text("Удалит читателей, издания, сотрудников, филиалы и историю
выдач. Это действие нельзя отменить.",
                        style = MaterialTheme.typography.bodySmall, color = LibraryD
esign.secondaryText)
                    QuietAction("Очистить данные", "Очистить все данные библиотеки",
LibraryIcon.Delete, !state.isBusy) { clearConfirmation = true }
                }
            }
        }
    }
    if (clearConfirmation) ConfirmDialog("Очистить базу?", "Будут удалены все записи
библиотеки, включая историю выдач. Отменить это действие нельзя.", "Очистить",
        onDismiss = { clearConfirmation = false }, onConfirm = { clearConfirmation =
false; vm.clearAll() })
}

@Composable
private fun InformationBlock(title: String, icon: LibraryIcon, content: @Composable
ColumnScope.() -> Unit) {
    Surface(Modifier.fillMaxWidth(), color = LibraryDesign.control, shape = RoundedC
ornerShape(16.dp)) {
        Column(Modifier.padding(24.dp), verticalArrangement = Arrangement.spacedBy(1
6.dp)) {
            Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangemen
t = Arrangement.spacedBy(12.dp)) {
                Glyph(icon)
                Text(title, style = MaterialTheme.typography.titleLarge)
            }
            content()
        }
    }
}

@Composable
private fun Explanation(title: String, description: String) {
    Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
        Text(title, style = MaterialTheme.typography.titleMedium)
        Text(description, style = MaterialTheme.typography.bodySmall, color = Librar
yDesign.secondaryText)
    }
}

@Composable
```

```
private fun RecordRow(content: @Composable RowScope.() -> Unit) {
    val interaction = remember { MutableInteractionSource() }
    val hovered by interaction.collectIsHoveredAsState()
    Column {
        Row(Modifier.fillMaxWidth().heightIn(min = 64.dp).hoverable(interaction)
            .background(if (hovered) LibraryDesign.control else Color.Transparent, R
oundedCornerShape(16.dp))
            .padding(horizontal = 12.dp, vertical = 16.dp), verticalAlignment = Alig
nment.CenterVertically,
            horizontalArrangement = Arrangement.spacedBy(16.dp), content = content)
        HorizontalDivider(color = LibraryDesign.divider)
    }
}

@Composable
private fun ListHeading(label: String, count: Int, filtered: Boolean = false) {
    Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.SpaceBetween)
{
        Text(label, style = MaterialTheme.typography.titleSmall, color = LibraryDesi
gn.secondaryText)
        Text(if (filtered) "Найдено: $count" else "Записей: $count", style = Materia
lTheme.typography.bodySmall, color = LibraryDesign.secondaryText)
    }
}

@Composable
private fun SearchField(query: String, onQuery: (String) -> Unit, label: String) {
    OutlinedTextField(query, onQuery, modifier = Modifier.fillMaxWidth().semantics {
contentDescription = label }, label = { Text(label) }, singleLine = true,
        shape = RoundedCornerShape(12.dp), leadingIcon = { Glyph(LibraryIcon.Search,
tint = LibraryDesign.secondaryText) },
        trailingIcon = if (query.isNotEmpty()) ({ IconButton(onClick = { onQuery("")
}, modifier = Modifier.focusRing()) {
            Glyph(LibraryIcon.Close, "Очистить поиск")
        } }) else null)
}

@Composable
private fun Field(label: String, value: String, onValue: (String) -> Unit, modifier:
Modifier = Modifier, enabled: Boolean = true) {
    OutlinedTextField(value, onValue, modifier.fillMaxWidth().semantics { contentDes
cription = label }, enabled = enabled, label = { Text(label) }, singleLine = true,
        shape = RoundedCornerShape(12.dp))
}

@Composable
private fun FieldsRow(narrow: Boolean, content: @Composable RowScope.() -> Unit) {
    if (narrow) {
        FlowRow(Modifier.fillMaxWidth(), verticalArrangement = Arrangement.spacedBy(
12.dp),
            maxItemsInEachRow = 1, content = content)
    } else {
        Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(12
.dp), content = content)
    }
}
```

```
@Composable
private fun ChoiceField(label: String, selected: String?, options: List<Pair<String,
String>>, enabled: Boolean = true, onSelected: (String) -> Unit) {
    var expanded by remember { mutableStateOf(false) }
    Column(verticalArrangement = Arrangement.spacedBy(6.dp)) {
        Text(label, style = MaterialTheme.typography.bodySmall, color = LibraryDesig
n.secondaryText)
        Box {
            OutlinedButton(onClick = { expanded = true }, enabled = enabled && optio
ns.isNotEmpty(),
                modifier = Modifier.fillMaxWidth().heightIn(min = 48.dp).focusRing(1
2.dp), shape = RoundedCornerShape(12.dp),
                contentPadding = PaddingValues(12.dp)) {
                Text(selected ?: "Выберите: ${label.lowercase()}", modifier = Modifi
er.weight(1f), style = MaterialTheme.typography.bodyMedium)
                Glyph(LibraryIcon.ChevronRight, tint = LibraryDesign.secondaryText)
            }
            DropdownMenu(expanded, { expanded = false }, modifier = Modifier.widthIn
(max = 500.dp).heightIn(max = 280.dp),
                containerColor = LibraryDesign.surface) {
                options.forEach { (id, title) ->
                    DropdownMenuItem(text = { Text(title, style = MaterialTheme.typo
graphy.bodySmall) },
                        onClick = { expanded = false; onSelected(id) }, modifier = M
odifier.heightIn(min = 48.dp))
                }
            }
        }
    }
}

@Composable
private fun EditorDialog(title: String, subtitle: String, busy: Boolean, error: Stri
ng?, onDismiss: () -> Unit,
                         saveLabel: String = "Сохранить", saveEnabled: Boolean = !bu
sy, onSave: () -> Unit,
                         content: @Composable ColumnScope.() -> Unit) {
    Dialog(onDismissRequest = { if (!busy) onDismiss() }) {
        Surface(Modifier.width(600.dp).heightIn(max = 700.dp), shape = RoundedCorner
Shape(24.dp), color = LibraryDesign.surface) {
            Column(Modifier.padding(24.dp), verticalArrangement = Arrangement.spaced
By(16.dp)) {
                Row(verticalAlignment = Alignment.CenterVertically) {
                    Column(Modifier.weight(1f)) {
                        Text(title, style = MaterialTheme.typography.titleLarge)
                        Text(subtitle, style = MaterialTheme.typography.bodySmall, c
olor = LibraryDesign.secondaryText)
                    }
                    IconButton(onClick = onDismiss, enabled = !busy, modifier = Modi
fier.focusRing()) { Glyph(LibraryIcon.Close, "Закрыть форму") }
                }
                Column(Modifier.weight(1f, fill = false).verticalScroll(rememberScro
llState()), verticalArrangement = Arrangement.spacedBy(12.dp)) {
                    content()
                    if (error != null) Notice(error, error = true)
                }
                Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spa
```

```
cedBy(12.dp, Alignment.End)) {
                    SecondaryButton("Отмена", enabled = !busy, onClick = onDismiss)
                    CatalogButton(if (busy) "Сохраняем…" else saveLabel, enabled = s
aveEnabled && !busy, onClick = onSave)
                }
            }
        }
    }
}

@Composable
private fun ConfirmDialog(title: String, description: String, action: String, onDism
iss: () -> Unit, onConfirm: () -> Unit) {
    AlertDialog(onDismissRequest = onDismiss, title = { Text(title) }, text = { Text
(description) },
        confirmButton = { CatalogButton(action, onClick = onConfirm) },
        dismissButton = { SecondaryButton("Отмена", onClick = onDismiss) },
        containerColor = LibraryDesign.surface, shape = RoundedCornerShape(24.dp))
}

@Composable
private fun CatalogButton(label: String, icon: LibraryIcon? = null, enabled: Boolean
= true, onClick: () -> Unit) {
    val interaction = remember { MutableInteractionSource() }
    val hovered by interaction.collectIsHoveredAsState()
    Button(onClick, enabled = enabled, interactionSource = interaction,
        modifier = Modifier.heightIn(min = 48.dp).hoverable(interaction, enabled).fo
cusRing(), shape = RoundedCornerShape(16.dp),
        contentPadding = PaddingValues(horizontal = 16.dp, vertical = 12.dp),
        colors = ButtonDefaults.buttonColors(containerColor = if (hovered) LibraryDe
sign.primaryHover else LibraryDesign.primary, contentColor = Color.White,
            disabledContainerColor = LibraryDesign.disabled, disabledContentColor =
LibraryDesign.secondaryText)) {
        if (icon != null) { Glyph(icon, tint = if (enabled) Color.White else Library
Design.secondaryText); Spacer(Modifier.width(8.dp)) }
        Text(label)
    }
}

@Composable
private fun SecondaryButton(label: String, enabled: Boolean = true, onClick: () -> U
nit) {
    val interaction = remember { MutableInteractionSource() }
    val hovered by interaction.collectIsHoveredAsState()
    Button(onClick, enabled = enabled, interactionSource = interaction,
        modifier = Modifier.heightIn(min = 48.dp).hoverable(interaction, enabled).fo
cusRing(), shape = RoundedCornerShape(16.dp),
        contentPadding = PaddingValues(horizontal = 16.dp, vertical = 12.dp),
        colors = ButtonDefaults.buttonColors(containerColor = if (hovered) LibraryDe
sign.controlHover else LibraryDesign.control, contentColor = LibraryDesign.primary,
            disabledContainerColor = LibraryDesign.disabled, disabledContentColor =
LibraryDesign.secondaryText)) { Text(label) }
}

@Composable
private fun QuietAction(label: String, description: String, icon: LibraryIcon, enabl
ed: Boolean = true, onClick: () -> Unit) {
```

```
    TextButton(onClick, enabled = enabled, modifier = Modifier.heightIn(min = 48.dp)
.focusRing().semantics { contentDescription = description },
        colors = ButtonDefaults.textButtonColors(contentColor = LibraryDesign.second
aryText)) {
        Glyph(icon, tint = LibraryDesign.secondaryText)
        Spacer(Modifier.width(8.dp))
        Text(label, style = MaterialTheme.typography.bodySmall)
    }
}

@Composable
private fun Glyph(icon: LibraryIcon, description: String? = null, tint: Color = Libr
aryDesign.primary) {
    Icon(icon.vector, contentDescription = description, modifier = Modifier.size(24.
dp), tint = tint)
}

@Composable
private fun Modifier.focusRing(radius: Dp = 16.dp): Modifier {
    var focused by remember { mutableStateOf(false) }
    return onFocusChanged { focused = it.isFocused }
        .border(if (focused) 2.dp else 0.dp, if (focused) LibraryDesign.primary else
Color.Transparent, RoundedCornerShape(radius))
}

@Composable
private fun Notice(text: String, loading: Boolean = false, error: Boolean = false)
{
    Surface(Modifier.fillMaxWidth(), shape = RoundedCornerShape(12.dp), color = if (
error) LibraryDesign.errorContainer else LibraryDesign.control) {
        Row(Modifier.padding(16.dp), verticalAlignment = Alignment.CenterVertically,
horizontalArrangement = Arrangement.spacedBy(12.dp)) {
            if (loading) CircularProgressIndicator(Modifier.size(20.dp), strokeWidth
= 2.dp)
            else Glyph(if (error) LibraryIcon.Warning else LibraryIcon.Info, tint =
if (error) LibraryDesign.error else LibraryDesign.secondaryText)
            Text(text, style = MaterialTheme.typography.bodySmall, color = if (error
) LibraryDesign.error else LibraryDesign.secondaryText)
        }
    }
}

@Composable
private fun EmptyState(title: String, description: String, clearSearch: (() -> Unit)
? = null) {
    Column(Modifier.fillMaxWidth().padding(vertical = 32.dp), verticalArrangement =
Arrangement.spacedBy(12.dp)) {
        Glyph(LibraryIcon.Search, tint = LibraryDesign.secondaryText)
        Text(title, style = MaterialTheme.typography.titleLarge)
        Text(description, style = MaterialTheme.typography.bodySmall, color = Librar
yDesign.secondaryText)
        if (clearSearch != null) SecondaryButton("Очистить поиск", onClick = clearSe
arch)
    }
}
```

app/src/commonMain/kotlin/ui/LibraryTheme.kt

```
package ui

import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.LocalRippleConfiguration
import androidx.compose.material3.RippleConfiguration
import androidx.compose.material.ripple.RippleAlpha
import androidx.compose.material3.Shapes
import androidx.compose.material3.Typography
import androidx.compose.material3.lightColorScheme
import androidx.compose.runtime.Composable
import androidx.compose.runtime.CompositionLocalProvider
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import org.jetbrains.compose.resources.Font
import ui.resources.Res
import ui.resources.inter_medium
import ui.resources.inter_regular
import ui.resources.inter_semibold

/** Application-owned tokens from Design.md, not official Yandex tokens. */
internal object LibraryDesign {
    val primary = Color(0xFF242432)
    val primaryHover = Color(0xFF101018)
    val primaryContainer = Color(0xFFEBEBF2)
    val background = Color(0xFFF7F7FA)
    val surface = Color.White
    val secondaryText = Color(0xFF626272)
    val control = Color(0xFFF4F4F8)
    val controlHover = Color(0xFFEAEAF1)
    val outline = Color(0xFF858594)
    val divider = Color(0xFFE5E5EC)
    val success = Color(0xFF1E6A43)
    val successContainer = Color(0xFFE9F7EF)
    val warning = Color(0xFF7A4B00)
    val warningContainer = Color(0xFFFFFF3DC)
    val error = Color(0xFFA12323)
    val errorContainer = Color(0xFFFFFF0F0)
    val disabled = Color(0xFFEDEDF1)
    val navigationLabels = listOf("Читатели", "Издания", "Выдачи", "Архитектура")
    const val windowWidthDp = 1180
    const val windowHeightDp = 780
}

@Composable
@OptIn(ExperimentalMaterial3Api::class)
internal fun LibraryTheme(content: @Composable () -> Unit) {
    val inter = FontFamily(
        Font(Res.font.inter_regular, weight = FontWeight.Normal),
        Font(Res.font.inter_medium, weight = FontWeight.Medium),
```

```
        Font(Res.font.inter_semibold, weight = FontWeight.SemiBold),
    )
    val colors = lightColorScheme(
        primary = LibraryDesign.primary,
        onPrimary = Color.White,
        primaryContainer = LibraryDesign.primaryContainer,
        onPrimaryContainer = LibraryDesign.primary,
        secondary = LibraryDesign.secondaryText,
        onSecondary = Color.White,
        secondaryContainer = LibraryDesign.control,
        onSecondaryContainer = LibraryDesign.primary,
        tertiary = LibraryDesign.success,
        onTertiary = Color.White,
        tertiaryContainer = LibraryDesign.successContainer,
        onTertiaryContainer = LibraryDesign.success,
        error = LibraryDesign.error,
        onError = Color.White,
        errorContainer = LibraryDesign.errorContainer,
        onErrorContainer = LibraryDesign.error,
        background = LibraryDesign.background,
        onBackground = LibraryDesign.primary,
        surface = LibraryDesign.surface,
        onSurface = LibraryDesign.primary,
        surfaceVariant = LibraryDesign.control,
        onSurfaceVariant = LibraryDesign.secondaryText,
        outline = LibraryDesign.outline,
        outlineVariant = LibraryDesign.divider,
        surfaceTint = Color.White,
    )
    // The design owns hover colors; a second Material hover overlay would alter the
tokens.
    CompositionLocalProvider(LocalRippleConfiguration provides RippleConfiguration(
        rippleAlpha = RippleAlpha(draggedAlpha = 0.16f, focusedAlpha = 0.12f, hovere
dAlpha = 0f, pressedAlpha = 0.12f),
    )) {
        MaterialTheme(
            colorScheme = colors,
            typography = Typography(
                headlineLarge = TextStyle(fontFamily = inter, fontSize = 32.sp, line
Height = 40.sp, fontWeight = FontWeight.SemiBold),
                headlineMedium = TextStyle(fontFamily = inter, fontSize = 24.sp, lin
eHeight = 32.sp, fontWeight = FontWeight.SemiBold),
                titleLarge = TextStyle(fontFamily = inter, fontSize = 20.sp, lineHei
ght = 26.sp, fontWeight = FontWeight.SemiBold),
                titleMedium = TextStyle(fontFamily = inter, fontSize = 16.sp, lineHe
ight = 24.sp, fontWeight = FontWeight.SemiBold),
                titleSmall = TextStyle(fontFamily = inter, fontSize = 14.sp, lineHei
ght = 20.sp, fontWeight = FontWeight.Medium),
                bodyLarge = TextStyle(fontFamily = inter, fontSize = 16.sp, lineHeig
ht = 24.sp),
                bodyMedium = TextStyle(fontFamily = inter, fontSize = 16.sp, lineHei
ght = 24.sp),
                bodySmall = TextStyle(fontFamily = inter, fontSize = 14.sp, lineHeig
ht = 20.sp),
                labelLarge = TextStyle(fontFamily = inter, fontSize = 15.sp, lineHei
ght = 20.sp, fontWeight = FontWeight.SemiBold),
                labelMedium = TextStyle(fontFamily = inter, fontSize = 14.sp, lineHe
```

```
ight = 20.sp, fontWeight = FontWeight.Medium),
                labelSmall = TextStyle(fontFamily = inter, fontSize = 12.sp, lineHei
ght = 16.sp),
            ),
            shapes = Shapes(
                extraSmall = RoundedCornerShape(8.dp),
                small = RoundedCornerShape(12.dp),
                medium = RoundedCornerShape(16.dp),
                large = RoundedCornerShape(24.dp),
                extraLarge = RoundedCornerShape(24.dp),
            ),
            content = content,
        )
    }
}
```

app/src/commonMain/kotlin/ui/LibraryViewModels.kt

```
package ui

import domain.Book
import domain.IdGenerator
import domain.LibraryBranch
import domain.LibraryService
import domain.Loan
import domain.Librarian
import domain.Periodical
import domain.Publication
import domain.Reader
import domain.UuidIdGenerator
import kotlinx.coroutines.CancellationException
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.SupervisorJob
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.SharingStarted
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.catch
import kotlinx.coroutines.flow.combine
import kotlinx.coroutines.flow.first
import kotlinx.coroutines.flow.map
import kotlinx.coroutines.flow.stateIn
import kotlinx.coroutines.cancel
import kotlinx.coroutines.delay
import kotlinx.coroutines.isActive
import kotlinx.coroutines.launch
import persistence.BranchRepository
import persistence.LibrarianRepository
import persistence.LoanRepository
import persistence.PublicationRepository
import persistence.ReaderRepository

private inline fun <T> runCatchingCancellable(block: () -> T): Result<T> = try {
    Result.success(block())
} catch (error: CancellationException) {
```

```
    throw error
} catch (error: Throwable) {
    Result.failure(error)
}

data class CollectionUiState<T>(
    val items: List<T> = emptyList(),
    val isLoading: Boolean = true,
    val error: String? = null,
)

class ReadersViewModel(
    private val repository: ReaderRepository,
    private val ids: IdGenerator = UuidIdGenerator(),
    private val service: LibraryService? = null,
    private val scope: CoroutineScope = CoroutineScope(SupervisorJob() + Dispatchers
.Default),
) : AutoCloseable {
    private val errors = MutableStateFlow<String?>(null)
    private val saveOperation = SaveOperation(scope) { errors.value = it }
    val saveState: StateFlow<SaveUiState> = saveOperation.state
    private val searchQuery = MutableStateFlow("")
    val query: StateFlow<String> = searchQuery.asStateFlow()
    val state: StateFlow<CollectionUiState<Reader>> = combine(repository.readers, se
archQuery) { readers, query ->
        CollectionUiState(
            items = readers.filter { it.fullName.contains(query.trim(), ignoreCase =
true) },
            isLoading = false,
        )
    }
        .catch { emit(CollectionUiState(isLoading = false, error = "Не удалось загру
зить читателей")) }
        .combine(errors) { value, error -> value.copy(error = error ?: value.error)
}
        .stateIn(scope, SharingStarted.Eagerly, CollectionUiState())

    fun setSearchQuery(value: String) {
        searchQuery.value = value
    }

    fun add(fullName: String, birthYear: String, address: String, organization: Stri
ng): Boolean {
        val year = birthYear.toIntOrNull()
        if (fullName.isBlank() || year == null || address.isBlank() || organization.
isBlank()) {
            saveOperation.reject("Введите имя, корректный год рождения, адрес и орга
низацию")
            return false
        }
        return saveOperation.submit {
            repository.save(Reader(ids.nextId(), fullName, year, address, organizati
on))
        }
    }

    fun delete(reader: Reader) {
```

```
        scope.launch {
            runCatchingCancellable { service?.deleteReader(reader.id) ?: repository.
delete(reader.id) }
                .onSuccess { errors.value = null }
                .onFailure { errors.value = it.message ?: "Не удалось удалить читате
ля" }
        }
    }
    override fun close() = scope.cancel()
}

class PublicationsViewModel(
    private val repository: PublicationRepository,
    private val branchesRepository: BranchRepository,
    private val ids: IdGenerator = UuidIdGenerator(),
    private val service: LibraryService? = null,
    private val scope: CoroutineScope = CoroutineScope(SupervisorJob() + Dispatchers
.Default),
) : AutoCloseable {
    private val errors = MutableStateFlow<String?>(null)
    private val saveOperation = SaveOperation(scope) { errors.value = it }
    val saveState: StateFlow<SaveUiState> = saveOperation.state
    private val searchQuery = MutableStateFlow("")
    val query: StateFlow<String> = searchQuery.asStateFlow()
    val state: StateFlow<CollectionUiState<Publication>> = combine(repository.public
ations, searchQuery) { publications, query ->
        val normalized = query.trim()
        CollectionUiState(
            items = publications.filter {
                normalized.isBlank() || it.title.contains(normalized, ignoreCase = t
rue) || it.authors.contains(normalized, ignoreCase = true)
            },
            isLoading = false,
        )
    }
        .catch { emit(CollectionUiState(isLoading = false, error = "Не удалось загру
зить издания")) }
        .combine(errors) { value, error -> value.copy(error = error ?: value.error)
}
        .stateIn(scope, SharingStarted.Eagerly, CollectionUiState())

    val branches: StateFlow<CollectionUiState<LibraryBranch>> = branchesRepository.b
ranches
        .map { CollectionUiState(items = it, isLoading = false) }
        .catch { emit(CollectionUiState(isLoading = false, error = "Не удалось загру
зить филиалы")) }
        .stateIn(scope, SharingStarted.Eagerly, CollectionUiState())

    fun setSearchQuery(value: String) {
        searchQuery.value = value
    }

    fun addBook(title: String, authors: String, publisher: String, year: String, cop
ies: String, branchId: String, isbn: String) {
        val parsedYear = year.toIntOrNull()
        val parsedCopies = copies.toIntOrNull()
        if (title.isBlank() || authors.isBlank() || publisher.isBlank() || parsedYea
```

```
r == null || parsedCopies == null || branchId.isBlank()) {
            saveOperation.reject("Введите название, авторов, издателя, год, число эк
земпляров и филиал")
            return
        }
        saveOperation.submit {
            repository.save(Book(ids.nextId(), title, authors, publisher, parsedYear
, parsedCopies, parsedCopies, branchId, isbn.ifBlank { null }))
        }
    }

    fun addPeriodical(title: String, editors: String, publisher: String, year: Strin
g, issue: String, copies: String, branchId: String) {
        val parsedYear = year.toIntOrNull()
        val parsedIssue = issue.toIntOrNull()
        val parsedCopies = copies.toIntOrNull()
        if (title.isBlank() || editors.isBlank() || publisher.isBlank() || parsedYea
r == null || parsedIssue == null || parsedCopies == null || branchId.isBlank()) {
            saveOperation.reject("Введите название, редакторов, издателя, год, номер
выпуска, число экземпляров и филиал")
            return
        }
        saveOperation.submit {
            repository.save(Periodical(ids.nextId(), title, editors, publisher, pars
edYear, parsedCopies, parsedCopies, branchId, parsedIssue))
        }
    }

    fun delete(publication: Publication) {
        scope.launch {
            runCatchingCancellable { service?.deletePublication(publication.id) ?: r
epository.delete(publication.id) }
                .onSuccess { errors.value = null }
                .onFailure { errors.value = it.message ?: "Не удалось удалить издани
e" }
        }
    }
    override fun close() = scope.cancel()
}

data class LoanRow(val loan: Loan, val readerName: String, val publicationTitle: Str
ing)

class LoansViewModel(
    private val service: LibraryService,
    private val loansRepository: LoanRepository,
    private val readersRepository: ReaderRepository,
    private val publicationsRepository: PublicationRepository,
    private val librariansRepository: LibrarianRepository,
    private val scope: CoroutineScope = CoroutineScope(SupervisorJob() + Dispatchers
.Default),
    private val overdueRefreshIntervalMillis: Long = 60_000L,
    private val currentTimeMillis: () -> Long = ::systemTimeMillis,
) : AutoCloseable {
    private val operationErrors = MutableStateFlow<String?>(null)
    private val actionOperation = SaveOperation(scope) { operationErrors.value = it
}
```

```
    val actionState: StateFlow<SaveUiState> = actionOperation.state
    private val refreshErrors = MutableStateFlow<String?>(null)
    val state: StateFlow<CollectionUiState<LoanRow>> = combine(
        loansRepository.loans,
        readersRepository.readers,
        publicationsRepository.publications,
    ) { loans, readers, publications ->
        val readerNames = readers.associateBy(Reader::id)
        val titles = publications.associateBy(Publication::id)
        CollectionUiState(
            items = loans.map { LoanRow(it, readerNames[it.readerId]?.fullName ?: "Н
еизвестный читатель", titles[it.publicationId]?.title ?: "Неизвестное издание") },
            isLoading = false,
        )
    }.catch { emit(CollectionUiState(isLoading = false, error = "Не удалось загрузит
ь выдачи")) }
        .combine(refreshErrors) { value, error -> value.copy(error = error ?: value.
error) }
        .combine(operationErrors) { value, error -> value.copy(error = error ?: valu
e.error) }
        .stateIn(scope, SharingStarted.Eagerly, CollectionUiState())

    init {
        require(overdueRefreshIntervalMillis > 0L) { "Интервал обновления просроченн
ых выдач должен быть положительным" }
        scope.launch {
            while (isActive) {
                delay(overdueRefreshIntervalMillis)
                runCatchingCancellable { loansRepository.refreshOverdue(currentTimeM
illis()) }
                    .onSuccess { refreshErrors.value = null }
                    .onFailure { refreshErrors.value = it.message ?: "Не удалось обн
овить просроченные выдачи" }
            }
        }
    }

    fun issue(reader: Reader?, publication: Publication?) {
        if (reader == null || publication == null) {
            actionOperation.reject("Выберите читателя по имени и издание по названию
")
            return
        }
        actionOperation.submit {
            val librarian = librariansRepository.librarians.first().firstOrNull()
                ?: error("Зарегистрируйте библиотекаря перед выдачей")
            @OptIn(kotlin.time.ExperimentalTime::class)
            val now = kotlin.time.Clock.System.now().toEpochMilliseconds()
            service.issueLoan(reader.id, publication.id, librarian.id, now)
        }
    }

    fun returnLoan(loan: Loan) {
        actionOperation.submit {
            @OptIn(kotlin.time.ExperimentalTime::class)
            service.returnLoan(loan.id, kotlin.time.Clock.System.now().toEpochMillis
econds())
```

```
        }
    }
    override fun close() = scope.cancel()
}

@OptIn(kotlin.time.ExperimentalTime::class)
private fun systemTimeMillis(): Long = kotlin.time.Clock.System.now().toEpochMillise
conds()

data class AdministrationUiState(
    val isBusy: Boolean = false,
    val message: String? = null,
    val error: String? = null,
)

class AdministrationViewModel(
    private val service: LibraryService,
    private val scope: CoroutineScope = CoroutineScope(SupervisorJob() + Dispatchers
.Default),
) : AutoCloseable {
    private val _state = MutableStateFlow(AdministrationUiState())
    val state: StateFlow<AdministrationUiState> = _state.asStateFlow()

    fun seedDemo() {
        scope.launch {
            _state.value = AdministrationUiState(isBusy = true)
            try {
                runCatchingCancellable { service.seedDemo() }
                    .onSuccess { inserted ->
                        _state.value = AdministrationUiState(message = if (inserted)
"Демонстрационные данные добавлены" else "Демонстрационные данные уже существуют")
                    }
                    .onFailure { _state.value = AdministrationUiState(error = it.mes
sage ?: "Не удалось добавить демонстрационные данные") }
            } finally {
                clearBusyAfterStandaloneCancellation()
            }
        }
    }

    fun clearAll() {
        scope.launch {
            _state.value = AdministrationUiState(isBusy = true)
            try {
                runCatchingCancellable { service.clearAll() }
                    .onSuccess { result -> _state.value = AdministrationUiState(mess
age = "Удалено записей: ${result.totalDeleted}") }
                    .onFailure { _state.value = AdministrationUiState(error = it.mes
sage ?: "Не удалось очистить данные") }
            } finally {
                clearBusyAfterStandaloneCancellation()
            }
        }
    }

    private fun clearBusyAfterStandaloneCancellation() {
        if (scope.isActive && _state.value.isBusy) {
```

```
            _state.value = _state.value.copy(isBusy = false)
        }
    }

    override fun close() = scope.cancel()
}
```

app/src/commonMain/kotlin/ui/LoanDates.kt

```
package ui

internal expect fun formatLoanDate(epochMillis: Long): String
```

app/src/commonMain/kotlin/ui/PublicationCover.kt

```
package ui

import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.Icon
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.ColorFilter
import androidx.compose.ui.graphics.ColorMatrix
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.semantics.contentDescription
import androidx.compose.ui.semantics.semantics
import androidx.compose.ui.unit.dp
import domain.Book
import domain.Publication
import org.jetbrains.compose.resources.painterResource
import ui.resources.*

/** Covers retain their identity; only presentation saturation is reduced. No networ
k requests. */
@Composable
internal fun PublicationCover(publication: Publication) {
    val book = (publication as? Book)?.isbn?.let(TestBookCatalogue::find)
    val asset = when (book?.coverKey) {
        "clean_code" -> Res.drawable.cover_clean_code
        "kotlin" -> Res.drawable.cover_kotlin
        "api_patterns" -> Res.drawable.cover_api_patterns
        "rust_patterns" -> Res.drawable.cover_rust_patterns
        "dotnet_patterns" -> Res.drawable.cover_dotnet_patterns
        "headfirst_patterns" -> Res.drawable.cover_headfirst_patterns
        else -> null
    }
    val shape = RoundedCornerShape(10.dp)
    Box(Modifier.size(68.dp, 96.dp).clip(shape).background(LibraryDesign.control)
```

```
        .border(1.dp, LibraryDesign.divider, shape).padding(5.dp)
        .semantics { contentDescription = if (asset == null) "Обложка недоступна: ${
publication.title}" else "Обложка: ${publication.title}" },
        contentAlignment = Alignment.Center) {
        if (asset != null) {
            Image(painterResource(asset), contentDescription = null, modifier = Modi
fier.size(58.dp, 86.dp),
                contentScale = ContentScale.Fit,
                colorFilter = ColorFilter.colorMatrix(ColorMatrix().apply { setToSat
uration(0.28f) }))
        } else {
            Icon(LibraryIcon.MenuBook.vector, contentDescription = null, tint = Libr
aryDesign.secondaryText, modifier = Modifier.size(28.dp))
        }
    }
}
```

app/src/commonMain/kotlin/ui/SaveOperation.kt

```
package ui

import kotlinx.coroutines.CancellationException
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.isActive
import kotlinx.coroutines.launch

data class SaveUiState(
    val isSaving: Boolean = false,
    val error: String? = null,
    val revision: Int = 0,
)

/** UI operation state: keep the form open until the actual write succeeds. */
internal class SaveOperation(
    private val scope: CoroutineScope,
    private val onError: (String?) -> Unit,
) {
    private val mutable = MutableStateFlow(SaveUiState())
    val state: StateFlow<SaveUiState> = mutable.asStateFlow()

    fun reject(message: String) {
        if (mutable.value.isSaving) return
        mutable.update { it.copy(error = message) }
        onError(message)
    }

    fun submit(operation: suspend () -> Unit): Boolean {
        if (!scope.isActive) return false
        val previous = mutable.value
        if (previous.isSaving || !mutable.compareAndSet(previous, previous.copy(isSa
ving = true, error = null))) return false
        onError(null)
        scope.launch {
```

```
            try {
                operation()
                mutable.update { it.copy(error = null, revision = it.revision + 1)
}
                onError(null)
            } catch (cancelled: CancellationException) {
                throw cancelled
            } catch (failure: Throwable) {
                val message = failure.message ?: "Не удалось сохранить запись"
                mutable.update { it.copy(error = message) }
                onError(message)
            } finally {
                mutable.update { it.copy(isSaving = false) }
            }
        }
        return true
    }
}
```

app/src/commonMain/kotlin/ui/TestBookCatalogue.kt

```
package ui

internal data class TestBook(val title: String, val authors: String, val year: Int,
val isbn: String, val coverKey: String)

internal object TestBookCatalogue {
    val books = listOf(
        TestBook("Чистый код: создание, анализ и рефакторинг. Библиотека программист
а", "Мартин Р.", 2025, "978-5-4461-0960-9", "clean_code"),
        TestBook("Head First. Kotlin", "Гриффитс Д.", 2023, "978-5-4461-1335-4", "ko
tlin"),
        TestBook("Паттерны проектирования API", "Гивакс Д.", 2026, "978-5-4461-1984-
4", "api_patterns"),
        TestBook("Паттерны проектирования Rust", "Мэтьюз Б.", 2026, "978-5-4461-4389
-4", "rust_patterns"),
        TestBook("Паттерны проектирования на платформе .NET", "Тепляков С. В.", 2021
, "978-5-4461-0843-5", "dotnet_patterns"),
        TestBook("Head First. Паттерны проектирования. 2-е издание", "Фримен Э., Роб
сон Э., Сьерра К., Бейтс Б.", 2025, "978-5-4461-1819-9", "headfirst_patterns"),
    )
    fun find(isbn: String?): TestBook? = isbn?.let { value -> books.firstOrNull { it
.isbn == value || it.isbn.replace("-", "") == value } }
}
```

app/src/commonMain/kotlin/ui/TestDataSeeder.kt

```
package ui

import domain.Book
import domain.LibraryEntityFactory
import kotlinx.coroutines.flow.first
import kotlinx.coroutines.sync.Mutex
import kotlinx.coroutines.sync.withLock
import persistence.LibraryRepositoryBundle
import kotlin.random.Random
```

```
internal interface TestDataSource {
    suspend fun addBooks(): Int
    suspend fun addReaders(): Int
}

/** Explicit, additive test-data actions. Never called during application startup. *
/
internal class TestDataSeeder(
    private val repositories: LibraryRepositoryBundle,
    private val random: Random = Random.Default,
) : TestDataSource {
    private val factory = LibraryEntityFactory()
    private val mutex = Mutex()

    override suspend fun addBooks(): Int = mutex.withLock {
        val existing = repositories.publications.publicationsOnce().filterIsInstance
<Book>()
            .mapNotNull { TestBookCatalogue.find(it.isbn)?.isbn }.toSet()
        val samples = TestBookCatalogue.books.filterNot { it.isbn in existing }.shuf
fled(random).take(3)
        if (samples.isEmpty()) return@withLock 0
        val branch = repositories.branches.branches.first().firstOrNull()
            ?: factory.branch("Тестовый филиал", "Учебный адрес").also { repositorie
s.branches.save(it) }
        if (repositories.librarians.librarians.first().isEmpty()) {
            repositories.librarians.save(factory.librarian("Тестовый библиотекарь",
"Учебная демонстрация", branch.id))
        }
        for (sample in samples) {
            repositories.publications.save(factory.book(sample.title, sample.authors
, "Питер", sample.year,
                random.nextInt(2, 6), branch.id, sample.isbn))
        }
        samples.size
    }

    override suspend fun addReaders(): Int = mutex.withLock {
        val existing = repositories.readers.readersOnce().map { it.fullName }.toSet(
)
        val names = readerNames.filterNot { it in existing }.shuffled(random).take(5
)
        for (name in names) {
            repositories.readers.save(factory.reader(name, random.nextInt(1980, 2008
),
                "Учебная улица, дом ${random.nextInt(1, 90)}", "Тестовые данные"))
        }
        names.size
    }

    private val readerNames = listOf(
        "Анна Соколова", "Илья Петров", "Мария Волкова", "Дмитрий Кузнецов", "Екатер
ина Орлова",
        "Алексей Морозов", "Ольга Смирнова", "Павел Крылов", "Дарья Новикова", "Ники
та Белов",
        "София Лебедева", "Артём Васильев", "Полина Романова", "Михаил Попов", "Елен
а Зайцева",
```

```
        "Роман Громов", "Алиса Фёдорова", "Максим Сидоров", "Вера Макарова", "Сергей
Егоров",
    )
}
```

app/src/commonMain/kotlin/ui/TestDataViewModel.kt

```
package ui

import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.SupervisorJob
import kotlinx.coroutines.cancel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.asStateFlow

internal data class TestDataMessage(val kind: String, val text: String)

internal class TestDataViewModel(
    private val source: TestDataSource,
    private val scope: CoroutineScope = CoroutineScope(SupervisorJob() + Dispatchers
.Default),
) : AutoCloseable {
    private val operation = SaveOperation(scope) {}
    val state = operation.state
    private val mutableMessage = MutableStateFlow<TestDataMessage?>(null)
    val message = mutableMessage.asStateFlow()

    fun addBooks() { operation.submit {
        mutableMessage.value = null
        val count = source.addBooks()
        mutableMessage.value = TestDataMessage("books", if (count == 0) "Все тестовы
е книги уже добавлены" else "Добавлено тестовых книг: $count")
    } }
    fun addReaders() { operation.submit {
        mutableMessage.value = null
        val count = source.addReaders()
        mutableMessage.value = TestDataMessage("readers", if (count == 0) "Все тесто
вые читатели уже добавлены" else "Добавлено тестовых читателей: $count")
    } }
    override fun close() = scope.cancel()
}
```

app/src/jvmMain/kotlin/main.kt

```
import androidx.compose.ui.unit.dp
import androidx.compose.ui.window.Window
import androidx.compose.ui.window.application
import androidx.compose.ui.window.rememberWindowState
import di.ApplicationContainer
import ui.App
import ui.LibraryDesign

fun main() {
    val container = ApplicationContainer()
    application {
```

```
        Window(
            state = rememberWindowState(
                width = LibraryDesign.windowWidthDp.dp,
                height = LibraryDesign.windowHeightDp.dp,
            ),
            onCloseRequest = {
                container.close()
                exitApplication()
            },
            title = "Библиотека курса ООП",
        ) {
            App(container)
        }
    }
}
```

app/src/jvmMain/kotlin/ui/LoanDatesJvm.kt

```
package ui

import java.time.Instant
import java.time.ZoneId
import java.time.format.DateTimeFormatter

internal actual fun formatLoanDate(epochMillis: Long): String =
    Instant.ofEpochMilli(epochMillis).atZone(ZoneId.systemDefault()).format(DateTime
Formatter.ofPattern("dd.MM.yyyy"))
```

core/src/commonMain/kotlin/application/RoomLibraryService.kt

```
package application

import domain.DefaultLoanPolicy
import domain.DemoData
import domain.IdGenerator
import domain.LibraryEntityFactory
import domain.LibraryService
import domain.Loan
import domain.LoanPolicy
import domain.LoanState
import domain.UuidIdGenerator
import persistence.LibraryRepositoryBundle

class RoomLibraryService(
    private val repositories: LibraryRepositoryBundle,
    private val policy: LoanPolicy = DefaultLoanPolicy(),
    private val ids: IdGenerator = UuidIdGenerator(),
) : LibraryService {
    override suspend fun issueLoan(readerId: String, publicationId: String, libraria
nId: String, now: Long): Loan {
        val reader = repositories.readers.findById(readerId) ?: error("Читатель не н
айден")
        val publication = repositories.publications.findById(publicationId) ?: error
("Издание не найдено")
        repositories.librarians.findById(librarianId) ?: error("Библиотекарь не найд
ен")
```

```
        val decision = policy.canIssue(reader, publication, repositories.loans.activ
eCountForReader(readerId))
        if (!decision.isAllowed) error(decision.reason ?: "Выдача запрещена")
        val loan = Loan(
            id = ids.nextId(),
            readerId = readerId,
            publicationId = publicationId,
            librarianId = librarianId,
            issuedAt = now,
            dueAt = policy.dueAt(now, publication),
            state = LoanState.Active,
        )
        if (!repositories.loans.issue(loan, policy.maxActiveLoansPerReader)) {
            error("Выдача запрещена или у издания нет доступных экземпляров")
        }
        return loan
    }

    override suspend fun returnLoan(loanId: String, now: Long): Loan =
        repositories.loans.returnLoan(loanId, now) ?: error("Выдача не найдена")

    override suspend fun deleteReader(readerId: String) {
        repositories.readers.delete(readerId)
    }

    override suspend fun deletePublication(publicationId: String) {
        repositories.publications.delete(publicationId)
    }

    override suspend fun seedDemo(): Boolean {
        val factory = LibraryEntityFactory(ids)
        val branch = factory.branch("Центральная библиотека", "Невский проспект, 28"
)
        val librarian = factory.librarian("Елена Волкова", "Старший библиотекарь", b
ranch.id)
        val reader = factory.reader("Анна Смирнова", 1995, "Санкт-Петербург", "ГУАП"
)
        val secondReader = factory.reader("Алексей Петров", 1994, "Санкт-Петербург",
"ГУАП")
        val book = factory.book("Объектно-ориентированное программирование", "Е. О.
Шумова", "ГУАП", 2026, 2, branch.id, "978-5-00000-001-0")
        val periodical = factory.periodical("Библиотечное дело", "Редакционная колле
гия", "Профессия", 2025, 1, 2, branch.id)
        val loan = factory.loan(reader.id, book.id, librarian.id, 1_700_000_000_000L
, 1_701_814_400_000L)
        return repositories.insertDemo(
            DemoData(
                branch = branch,
                librarian = librarian,
                readers = listOf(reader, secondReader),
                publications = listOf(book, periodical),
                loan = loan,
            ),
        )
    }
```

```
    override suspend fun clearAll() = repositories.clearAll()
}
```

core/src/commonMain/kotlin/domain/DomainModels.kt

```
package domain

/** A person registered as a library reader. */
data class Reader(
    override val id: String,
    override val fullName: String,
    val birthYear: Int,
    val address: String,
    val organization: String,
) : Person {
    init {
        validateText(fullName, "fullName")
        require(birthYear in 1800..2100) { "Год рождения вне поддерживаемого диапазо
на" }
        validateText(address, "address")
        validateText(organization, "organization")
    }
}

/** A library employee who can issue publications. */
data class Librarian(
    override val id: String,
    override val fullName: String,
    val position: String,
    val branchId: String,
) : Person {
    init {
        validateText(fullName, "fullName")
        validateText(position, "position")
        validateText(branchId, "branchId")
    }
}

/** A physical or logical library location. */
data class LibraryBranch(
    override val id: String,
    val name: String,
    val address: String,
) : LibraryEntity {
    init {
        validateText(name, "name")
        validateText(address, "address")
    }
}

/** A book in the collection. */
data class Book(
    override val id: String,
    override val title: String,
    override val authors: String,
    override val publisher: String,
    override val publicationYear: Int,
```

```
    override val totalCopies: Int,
    override val availableCopies: Int,
    override val branchId: String,
 val isbn: String?,
 ) : Publication {
    init {
        validateText(title, "title")
        validateText(authors, "authors")
        validateText(publisher, "publisher")
        validateText(branchId, "branchId")
        require(publicationYear in 1450..2100) { "Год издания вне поддерживаемого ди
апазона" }
        validatePublicationCopies(totalCopies, availableCopies)
    }
}

/** A periodical issue in the collection. */
data class Periodical(
    override val id: String,
    override val title: String,
    override val authors: String,
    override val publisher: String,
    override val publicationYear: Int,
    override val totalCopies: Int,
    override val availableCopies: Int,
    override val branchId: String,
    val issueNumber: Int,
) : Publication {
    init {
        validateText(title, "title")
        validateText(authors, "authors")
        validateText(publisher, "publisher")
        validateText(branchId, "branchId")
        require(publicationYear in 1450..2100) { "Год издания вне поддерживаемого ди
апазона" }
        require(issueNumber > 0) { "Номер выпуска должен быть положительным" }
        validatePublicationCopies(totalCopies, availableCopies)
    }
}

/** State pattern for a loan lifecycle. */
sealed interface LoanState {
    val status: String

    data object Active : LoanState {
        override val status: String = "ACTIVE"
        override fun toString(): String = status
    }

    data object Overdue : LoanState {
        override val status: String = "OVERDUE"
        override fun toString(): String = status
    }

    data class Returned(val returnedAt: Long) : LoanState {
        override val status: String = "RETURNED"
    }
```

```
}

fun LoanState.returnAt(timestamp: Long): LoanState = when (this) {
    LoanState.Active, LoanState.Overdue -> LoanState.Returned(timestamp)
    is LoanState.Returned -> error("Выдача уже возвращена")
}

data class Loan(
    override val id: String,
    val readerId: String,
    val publicationId: String,
    val librarianId: String,
    val issuedAt: Long,
    val dueAt: Long,
    val state: LoanState = LoanState.Active,
) : LibraryEntity {
    init {
        validateText(readerId, "readerId")
        validateText(publicationId, "publicationId")
        validateText(librarianId, "librarianId")
        require(dueAt >= issuedAt) { "Дата возврата не может быть раньше даты выдачи
" }
    }
}
```

core/src/commonMain/kotlin/domain/LibraryEntity.kt

```
@file:OptIn(kotlin.uuid.ExperimentalUuidApi::class)

package domain

import kotlin.uuid.Uuid

/** Root contract for all domain entities. IDs are generated internally and opaque t
o users. */
interface LibraryEntity {
    val id: String
}

interface IdGenerator {
    fun nextId(): String
}

class UuidIdGenerator : IdGenerator {
    override fun nextId(): String = Uuid.random().toString()
}

interface Person : LibraryEntity {
    val fullName: String
}

interface Publication : LibraryEntity {
    val title: String
    val authors: String
    val publisher: String
    val publicationYear: Int
    val totalCopies: Int
```

```
    val availableCopies: Int
    val branchId: String
}

fun validateText(value: String, field: String) {
    val label = mapOf(
        "fullName" to "Имя",
        "address" to "Адрес",
        "organization" to "Организация",
        "position" to "Должность",
        "branchId" to "Филиал",
        "name" to "Название",
        "title" to "Название",
        "authors" to "Авторы",
        "publisher" to "Издатель",
        "readerId" to "Читатель",
        "publicationId" to "Издание",
        "librarianId" to "Библиотекарь",
    )[field] ?: field
    require(value.isNotBlank()) { "$label не должно быть пустым" }
}

fun validatePublicationCopies(totalCopies: Int, availableCopies: Int) {
    require(totalCopies > 0) { "Общее число экземпляров должно быть положительным"
}
    require(availableCopies in 0..totalCopies) { "Доступное число экземпляров должно
быть от 0 до общего числа" }
}
```

core/src/commonMain/kotlin/domain/LibraryEntityFactory.kt

```
package domain

class LibraryEntityFactory(private val ids: IdGenerator = UuidIdGenerator()) {
    fun reader(fullName: String, birthYear: Int, address: String, organization: Stri
ng): Reader =
        Reader(ids.nextId(), fullName, birthYear, address, organization)

    fun librarian(fullName: String, position: String, branchId: String): Librarian
=
        Librarian(ids.nextId(), fullName, position, branchId)

    fun branch(name: String, address: String): LibraryBranch =
        LibraryBranch(ids.nextId(), name, address)

    fun book(
        title: String,
        authors: String,
        publisher: String,
        publicationYear: Int,
        totalCopies: Int,
        branchId: String,
        isbn: String? = null,
    ): Book = Book(
        ids.nextId(), title, authors, publisher, publicationYear,
        totalCopies, totalCopies, branchId, isbn,
    )
```

```
    fun periodical(
        title: String,
        authors: String,
        publisher: String,
        publicationYear: Int,
        issueNumber: Int,
        totalCopies: Int,
        branchId: String,
    ): Periodical = Periodical(
        ids.nextId(), title, authors, publisher, publicationYear,
        totalCopies, totalCopies, branchId, issueNumber,
    )

    fun loan(
        readerId: String,
        publicationId: String,
        librarianId: String,
        issuedAt: Long,
        dueAt: Long,
    ): Loan = Loan(ids.nextId(), readerId, publicationId, librarianId, issuedAt, due
At)
}
```

core/src/commonMain/kotlin/domain/LibraryService.kt

```
package domain

/** Domain-facing facade; the persistence-backed implementation lives in the applica
tion layer. */
interface LibraryService {
    suspend fun issueLoan(readerId: String, publicationId: String, librarianId: Stri
ng, now: Long): Loan
    suspend fun returnLoan(loanId: String, now: Long): Loan
    suspend fun deleteReader(readerId: String)
    suspend fun deletePublication(publicationId: String)
    suspend fun seedDemo(): Boolean
    suspend fun clearAll(): ClearAllResult
}

data class DemoData(
    val branch: LibraryBranch,
    val librarian: Librarian,
    val readers: List<Reader>,
    val publications: List<Publication>,
    val loan: Loan,
)

data class ClearAllResult(
    val loans: Int,
    val publications: Int,
    val readers: Int,
    val librarians: Int,
    val branches: Int,
) {
    val totalDeleted: Int
```

```
        get() = loans + publications + readers + librarians + branches
}
```

core/src/commonMain/kotlin/domain/LoanPolicy.kt

```
package domain

interface LoanPolicy {
    val maxActiveLoansPerReader: Int
        get() = Int.MAX_VALUE

    fun canIssue(reader: Reader, publication: Publication, activeLoansForReader: Int
): PolicyDecision
    fun dueAt(issuedAt: Long, publication: Publication): Long
}

data class PolicyDecision(val isAllowed: Boolean, val reason: String? = null)

class DefaultLoanPolicy(
    override val maxActiveLoansPerReader: Int = 5,
    private val bookLoanDays: Long = 21,
    private val periodicalLoanDays: Long = 7,
) : LoanPolicy {
    init {
        require(maxActiveLoansPerReader > 0)
        require(bookLoanDays > 0)
        require(periodicalLoanDays > 0)
    }

    override fun canIssue(
        reader: Reader,
        publication: Publication,
        activeLoansForReader: Int,
    ): PolicyDecision = when {
        activeLoansForReader >= maxActiveLoansPerReader ->
            PolicyDecision(false, "Читатель достиг лимита активных выдач")
        publication.availableCopies <= 0 ->
            PolicyDecision(false, "У издания нет доступных экземпляров")
        else -> PolicyDecision(true)
    }

    override fun dueAt(issuedAt: Long, publication: Publication): Long {
        val days = if (publication is Periodical) periodicalLoanDays else bookLoanDa
ys
        return issuedAt + days * MILLIS_PER_DAY
    }

    private companion object {
        const val MILLIS_PER_DAY = 86_400_000L
    }
}
```

core/src/commonMain/kotlin/persistence/Database.kt

```
package persistence
```

```
import androidx.room3.RoomDatabase
import androidx.sqlite.driver.bundled.BundledSQLiteDriver
import kotlinx.coroutines.Dispatchers

fun buildDatabase(builder: RoomDatabase.Builder<LibraryDatabase>): LibraryDatabase
=
    builder
        .setDriver(BundledSQLiteDriver())
        .setQueryCoroutineContext(Dispatchers.IO)
        .build()
```

core/src/commonMain/kotlin/persistence/LibraryDao.kt

```
package persistence

import androidx.room3.Dao
import androidx.room3.Insert
import androidx.room3.OnConflictStrategy
import androidx.room3.Query
import androidx.room3.Transaction
import androidx.room3.Upsert
import kotlinx.coroutines.flow.Flow

@Dao
interface LibraryDao {
    @Query("SELECT * FROM readers ORDER BY fullName")
    fun observeReaders(): Flow<List<ReaderRecord>>

    @Query("SELECT * FROM readers ORDER BY fullName")
    suspend fun getReaders(): List<ReaderRecord>

    @Query("SELECT * FROM readers WHERE id = :id")
    suspend fun getReader(id: String): ReaderRecord?

    @Upsert
    suspend fun saveReader(record: ReaderRecord)

    @Query("DELETE FROM readers WHERE id = :id")
    suspend fun deleteReader(id: String): Int

    @Query("SELECT COUNT(*) FROM loans WHERE readerId = :id AND returnedAt IS NULL"
)
    suspend fun countActiveLoansForReader(id: String): Int

    @Query("SELECT COUNT(*) FROM loans WHERE readerId = :id")
    suspend fun countLoansForReader(id: String): Int

    @Transaction
    suspend fun deleteReaderIfUnused(id: String): Int {
        if (getReader(id) == null) return -1
        if (countLoansForReader(id) > 0) return 0
        return deleteReader(id)
    }

    @Query("SELECT * FROM librarians ORDER BY fullName")
    fun observeLibrarians(): Flow<List<LibrarianRecord>>
```

```
@Query("SELECT * FROM librarians ORDER BY fullName")
suspend fun getLibrarians(): List<LibrarianRecord>

@Query("SELECT * FROM librarians WHERE id = :id")
suspend fun getLibrarian(id: String): LibrarianRecord?

@Upsert
suspend fun saveLibrarian(record: LibrarianRecord)

@Query("SELECT * FROM branches ORDER BY name")
fun observeBranches(): Flow<List<BranchRecord>>

@Query("SELECT * FROM branches ORDER BY name")
suspend fun getBranches(): List<BranchRecord>

@Query("SELECT COUNT(*) FROM branches")
suspend fun countBranches(): Int

@Upsert
suspend fun saveBranch(record: BranchRecord)

@Query("SELECT * FROM publications ORDER BY title")
fun observePublications(): Flow<List<PublicationRecord>>

@Query("SELECT * FROM publications ORDER BY title")
suspend fun getPublications(): List<PublicationRecord>

@Query("SELECT * FROM publications WHERE id = :id")
suspend fun getPublication(id: String): PublicationRecord?

@Insert(onConflict = OnConflictStrategy.IGNORE)
suspend fun insertPublicationIfAbsent(record: PublicationRecord): Long

@Query("""
    UPDATE publications SET
        kind = :kind,
        title = :title,
        authors = :authors,
        publisher = :publisher,
        publicationYear = :publicationYear,
        isbn = :isbn,
        issueNumber = :issueNumber,
        branchId = :branchId
    WHERE id = :id
""")
suspend fun updatePublicationMetadata(
    id: String,
    kind: String,
    title: String,
    authors: String,
    publisher: String,
    publicationYear: Int,
    isbn: String?,
    issueNumber: Int?,
    branchId: String,
): Int
```

```
    @Transaction
    suspend fun savePublication(record: PublicationRecord) {
        if (insertPublicationIfAbsent(record) == -1L) {
            updatePublicationMetadata(
                record.id,
                record.kind,
                record.title,
                record.authors,
                record.publisher,
                record.publicationYear,
                record.isbn,
                record.issueNumber,
                record.branchId,
            )
        }
    }

    @Query("UPDATE publications SET availableCopies = availableCopies - 1 WHERE id =
:publicationId AND availableCopies > 0")
    suspend fun decrementAvailable(publicationId: String): Int

    @Query("UPDATE publications SET availableCopies = availableCopies + 1 WHERE id =
:publicationId AND availableCopies < totalCopies")
    suspend fun incrementAvailable(publicationId: String): Int

    @Query("DELETE FROM publications WHERE id = :id")
    suspend fun deletePublication(id: String): Int

    @Query("SELECT COUNT(*) FROM loans WHERE publicationId = :id AND returnedAt IS N
ULL")
    suspend fun countActiveLoansForPublication(id: String): Int

    @Query("SELECT COUNT(*) FROM loans WHERE publicationId = :id")
    suspend fun countLoansForPublication(id: String): Int

    @Transaction
    suspend fun deletePublicationIfUnused(id: String): Int {
        if (getPublication(id) == null) return -1
        if (countLoansForPublication(id) > 0) return 0
        return deletePublication(id)
    }

    @Query("SELECT * FROM loans ORDER BY issuedAt DESC")
    fun observeLoans(): Flow<List<LoanRecord>>

    @Query("SELECT * FROM loans ORDER BY issuedAt DESC")
    suspend fun getLoans(): List<LoanRecord>

    @Query("SELECT * FROM loans WHERE id = :id")
    suspend fun getLoan(id: String): LoanRecord?

    @Query("SELECT COUNT(*) FROM loans WHERE readerId = :readerId AND returnedAt IS
NULL")
    suspend fun countActiveLoans(readerId: String): Int

    @Query("UPDATE loans SET status = 'OVERDUE' WHERE status = 'ACTIVE' AND returned
At IS NULL AND dueAt < :now")
```

```
    suspend fun markOverdueLoans(now: Long): Int

    @Insert
    suspend fun insertLoan(record: LoanRecord)

    @Query("UPDATE loans SET returnedAt = :returnedAt, status = 'RETURNED' WHERE id
= :loanId AND returnedAt IS NULL")
    suspend fun markReturned(loanId: String, returnedAt: Long): Int

    @Transaction
    suspend fun issueLoanAtomically(record: LoanRecord, maxActiveLoansPerReader: Int
): Boolean {
        if (record.status != "ACTIVE" || record.returnedAt != null) return false
        if (countActiveLoans(record.readerId) >= maxActiveLoansPerReader) return fal
se
        if (decrementAvailable(record.publicationId) != 1) return false
        insertLoan(record)
        return true
    }

    @Transaction
    suspend fun issueLoanAtomically(record: LoanRecord): Boolean =
        issueLoanAtomically(record, Int.MAX_VALUE)

    @Transaction
    suspend fun returnLoanAtomically(loanId: String, returnedAt: Long): LoanRecord?
{
        val current = getLoan(loanId) ?: return null
        require(returnedAt >= current.issuedAt) { "Дата возврата не может быть раньш
е даты выдачи" }
        if (current.returnedAt != null) return current
        if (markReturned(loanId, returnedAt) != 1) return getLoan(loanId)
        if (incrementAvailable(current.publicationId) != 1) {
            error("Нарушен инвариант складского остатка издания")
        }
        return getLoan(loanId)
    }

    @Query("SELECT COUNT(*) FROM loans")
    suspend fun countLoans(): Int

    @Query("SELECT COUNT(*) FROM publications")
    suspend fun countPublications(): Int

    @Query("SELECT COUNT(*) FROM readers")
    suspend fun countReaders(): Int

    @Query("SELECT COUNT(*) FROM librarians")
    suspend fun countLibrarians(): Int

    @Query("DELETE FROM loans")
    suspend fun deleteAllLoans(): Int

    @Query("DELETE FROM publications")
    suspend fun deleteAllPublications(): Int

    @Query("DELETE FROM readers")
```

```
    suspend fun deleteAllReaders(): Int

    @Query("DELETE FROM librarians")
    suspend fun deleteAllLibrarians(): Int

    @Query("DELETE FROM branches")
    suspend fun deleteAllBranches(): Int

    @Transaction
    suspend fun clearAllData(): ClearAllCounts {
        val counts = ClearAllCounts(countLoans(), countPublications(), countReaders(
), countLibrarians(), countBranches())
        // Loans must be removed first because all three entity relationships are RE
STRICT.
        deleteAllLoans()
        deleteAllPublications()
        deleteAllReaders()
        deleteAllLibrarians()
        deleteAllBranches()
        return counts
    }

    @Transaction
    suspend fun insertDemoData(
        branch: BranchRecord,
        librarian: LibrarianRecord,
        readers: List<ReaderRecord>,
        publications: List<PublicationRecord>,
        loan: LoanRecord,
    ): Boolean {
        if (countBranches() > 0) return false
        saveBranch(branch)
        saveLibrarian(librarian)
        readers.forEach { saveReader(it) }
        publications.forEach { savePublication(it) }
        if (!issueLoanAtomically(loan)) error("Не удалось оформить демонстрационную
выдачу; транзакция отменена")
        return true
    }
}
```

core/src/commonMain/kotlin/persistence/LibraryDatabase.kt

```
package persistence

import androidx.room3.ConstructedBy
import androidx.room3.Database
import androidx.room3.RoomDatabase
import androidx.room3.RoomDatabaseConstructor

@Database(
    entities = [ReaderRecord::class, LibrarianRecord::class, BranchRecord::class, Pu
blicationRecord::class, LoanRecord::class],
    version = 1,
    exportSchema = true,
)
@ConstructedBy(LibraryDatabaseConstructor::class)
```

```
abstract class LibraryDatabase : RoomDatabase() {
    abstract fun libraryDao(): LibraryDao
}

@Suppress("KotlinNoActualForExpect")
expect object LibraryDatabaseConstructor : RoomDatabaseConstructor<LibraryDatabase>
{
    override fun initialize(): LibraryDatabase
}
```

core/src/commonMain/kotlin/persistence/Records.kt

```
package persistence

import androidx.room3.Entity
import androidx.room3.ForeignKey
import androidx.room3.Index
import androidx.room3.PrimaryKey

@Entity(tableName = "readers")
data class ReaderRecord(
    @PrimaryKey val id: String,
    val fullName: String,
    val birthYear: Int,
    val address: String,
    val organization: String,
)

@Entity(
    tableName = "librarians",
    foreignKeys = [ForeignKey(
        entity = BranchRecord::class,
        parentColumns = ["id"],
        childColumns = ["branchId"],
        onDelete = ForeignKey.RESTRICT,
    )],
    indices = [Index("branchId")],
)
data class LibrarianRecord(
    @PrimaryKey val id: String,
    val fullName: String,
    val position: String,
    val branchId: String,
)

@Entity(tableName = "branches")
data class BranchRecord(
    @PrimaryKey val id: String,
    val name: String,
    val address: String,
)

@Entity(
    tableName = "publications",
    foreignKeys = [ForeignKey(
        entity = BranchRecord::class,
        parentColumns = ["id"],
```

```
            childColumns = ["branchId"],
            onDelete = ForeignKey.RESTRICT,
        )],
        indices = [Index("branchId")],
    )
    data class PublicationRecord(
        @PrimaryKey val id: String,
        val kind: String,
        val title: String,
        val authors: String,
        val publisher: String,
        val publicationYear: Int,
        val isbn: String?,
        val issueNumber: Int?,
        val totalCopies: Int,
        val availableCopies: Int,
        val branchId: String,
    )

    @Entity(
        tableName = "loans",
        foreignKeys = [
            ForeignKey(
                entity = ReaderRecord::class,
                parentColumns = ["id"],
                childColumns = ["readerId"],
                onDelete = ForeignKey.RESTRICT,
            ),
            ForeignKey(
                entity = PublicationRecord::class,
                parentColumns = ["id"],
                childColumns = ["publicationId"],
                onDelete = ForeignKey.RESTRICT,
            ),
            ForeignKey(
                entity = LibrarianRecord::class,
                parentColumns = ["id"],
                childColumns = ["librarianId"],
                onDelete = ForeignKey.RESTRICT,
            ),
        ],
        indices = [Index("readerId"), Index("publicationId"), Index("librarianId")],
    )
    data class LoanRecord(
        @PrimaryKey val id: String,
        val readerId: String,
        val publicationId: String,
        val librarianId: String,
        val issuedAt: Long,
        val dueAt: Long,
        val returnedAt: Long?,
        val status: String,
    )

    data class ClearAllCounts(
        val loans: Int,
        val publications: Int,
```

```
    val readers: Int,
    val librarians: Int,
    val branches: Int,
)
```

core/src/commonMain/kotlin/persistence/Repositories.kt

```
package persistence

import domain.Book
import domain.ClearAllResult
import domain.DemoData
import domain.LibraryBranch
import domain.Librarian
import domain.Loan
import domain.LoanState
import domain.Periodical
import domain.Publication
import domain.Reader
import androidx.sqlite.SQLiteException
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.emitAll
import kotlinx.coroutines.flow.map

interface ReaderRepository {
    val readers: Flow<List<Reader>>
    suspend fun readersOnce(): List<Reader>
    suspend fun findById(id: String): Reader?
    suspend fun save(reader: Reader)
    suspend fun delete(id: String)
}

interface LibrarianRepository {
    val librarians: Flow<List<Librarian>>
    suspend fun findById(id: String): Librarian?
    suspend fun save(librarian: Librarian)
}

interface BranchRepository {
    val branches: Flow<List<LibraryBranch>>
    suspend fun findById(id: String): LibraryBranch?
    suspend fun save(branch: LibraryBranch)
}

interface PublicationRepository {
    val publications: Flow<List<Publication>>
    suspend fun publicationsOnce(): List<Publication>
    suspend fun findById(id: String): Publication?
    suspend fun save(publication: Publication)
    suspend fun delete(id: String)
}

interface LoanRepository {
    val loans: Flow<List<Loan>>
    suspend fun findById(id: String): Loan?
    suspend fun issue(loan: Loan): Boolean
    suspend fun issue(loan: Loan, maxActiveLoansPerReader: Int): Boolean = issue(loa
```

```
n)
    suspend fun returnLoan(id: String, returnedAt: Long): Loan?
    suspend fun activeCountForReader(readerId: String): Int
    suspend fun refreshOverdue(now: Long): Int = 0
}

interface LibraryAdministrationRepository {
    suspend fun insertDemo(data: DemoData): Boolean
    suspend fun clearAll(): ClearAllResult
}

interface LibraryRepositoryBundle : LibraryAdministrationRepository {
    val readers: ReaderRepository
    val librarians: LibrarianRepository
    val branches: BranchRepository
    val publications: PublicationRepository
    val loans: LoanRepository
}

class ActiveLoanDeletionException(message: String, cause: Throwable? = null) : Illeg
alStateException(message, cause)
class InventoryException(message: String) : IllegalStateException(message)

class RoomReaderRepository(private val dao: LibraryDao) : ReaderRepository {
    override val readers: Flow<List<Reader>> = dao.observeReaders().map { records ->
records.map { readerRecordToDomain(it) } }
    override suspend fun readersOnce(): List<Reader> = dao.getReaders().map { reader
RecordToDomain(it) }
    override suspend fun findById(id: String): Reader? = dao.getReader(id)?.let { re
aderRecordToDomain(it) }
    override suspend fun save(reader: Reader) = dao.saveReader(reader.toRecord())
    override suspend fun delete(id: String) {
        try {
            when (dao.deleteReaderIfUnused(id)) {
                -1 -> Unit
                0 -> throw ActiveLoanDeletionException("Нельзя удалить читателя: ест
ь история или активные выдачи")
            }
        } catch (error: ActiveLoanDeletionException) {
            throw error
        } catch (error: SQLiteException) {
            throw ActiveLoanDeletionException("Нельзя удалить читателя: связи с выда
чами изменились", error)
        }
    }
}

class RoomLibrarianRepository(private val dao: LibraryDao) : LibrarianRepository {
    override val librarians: Flow<List<Librarian>> = dao.observeLibrarians().map { r
ecords -> records.map { librarianRecordToDomain(it) } }
    override suspend fun findById(id: String): Librarian? = dao.getLibrarian(id)?.le
t { librarianRecordToDomain(it) }
    override suspend fun save(librarian: Librarian) = dao.saveLibrarian(librarian.to
Record())
}

class RoomBranchRepository(private val dao: LibraryDao) : BranchRepository {
```

```
    override val branches: Flow<List<LibraryBranch>> = dao.observeBranches().map { r
ecords -> records.map { branchRecordToDomain(it) } }
    override suspend fun findById(id: String): LibraryBranch? = dao.getBranches().fi
rstOrNull { it.id == id }?.let { branchRecordToDomain(it) }
    override suspend fun save(branch: LibraryBranch) = dao.saveBranch(branch.toRecor
d())
}

class RoomPublicationRepository(private val dao: LibraryDao) : PublicationRepository
{
    override val publications: Flow<List<Publication>> = dao.observePublications().m
ap { records -> records.map { publicationRecordToDomain(it) } }
    override suspend fun publicationsOnce(): List<Publication> = dao.getPublications
().map { publicationRecordToDomain(it) }
    override suspend fun findById(id: String): Publication? = dao.getPublication(id)
?.let { publicationRecordToDomain(it) }
    override suspend fun save(publication: Publication) = dao.savePublication(public
ation.toRecord())
    override suspend fun delete(id: String) {
        try {
            when (dao.deletePublicationIfUnused(id)) {
                -1 -> Unit
                0 -> throw ActiveLoanDeletionException("Нельзя удалить издание: есть
история или активные выдачи")
            }
        } catch (error: ActiveLoanDeletionException) {
            throw error
        } catch (error: SQLiteException) {
            throw ActiveLoanDeletionException("Нельзя удалить издание: связи с выдач
ами изменились", error)
        }
    }
}

class RoomLoanRepository(private val dao: LibraryDao) : LoanRepository {
    @OptIn(kotlin.time.ExperimentalTime::class)
    override val loans: Flow<List<Loan>> = kotlinx.coroutines.flow.flow {
        dao.markOverdueLoans(kotlin.time.Clock.System.now().toEpochMilliseconds())
        emitAll(dao.observeLoans())
    }.map { records -> records.map { loanRecordToDomain(it) } }

    @OptIn(kotlin.time.ExperimentalTime::class)
    override suspend fun findById(id: String): Loan? {
        dao.markOverdueLoans(kotlin.time.Clock.System.now().toEpochMilliseconds())
        return dao.getLoan(id)?.let { loanRecordToDomain(it) }
    }

    override suspend fun issue(loan: Loan): Boolean = issue(loan, Int.MAX_VALUE)
    override suspend fun issue(loan: Loan, maxActiveLoansPerReader: Int): Boolean {
        if (loan.state != LoanState.Active) return false
        require(maxActiveLoansPerReader > 0)
        return dao.issueLoanAtomically(loan.toRecord(), maxActiveLoansPerReader)
    }

    override suspend fun returnLoan(id: String, returnedAt: Long): Loan? =
        dao.returnLoanAtomically(id, returnedAt)?.let { loanRecordToDomain(it) }
```

```
    override suspend fun activeCountForReader(readerId: String): Int = dao.countActi
veLoans(readerId)
    override suspend fun refreshOverdue(now: Long): Int = dao.markOverdueLoans(now)
}

class RoomRepositoryBundle(val database: LibraryDatabase) : LibraryRepositoryBundle
{
    private val dao = database.libraryDao()
    override val readers = RoomReaderRepository(dao)
    override val librarians = RoomLibrarianRepository(dao)
    override val branches = RoomBranchRepository(dao)
    override val publications = RoomPublicationRepository(dao)
    override val loans = RoomLoanRepository(dao)

    override suspend fun insertDemo(data: DemoData): Boolean = dao.insertDemoData(
        data.branch.toRecord(),
        data.librarian.toRecord(),
        data.readers.map { it.toRecord() },
        data.publications.map { it.toRecord() },
        data.loan.toRecord(),
    )

    override suspend fun clearAll(): ClearAllResult = dao.clearAllData().let {
        ClearAllResult(it.loans, it.publications, it.readers, it.librarians, it.bran
ches)
    }

    /** Compatibility entry point for persistence-focused rollback tests. */
    suspend fun insertDemoData(
        branch: BranchRecord,
        librarian: LibrarianRecord,
        readers: List<ReaderRecord>,
        publications: List<PublicationRecord>,
        loan: LoanRecord,
    ): Boolean = dao.insertDemoData(branch, librarian, readers, publications, loan)
}

private fun Reader.toRecord() = ReaderRecord(id, fullName, birthYear, address, organ
ization)
private fun readerRecordToDomain(record: Reader) = record
private fun readerRecordToDomain(record: ReaderRecord) = Reader(record.id, record.fu
llName, record.birthYear, record.address, record.organization)
private fun Librarian.toRecord() = LibrarianRecord(id, fullName, position, branchId
)
private fun librarianRecordToDomain(record: LibrarianRecord) = Librarian(record.id,
record.fullName, record.position, record.branchId)
private fun LibraryBranch.toRecord() = BranchRecord(id, name, address)
private fun branchRecordToDomain(record: BranchRecord) = LibraryBranch(record.id, re
cord.name, record.address)
private fun Publication.toRecord() = when (this) {
    is Book -> PublicationRecord(id, "BOOK", title, authors, publisher, publicationY
ear, isbn, null, totalCopies, availableCopies, branchId)
    is Periodical -> PublicationRecord(id, "PERIODICAL", title, authors, publisher,
publicationYear, null, issueNumber, totalCopies, availableCopies, branchId)
    else -> error("Неподдерживаемый тип издания")
}
private fun publicationRecordToDomain(record: PublicationRecord): Publication = when
```

```
(record.kind) {
    "BOOK" -> Book(record.id, record.title, record.authors, record.publisher, record
.publicationYear, record.totalCopies, record.availableCopies, record.branchId, recor
d.isbn)
    "PERIODICAL" -> Periodical(record.id, record.title, record.authors, record.publi
sher, record.publicationYear, record.totalCopies, record.availableCopies, record.bra
nchId, record.issueNumber ?: 1)
    else -> error("Неизвестный тип издания: ${record.kind}")
}
private fun Loan.toRecord() = LoanRecord(id, readerId, publicationId, librarianId, i
ssuedAt, dueAt, (state as? LoanState.Returned)?.returnedAt, state.status)
private fun loanRecordToDomain(record: LoanRecord): Loan {
    val state = when (record.status) {
        "ACTIVE" -> LoanState.Active
        "OVERDUE" -> LoanState.Overdue
        "RETURNED" -> LoanState.Returned(record.returnedAt ?: error("У возвращённой
выдачи нет даты возврата"))
        else -> error("Неизвестный статус выдачи: ${record.status}")
    }
    return Loan(record.id, record.readerId, record.publicationId, record.librarianId
, record.issuedAt, record.dueAt, state)
}
```

core/src/jvmMain/kotlin/persistence/DatabaseJvm.kt

```
package persistence

import androidx.room3.Room
import androidx.room3.RoomDatabase
import java.io.File

fun databaseBuilder(databaseFile: File): RoomDatabase.Builder<LibraryDatabase> =
    Room.databaseBuilder<LibraryDatabase>(name = databaseFile.absolutePath)

fun defaultDatabaseFile(): File {
    val home = System.getProperty("user.home")
    val os = System.getProperty("os.name").lowercase()
    val path = when {
        os.contains("mac") -> File(home, "Library/Application Support/CourseOopLibra
ry/library.db")
        os.contains("win") -> File(
            System.getenv("LOCALAPPDATA") ?: File(home, "AppData/Local").absolutePat
h,
            "CourseOopLibrary/library.db",
        )
        else -> File(home, ".local/share/course-oop-library/library.db")
    }
    path.parentFile.mkdirs()
    return path
}

fun openDefaultDatabase(): LibraryDatabase = buildDatabase(databaseBuilder(defaultDa
tabaseFile()))
```